

计算机视觉 life 平台介绍

计算机视觉 life 是国内较早分享「计算机视觉」技术的原创平台，公众号订阅用户 **11 万+**。同时有**微信视频号**、**哔哩哔哩**、**知乎**、**今日头条** 同名账户，全网订阅用户 **26 万+**。在行业内积累了极高的知名度，受到不少从业者的好评和推荐。

平台主要面向**智能机器人、自动驾驶汽车、无人机、增强现实、计算摄影、视觉/激光/惯性传感器、计算机视觉**等方向提供前沿技术分享、系统教学课程。

主要业务如下：

1、 AI 职业教育

面向工业实战的源码解析课程，官网 cvlife.net

- 视觉 **SLAM**：ORB-SLAM2（必学基础）
- 视觉惯性 **SLAM**：ORB-SLAM3，VINS-Mono+Fusion
- 激光/多传感器融合 **SLAM**：Cartographer，LeGO-LOAM、LIO-SAM、LVI-SAM
- 三维重建：视觉几何 OpenMVS，深度学习重建 MVSNet、PatchMatchNet、JDACS-MS；神经辐射场 NeRF 重建
- 机器人运动规划：Dijkstra、A-Star、DWA、TEB
- 相机标定：单目/鱼眼/双目/阵列
- 视觉深度估计：单目/双目深度估计
- C++编程入门
- 计算摄影（敬请期待）
- 深度学习 **SLAM**（敬请期待）
- 基于滤波的 **SLAM**（敬请期待）



扫描开始学习 ↑

2、 机器人 SLAM 开发者社区

我们建立了全国最大的机器人 SLAM 开发者交流社区，2018 年开始运营至今，已经有 3900+会员、6500+主题分享、9500+问答评论、130+教学视频。

- **领域**：机器人 SLAM、三维视觉、自动驾驶、增强现实、无人机、图像处理
- **直播**：每月 6/16/26 日 固定直播，涵盖大佬分享、学习经验、求职面试、实习历程、行业内幕
- **教程**：图文视频教程：涉及代码调试、OpenCV、PCL、G2O、Ceres、视觉 SLAM 十四讲、LVISAM、R3LIVE、Fastlio2、GVINS、机器人学中的状态估计、语义 SLAM 论文阅读 ……
- **答疑**：每日星主答疑、嘉宾答疑、星友互助答疑，搜索关键字几乎所有问题都能找到答案
- **资讯**：每日论文翻译、行业资讯汇总、每周汇总、精华汇总
- **活动**：学习小组、行业资源对接、会员激励、有偿招募助教/兼职
- **求职**：经验分享、内推职位、SLAM 面试题、笔试练习



扫描领取优惠券 ↑

招聘、产品宣传等商务合作，联系 simiter@126.com

Cartographer 从入门到精通：原理 深剖+源码逐行讲解【课件】

目录

第一章 论文带读、编译运行及调参指导	4
1.1 【开课直播】Cartographer 论文带读	4
1.2 代码的编译与运行	4
1.3 如何配置 Launch 与 Lua 文件	7
1.4 常见运行错误汇总与解决	9
1.5 参数的详解与调参总结	11
第二章 Cartographer_ros 代码入门	13
2.1 VScode 的配置与使用	13
2.2 Cartographer_ros 的功能与框架分析	13
看视频的注意事项	14
2.3 CMakeLists.txt 文件讲解	14
2.4 Main 函数使用的依赖库简介	14
2.5 Main 函数讲解	16
2.6 配置文件的加载	16
第三章 Cartographer_ros 代码进阶	16
3.1 开始轨迹的相关处理	18
3.2 数据采样器	18
3.3 订阅话题与注册回调函数	19
3.4 传感器数据的走向与传递	19
4.1 三维刚体坐标变换的理论与实现	20
4.2 IMU 数据、里程计数据与 Landmark 数据的格式转换与坐标变换	21
4.3 GPS 数据的处理与编程实践	21
4.4 雷达数据的格式转换与坐标变换	22
4.5 Cartographer_ros 中的传感器数据的传递过程总结	23
第五章 传感器数据的分发处理	24
5.1 MapBuilder 的声明与构造	24
5.2 开始轨迹时的处理	26
5.3 传感器数据走向分析	27
5.4 传感器数据分发器的创建	31
5.5 阻塞队列与生产者消费者模式	35
5.6 传感器数据的分发处理过程总结	35
第六章 点云数据的预处理	35
传感器数据的走向	35
6.1 GlobalTrajectoryBuilder 类讲解	35
6.2 多个激光雷达数据的时间同步与融合	36
6.3 点云数据的运动畸变的校正	36
6.4 点云的坐标变换与 z 轴的过滤	37
6.5 体素滤波与之后的处理	39

6.6 3D 情况下的激光雷达数据的预处理	40
第七章 初始位姿估计	40
7.1 代码总体调用流程分析	40
详见附图	40
7.2 基于 IMU 与里程计的位姿推测器	40
7.3 位姿推测器的优缺点分析与总结	41
第八章 概率栅格地图	43
8.1 概率栅格地图简介	43
8.2 地图的坐标系与原点	44
8.3 栅格地图中栅格值的更新	47
8.4 Grid2D 类讲解	48
8.5 ProbabilityGrid 类讲解	49
8.6 将点云数据写入到栅格地图中	49
8.7 贝汉明算法	49
8.8 栅格地图的总结	50
第九章 Ceres 实现 2D 扫描匹配	50
9.1 扫描匹配的概念与公式推导	50
9.2 实时相关性扫描匹配	52
9.3 Ceres 简介	52
9.4 Ceres 编程实践	54
9.5 基于优化的扫描匹配	54
9.6 前端扫描匹配的总结	55
第十章 2D 后端优化（上）	55
10.1 后端优化的概念与理论过程分析	55
10.2 后端优化整体函数调用流程分析	59
10.3 添加节点相关函数	59
10.4 子图的保存与任务处理函数	62
10.5 线程池与任务调度	63
第十一章 2D 后端优化（中）	63
11.1 子图内约束的计算	64
11.2 子图间约束的计算(回环检测)	66
11.3 使用滑动窗口算法生成多分辨率栅格地图	66
11.4 滑动窗口算法编程实践	66
11.5 基于多分辨率地图的分支定界粗匹配	66
第十二章 2D 后端优化（下）	67
12.1 优化时的函数调用流程分析	67
12.2 后端计算位姿的方法总结	67
12.3 第一种残差的构建	70
12.4 第二种残差的构建-Landmark 残差	70
12.5 其余三种残差的构建	72
12.6 使用 Ceres 进行全局优化求解	73
12.7 后端优化部分的总结	73
第十三章 SLAM 的结果输出	75
13.1 优化后结果的传递与处理	75
13.2 TF 的发布	77
13.3 轨迹与约束的可视化	77
13.4 ROS 地图的发布	77
13.5 结束轨迹时的处理	77
13.6 Pbstream 文件的保存	77

第十四章 纯定位模式与 3D 建图.....	78
14.1 纯定位模式函数调用流程分析.....	78
14.2 Pbstream 文件的加载.....	80
14.3 子图的修剪.....	80
14.4 TSDF 地图简介.....	80
14.5 3D 网格地图.....	83
14.6 3D 扫描匹配.....	83
14.7 3D 后端优化.....	84
第十五章 总结与工程化建议.....	85
15.1 代码的全面总结.....	85
15.2 Cartographer 的优缺点分析.....	85
15.3 工程化建议.....	87

该内容是公众号 计算机视觉 life 旗下课程《[Cartographer 从入门到精通: 原理深剖+源码逐行详解](#)》的课件编纂成的课程手册。持续更新。课程官网：[cvlife.net](#)

作者：李想，高级激光 SLAM 算法工程师，参与多个 AVG 项目落地，有丰富的项目实战经验。

修订时间：2022 年 8 月 16 日

扫描一下，或登录 [cvlife.net](#) 系统学习视觉/激光/多传感器融合 SLAM、三维重建等精品课程：



扫描学习SLAM、三维重建

关于课程及交流群：如果想要咨询、购买计算机视觉 life 课程，请添加下面微信，备注：“**咨询或已购课程**”。请按照格式备注，否则不予通过。



第一章 论文带读、编译运行及调参指导

1.1 【开课直播】Cartographer 论文带读

1.2 代码的编译与运行

官方文档

官方文档的地址为

- <https://google-cartographer-ros.readthedocs.io/en/latest/compilation.html>
- <https://google-cartographer.readthedocs.io/en/latest/index.html>

cartographer 的编译

系统环境

- 推荐在 ubuntu16.04 或者 18.04 上进行编译
- 推荐使用刚装好的 ubuntu 系统
- 将 ubuntu 的软件源设置成清华的或者其他的中国境内的源, 更换完源之后要执行一下 `sudo apt-get update`
- 照着 wiki.ros.org 安装完对应版本的 **ROS**
- 并在 `.bashrc` 中的末尾添加这条语句, 要把 `<distro>` 改成对应的 ROS 版本
`source /opt/ros/<distro>/setup.bash`
- 其中 `sudo rosdep init; rosdep update` 这条语句执行不成功, 不管他, 不是必须执行的

依赖库的安装

cartographer 的依赖库脚本完成一键安装.

如下是具体步骤:

第一步: 下载安装包

课程的所有资料都放在百度网盘中, 链接为

链接: <https://pan.baidu.com/s/1q9f5aNTfrbpFpwGCwD6E4A>

提取码: **slam**

在网盘链接里 安装包文件夹 中下载 `cartographer_install-20210613.zip`, 放到 ubuntu 系统里并解压

第二步: 打开终端

进入到 `cartographer_install` 文件夹, 在文件夹内部空白处单击鼠标右键, 打开终端

第三步 使用脚本完成依赖项的安装

在终端中输入 `./auto-carto-build.sh`

如果提示脚本不是一个可执行的文件, 那就通过 `chmod +x auto-carto-build.sh` 为这个脚本添加可执行权限即可, 之后在通过 `./auto-carto-build.sh` 执行脚本.

解释说明

脚本里进行了 4 个库的编译与安装

`cartographer` 的依赖库有 3 个, 分别是 `abseil-cpp`, `ceres-solver`, `protobuf`.

在脚本的最后, 也将 `cartographer` 安装到系统里了. 安装 `cartographer` 后, 就可以在工作空间中只对 `cartographer_ros` 进行编译了, 这样修改 `launch` 和 `lua` 文件后就不需要执行一次编译指令了.

注意: 如果之前安装过 `ceres-solver`, `protobuf`, `cartographer` 的话, 执行脚本时可能会报有些函数没有定义的错误, 这是由于 `protobuf` 或者 `cartographer` 不一致版本导致的, 可以先将之前安装的库卸载掉, 再执行安装依赖的脚本

一定要确保脚本执行之后没有错误输出, 否则之后编译代码会编译不通过.

cartographer 的编译

第一步: 下载注释的代码

讲师注释的代码的 github 地址为

https://github.com/xiangli0608/cartographer_detailed_comments_ws

在某个文件夹内部空白处单击鼠标右键, 打开终端, 输入如下执行进行代码的下载.

`git clone`

`https://github.com/xiangli0608/cartographer_detailed_comments_ws.git`

注意, 一定要通过 `git clone` 的方式下载代码, 因为代码的注释是处于更新状态的, 要定期通过 `git pull origin master` 来更新代码.

要是在 github 中下载代码的 zip 压缩文件, 这样下载的代码里不含有 `.git` 文件夹, 就不能通过 `git pull` 进行代码的更新了.

如果执行 `git clone` 后没有下载代码, 是由于没有连上 github 导致的, 可以按 `ctrl c` 中断当前下载, 再次执行 `git clone`, 重复几次试试, 如果还不行, 就在群里问问吧.

第二步: `cartographer` 的编译

进入到 `cartographer_detailed_comments_ws` 内, 在文件夹内打开终端, 输入

`./catkin_make.sh`

完成对 `cartographer` 与 `cartographer_ros` 的编译.

注意：一定要确保之前的依赖项全部安装成功，否则这里的编译就会报错。

cartographer 的运行

下载数据集

在网盘链接里 数据集文件夹 中下载数据集，使用的数据集有 3 个，

- `rslidar-outdoor-gps.zip` 这个是讲师自己录制的数据，包括了速腾 16 线点云数据，由点云数据生成的单线雷达数据，imu，里程计，rtk/gps，tf 与 tf_static
- `rslidar-outdoor-gps-notf.zip` 这个与上边的是同一个，只不过这个将 tf 的话题去掉了
- `landmarks_demo_uncalibrated.zip` 是从 cartographer 官网下载的数据，旨在展示一下如何在 cartographer 中使用 landmark

将 bag 文件放到指定的文件夹下

首先，在用户文件夹下新建一个文件夹，并命名为 **bagfiles**，然后将下载好的 3 个 bag 文件放入 bagfiles 文件夹中。

开始二维建图

进入到 `cartographer_detailed_comments_ws` 文件夹内，在文件夹内右键打开终端，输入

```
Plaintext
source install_isolated/setup.bash
rospack profile
roslaunch cartographer_ros lx_rs16_2d_outdoor.launch
```

source 命令的目的

source 命令是设置当前的终端 可以执行的包与节点的地址集合的，不执行 `source install_isolated/setup.bash` 这个命令，直接启动 `roslaunch` 时就会出现 找不到包或找不到节点的错误提示。

为了防止每次启动终端都要执行一次 `source install_isolated/setup.bash` 这个指令，我们可以将这个指令写到 `~/.bashrc` 文件中，每次启动终端时，会自动 source `~/.bashrc` 这个文件，这个文件中的所有内容都会被 source 一遍。

~ 代表用户文件夹，例如我自己电脑的用户名为 lx，我自己的用户文件夹就是 `/home/lx/`，这个文件夹地址也可以用 ~ 来代替。

按 `ctrl h` 可以显示隐藏文件，找到 `.bashrc` 文件进行编辑，在文件底部输入

```
source ~/path-to-
carto/cartographer_detailed_comments_ws/install_isolated/setup.bash
```

其中 `path-to-carto` 要改成自己实际的文件夹名字。

rospack profile 命令的目的

新下载的包在编译之后, 有时没有被加载到 ros 的包的地址集合中,.

这时即使执行 `source` 指令, 在启动 `roslaunch` 时也可能报错, 提示找不到包或者可执行节点.

通过 `rospack profile` 命令可以将新下载的包加载到 ros 的包的地址集合, 再执行 `roslaunch` 时就不会再报错了.

保存 2d 轨迹, 并生成 ros 格式的地图

```
./finish_slam_2d.sh
```

其中 `map_dir` 是保存地图的文件夹名字, 可以改成自己想要保存的文件夹地址.

纯定位模式

```
roslaunch cartographer_ros lx_rs16_2d_outdoor_localization.launch
```

3d 建图指令

```
roslaunch cartographer_ros lx_rs16_3d.launch
```

保存 3d 轨迹

```
./finish_slam_3d.sh
```

使用 asset 生成 ros 格式的 2d 栅格地图

```
roslaunch cartographer_ros assets_writer_2d.launch
```

使用 asset 生成 3d 点云地图

```
roslaunch cartographer_ros assets_writer_3d.launch
```

landmark 使用示例

```
roslaunch cartographer_ros landmark_mir_100.launch
```

1.3 如何配置 Launch 与 Lua 文件

第一步 了解 bag 文件

播放 bag 文件需要在 **bag** 的文件夹内启动五个终端

- 第一个终端执行 `roscore`
- 第二个终端执行 `rosbag info rslidar-outdoor-gps.bag`
了解 bag 中 topic 的名称与类型
- 第三个执行 `rosbag play rslidar-outdoor-gps.bag`
- 第四个终端执行 `rqt`
- 了解 bag 中的 tf 树.

- 注意, rqt 的所有显示插件都在菜单栏的 `plugins` 内, 再点击 `visualization`, 再点击 `tf tree`, 即可显示视频中的 `tf` 树的可视化界面.
- 第五个终端执行 `rviz`
- 可视化雷达数据与 `tf` 数据.
- `rviz` 左侧最顶上的 `Fixed frame` 除了下拉菜单可以选择之外, 还是可以手动输入的, 现在手动输入成 `odom` ,
- `rviz` 左侧下方的 `Add` 按钮, 可以添加显示的插件, 自己选择添加 `pointcloud2`, `laserscan` 等格式的数据, 之后把插件订阅的 `topic` 选择一下, 如果不会就自己摸索一下, 很简单的.

第二步 配置 launch 文件

launch 文件中需要如下几个设置

- bag 文件的地址与 bag 文件的名字
- lua 文件的名字
- topic 需要 remap 成 bag 文件中发布的 topic

第三步 配置 lua 文件

- `tracking_frame` 有 imu 的 link 就设置成 imu 的 link, 没有就设置成 `base_link`
- `published_frame` cartographer 发布的 `tf` 的最下边一个坐标系, 就是 bag 文件中 `tf` 树的最上边的一个坐标系
- `provide_odom_frame` 是否提供里程计, 如果 bag 中有里程计的坐标系, 这个就是 `false`, 如果没有, 就根据需要决定是否提供里程计的坐标系
- `use_pose_extrapolator = false` 这个一定要设置成 `false`
- `use_odometry` 是否使用里程计的传感器数据, 如果为 `true`, `tf` 树中一定要存在 `odom` 这个坐标系
- `use_nav_sat/use_landmarks` 是否订阅里程计话题的数据, 以及 `landmark` 话题的数据
- `num_laser_scans/num_point_clouds` 单线点云与多线点云的话题的数量, 可以同时为 1, 不可以同时为 0
- `num_subdivisions_per_laser_scan` 一帧点云数据会被分成几次处理, 设置成 1 就行了
- `MAP_BUILDER.use_trajectory_builder_2d = true` 建二维图时一定要有这句话, 建三维图就把 2d 改成 3d

- TRAJECTORY_BUILDER_2D.use_imu_data 是否使用 imu, 如果用 imu, tracking_frame 一定要设置成 imu 的 link
- TRAJECTORY_BUILDER_2D.min_z 点云的最小 z 的范围, 单线点云不能设置大于 0 的值(不设置), 多线点云的这个值要大于 0

1.4 常见运行错误汇总与解决

bag 文件的地址或名称不对

当 bag 文件的地址没设置对时, 会产生如下错误

```

resolver.cc:53] Found '/home/lx/carto_ws/cartographer_detailed_comments_ws/install_isolated/share/cartographer/configuration_files/trajectory_builder_3d.lua' for
'trajectory_builder_3d.lua'.
[ INFO] [1644379929.682303277]: I0209 12:12:09.000000 26649 configuration file r
resolver.cc:53] Found '/home/lx/carto_ws/cartographer_detailed_comments_ws/install_isolated/share/cartographer/configuration_files/trajectory_builder_3d.lua' for
'trajectory_builder_3d.lua'.
[ INFO] [1644379929.692151035]: I0209 12:12:09.000000 26649 map_builder_bridge.c
c:156] Added trajectory with ID '0'.
[playbag-5] process has died [pid 26670, exit code 1, cmd /opt/ros/kinetic/lib/rosbag/play --clock /home/lx/bagfile/rslidar-outdoor-gps.bag __name:=playbag __log:=/home/lx/.ros/log/72efb5e4-895e-11ec-9b29-44032c71fb4c/playbag-5.log].
log file: /home/lx/.ros/log/72efb5e4-895e-11ec-9b29-44032c71fb4c/playbag-5*.log
[ WARN] [1644379932.700729413]: W0209 12:12:12.000000 26649 node.cc:1264] Expected topic "points2" (trajectory 0) (resolved topic "/rslidar_points") but no publisher is currently active.
[ WARN] [1644379932.700812125]: W0209 12:12:12.000000 26649 node.cc:1264] Expected topic "imu" (trajectory 0) (resolved topic "/imu") but no publisher is currently active.
[ WARN] [1644379932.700870009]: W0209 12:12:12.000000 26649 node.cc:1274] Currently available topics are: /move_base_simple/goal,/constraint_list,/submap_list,/scan_matched_points2,/rosout,/tf,/clicked_point,/initialpose,/rosout_agg,/map,/trajectory_node_list,/landmark_pose_list

```

可以看到, 是[playbag-5]出错了. 原因可以看到, 图片中 bag 的地址是 /home/lx/bagfile/rslidar-outdoor-gps.bag, 但是实际的文件夹地址是 bagfiles, 单词差了一个 s, 找不到文件, 所以报错了.

launch 订阅话题的名字

当 launch 文件中 topic 名字没设置对, 会在运行时产生如下错误


```
[ INFO] [1644380207.318822374]: I0209 12:16:47.000000 28473 configuration_file_resolver.cc:53] Found '/home/lx/carto_ws/cartographer_detailed_comments_ws/install_isolated/share/cartographer/configuration_files/trajectory_builder_3d.lua' for 'trajectory_builder_3d.lua'.
[ INFO] [1644380207.318879312]: I0209 12:16:47.000000 28473 configuration_file_resolver.cc:53] Found '/home/lx/carto_ws/cartographer_detailed_comments_ws/install_isolated/share/cartographer/configuration_files/trajectory_builder_3d.lua' for 'trajectory_builder_3d.lua'.
[ INFO] [1644380207.328950074]: I0209 12:16:47.000000 28473 map_builder_bridge.cc:156] Added trajectory with ID '0'.
[ WARN] [1644380210.338268121, 1606808652.135240403]: W0209 12:16:50.000000 28473 node.cc:1264] Expected topic "points2" (trajectory 0) (resolved topic "/rslidar_point") but no publisher is currently active.
[ WARN] [1644380210.338600884, 1606808652.135240403]: W0209 12:16:50.000000 28473 node.cc:1274] Currently available topics are: /front_scan,/fix,/constraint_list,/submap_list,/scan_matched_points2,/heading,/rosout,/initialpose,/tf,/move_base_simple/goal,/tf_static,/clock,/imu,/odom_scout,/rosout_agg,/map,/clicked_point,/trajectory_node_list,/rslidar_points,/landmark_poses_list,
[ WARN] [1644380212.681575765, 1606808654.478415036]: W0209 12:16:52.000000 28473 ordered_multi_queue.cc:229] Queue waiting for data: (0, points2)
[ WARN] [1644380213.289483279, 1606808655.095254181]: W0209 12:16:53.000000 28473 ordered_multi_queue.cc:229] Queue waiting for data: (0, points2)
[ WARN] [1644380213.882174184, 1606808655.681479355]: W0209 12:16:53.000000 28473 ordered_multi_queue.cc:229] Queue waiting for data: (0, points2)
```

图片中显示, 想要订阅 `rslidar_point` 这个 topic, 但是没订阅到, 所以报错了, 同时 `carto` 还将当前能够订阅到的所有 topic 的名字打印出来了, 其中有 `rslidar_points` 这个 topic, 可见, 应该是我的 launch 文件中 `remap` 时打错了。

坐标系对不上

lua 文件中坐标系设置的不对, 导致 tf 树连不起来, 会在运行时产生 `passed to lookupTransform argument source_frame does not exist` 这种问题。

bag 的最上边一个坐标系是 `odom`, 我在 lua 里的 `published_frame` 设置的是 `odom2`, 就会产生如下错误。

```
[ INFO] [1644381701.023494575]: I0209 12:41:41.000000 29477 map_builder_bridge.cc:156] Added trajectory with ID '0'.
[ INFO] [1644381701.419739524, 1606808649.540660002]: I0209 12:41:41.000000 29477 ordered_multi_queue.cc:265] All sensor data for trajectory 0 is available starting at '637424054495384530'
[ INFO] [1644381701.486082016, 1606808649.611257655]: I0209 12:41:41.000000 29477 pose_graph_2d.cc:212] Inserted submap (0, 0).
[ WARN] [1644381701.686929187, 1606808649.812591569]: W0209 12:41:41.000000 29477 tf_bridge.cc:55] "odom2" passed to lookupTransform argument source_frame does not exist.
[ WARN] [1644381701.917529182, 1606808650.044719478]: W0209 12:41:41.000000 29477 tf_bridge.cc:55] "odom2" passed to lookupTransform argument source_frame does not exist.
[ WARN] [1644381702.149598670, 1606808650.276911390]: W0209 12:41:42.000000 29477 tf_bridge.cc:55] "odom2" passed to lookupTransform argument source_frame does not exist.
[ WARN] [1644381702.391359148, 1606808650.518924169]: W0209 12:41:42.000000 29477 tf_bridge.cc:55] "odom2" passed to lookupTransform argument source_frame does not exist.
[ WARN] [1644381702.653686232, 1606808650.781340801]: W0209 12:41:42.000000 29477 tf_bridge.cc:55] "odom2" passed to lookupTransform argument source_frame does not exist.
[ WARN] [1644381702.906033375, 1606808651.033620336]: W0209 12:41:42.000000 29477 tf_bridge.cc:55] "odom2" passed to lookupTransform argument source_frame does not exist.
```

图中显示, odom2 这个坐标系与 tracking_frame 的坐标系找不到连接关系, 所以报错.

所以 published_frame 的坐标系一定要是 bag 中或者是机器人上的最顶上的一个坐标系, 要使得 tf 树连起来.

订阅单线点云时设置了 min_z

当我只订阅单线点云, 同时将 min_z 设置成了 0.8, 这时启动建图就会报错

```
[ WARN] [1644382101.983210421, 1606808649.852765685]: W0209 12:48:21.000000 3198
0 local_trajectory_builder_2d.cc:300] Dropped empty horizontal range data.
[ WARN] [1644382102.034195666, 1606808649.903259830]: W0209 12:48:22.000000 3198
0 local_trajectory_builder_2d.cc:300] Dropped empty horizontal range data.
[ WARN] [1644382102.085811123, 1606808649.963667797]: W0209 12:48:22.000000 3198
0 local_trajectory_builder_2d.cc:300] Dropped empty horizontal range data.
[ WARN] [1644382102.138527665, 1606808650.014174489]: W0209 12:48:22.000000 3198
0 local_trajectory_builder_2d.cc:300] Dropped empty horizontal range data.
[ WARN] [1644382102.189035263, 1606808650.064645936]: W0209 12:48:22.000000 3198
0 local_trajectory_builder_2d.cc:300] Dropped empty horizontal range data.
[ WARN] [1644382102.242094697, 1606808650.115152261]: W0209 12:48:22.000000 3198
0 local_trajectory_builder_2d.cc:300] Dropped empty horizontal range data.
[ WARN] [1644382102.294921266, 1606808650.165585889]: W0209 12:48:22.000000 3198
0 local_trajectory_builder_2d.cc:300] Dropped empty horizontal range data.
[ WARN] [1644382102.345448859, 1606808650.216044389]: W0209 12:48:22.000000 3198
0 local_trajectory_builder_2d.cc:300] Dropped empty horizontal range data.
[ INFO] [1644382102.403596183, 1606808650.276691176]: I0209 12:48:22.000000 3198
0 pose_graph_2d.cc:212] Inserted submap (0, 0).
[ WARN] [1644382102.452991628, 1606808650.327122076]: W0209 12:48:22.000000 3198
0 local_trajectory_builder_2d.cc:300] Dropped empty horizontal range data.
```

所以, 订阅单线点云时不要设置 min_z 这个参数.

odom 坐标系发布重复

当 bag 或者机器人中本身存在 odom 坐标系, 而 provide_odom_frame 又设置成了 true, 就会导致 odom 坐标系重复发布.

会导致机器人位姿发生来回的跳动.

1.5 参数的详解与调参总结

参数的详解

详见视频.

调参总结

详见附图

常调的参数

Lua

```
TRAJECTORY_BUILDER_2D.min_range = 0.3
TRAJECTORY_BUILDER_2D.max_range = 100.
TRAJECTORY_BUILDER_2D.min_z = 0.2 -- / -0.8
TRAJECTORY_BUILDER_2D.voxel_filter_size = 0.02

TRAJECTORY_BUILDER_2D.ceres_scan_matcher.occupied_space_weight =
10.
TRAJECTORY_BUILDER_2D.ceres_scan_matcher.translation_weight = 1.
TRAJECTORY_BUILDER_2D.ceres_scan_matcher.rotation_weight = 1.

TRAJECTORY_BUILDER_2D.submaps.num_range_data = 80.
TRAJECTORY_BUILDER_2D.submaps.grid_options_2d.resolution = 0.1 -- /
0.02

POSE_GRAPH.optimize_every_n_nodes = 160. -- 2 倍的 num_range_data 以
上
POSE_GRAPH.constraint_builder.sampling_ratio = 0.3
POSE_GRAPH.constraint_builder.max_constraint_distance = 15.
POSE_GRAPH.constraint_builder.min_score = 0.48
POSE_GRAPH.constraint_builder.global_localization_min_score = 0.60
```

降低延迟与减小计算量

前端部分

- 减小 max_range, 减小了需要处理的点数, 在雷达数据远距离的点不准时一定要减小这个值
- 增大 voxel_filter_size, 相当于减小了需要处理的点数
- 增大 submaps.resolution, 相当于减小了匹配时的搜索量
- 对于自适应体素滤波 减小 min_num_points 与 max_range, 增大 max_length, 相当于减小了需要处理的点数

后端部分

- 增大 optimize_every_n_nodes, 降低优化频率, 减小了计算量
- 增大 MAP_BUILDER.num_background_threads, 增加计算速度
- 减小 global_sampling_ratio, 减小计算全局约束的频率
- 减小 constraint_builder.sampling_ratio, 减少了约束的数量
- 增大 constraint_builder.min_score, 减少了约束的数量

- 减小分枝定界搜索窗的大小, 包括
linear_xy_search_window, linear_z_search_window, angular_search_window
- 增大 global_constraint_search_after_n_seconds, 减小计算全局约束的频率
- 减小 max_num_iterations, 减小迭代次数

降低内存

增大子图的分辨率 submaps.resolution

降低纯定位时错误重定位的概率

修改 pose_graph.lua 中的如下参数

- 提高 optimize_every_n_nodes, 减小了优化的次数
- 减小 sampling_ratio, 减小了计算约束的次数
- 减小 max_constraint_distance, 减小了计算约束的距离
- 提高 min_score, 减小了计算约束的数量, 提高约束正确的概率
- 提高 global_localization_min_score, 减小了计算约束的数量, 提高约束正确的概率
- 提高 global_constraint_search_after_n_seconds, 减小了回环的次数(隔多长时间计算一次)

第二章 Cartographer_ros 代码入门

2.1 VScode 的配置与使用

详见视频

2.2 Cartographer_ros 的功能与框架分析

cartographer_ros 是 cartographer 算法库针对 ROS 开发的接口库.

详见附图

主要的作用是

- 通过 ros 的订阅机制接收传感器数据, 并将传感器数据做坐标变换以及数据类型的转换, 转换完成之后送到 cartographer 算法库中进行具体的 SLAM 的计算
- 根据前端与后端计算的结果, 发布轨迹, tf, 栅格地图, 约束信息, 以及 landmark
- 提供了外部控制的接口, 如开始结束轨迹, 保存数据, 获取数据等

cartographer_ros 涉及到的文件或者类

- node_main.cc: 主函数 - 第二章
- Node 类 - 第三章
- MapBuilderBridge 类 - 第三章
- SensorBridge 类 - 第四章
- TfBridge 类 - 第四章
- msg_conversion.cc: 包含数据类型转换的相关函数 - 第四章

看视频的注意事项

- 看视频不等于看懂代码: 看视频只能大致听懂代码, 不是自己真正理解代码, 光看视频很容易看完就忘
- 要有自己的思考过程: 自己读代码 / 自己手动注释代码 / 写技术博客或者笔记
- 尽量不要跳着看视频, 有时跳着看会错过重要知识点, 可以 1.25/2.0 倍速观看
- 可以只听视频, 自己看代码
- 每次听代码讲解的视频之前, 要先将代码更新一下

2.3 CMakeLists.txt 文件讲解

听不懂没关系, 不影响 cartographer 的学习

2.4 Main 函数使用的依赖库简介

Todo Tree 说明

- **?**: 目前我不理解的地方的标注
- **note**: 重点库, 重点概念, 重点功能的语句的标注
- **c++11**: c++11 新标准与 c++不常用语法的标注
- **Step**: 代码步骤说明

gflag 简介

gflags 是 google 开源的命令行标记处理库

命令行标记,顾名思义就是当运行一个可执行文件时, 由用户为其指定的标记, 形如:

```
fgrep -l -f ./test ccc jjj
```

-l 与 -f 是命令行标记, 而 ccc 与 jjj 是命令行参数, 因为这两者不是以 - 开头的.

一般的一个可执行文件, 允许用户为其传入命令行标记以及参数.

如上述例子, -l 是一个不带参数的标记, -f 是一个带了参数 ./test 的标记, 而 gflags 可以解析这些标记以及参数并将其存储在某些数据结构中.

gflags 主要支持的参数类型包括 bool, int32, int64, uint64, double, string 等

定义参数通过 DEFINE_type 宏实现, 该宏的三个参数含义分别为命令行参数名, 参数默认值, 以及参数的帮助信息

当参数被定义后, 通过 **FLAGS_name** 就可访问到对应的参数

引用于 <https://zhuanlan.zhihu.com/p/108477489>

glog 简介

Google glog 是一个应用级别的日志系统库. 它提供基于 C++ 风格的流和各种辅助宏的日志 API. 支持以下功能:

- 参数设置, 以命令行参数的方式设置标志参数来控制日志记录行为
- 严重性分级, 根据日志严重性分级记录日志 - INFO WARNING ERROR FATAL
- 可有条件地记录日志信息 - LOG_IF LOG_EVERY_N LOG_IF_EVERY_N LOG_FIRST_N
- 条件中止程序. 丰富的条件判定宏, 可预设程序终止条件 - CHECK 宏
- 异常信号处理. 程序异常情况, 可自定义异常处理过程
- 支持 debug 功能
- 自定义日志信息
- 线程安全日志记录方式
- 系统级日志记录
- google perror 风格日志信息
- 精简日志字符串信息

最终的结果不仅会在屏幕终端显示出来, 同时会将 log 日志写入到/tmp/...log....这个文件中

glog 比较语句的源码

C++

```
#define CHECK_EQ(val1, val2) CHECK_OP(_EQ, ==, val1, val2)
#define CHECK_NE(val1, val2) CHECK_OP(_NE, !=, val1, val2)
#define CHECK_LE(val1, val2) CHECK_OP(_LE, <=, val1, val2)
#define CHECK_LT(val1, val2) CHECK_OP(_LT, < , val1, val2)
#define CHECK_GE(val1, val2) CHECK_OP(_GE, >=, val1, val2)
#define CHECK_GT(val1, val2) CHECK_OP(_GT, > , val1, val2)
```

2.5 Main 函数讲解

详见视频

2.6 配置文件的加载

详见视频

第三章 Cartographer_ros 代码进阶

函数指针

C++

```
int function(int a, int b)
{
    // 执行代码
}

int main(void)
{
    int (*FP)(int, int); // 函数指针的声明方式
    FP = function;        // 第一种赋值方法
    // FP = &function;    // 第二种赋值方法
    FP(1,2);              // 第一种调用方法
    // (*FP)(1,2);        // 第二种调用方法
    return 0;
}

class MyClass {
public:
    int Fun(int a, int b) {
        cout << "call Fun" << endl;
        return a + b;
    }
}
```

```
};

int main() {
    MyClass* obj = new MyClass;
    int (MyClass::*pFun)(int, int) = &MyClass::Fun;    // 成员函数指针的声明与初始化
    (obj->*pFun)(1, 2);
    // 通过对象来调用成员函数指针
    delete obj;
    return 0;
}
```

lambda 表达式

C++
 [函数对象参数 = &] (操作符重载函数参数) mutable 或 exception 声明 (-> 返回值类型) {函数体}

eg1
 [node, handler, trajectory_id, topic](const typename MessageType::ConstPtr& msg)
 {
 (node->*handler)(trajectory_id, topic, msg);
 }

eg2
 const Rigid3d tracking_to_local = [&] {
 // 是否将变换投影到平面上
 if
 (trajectory_data.trajectory_options.publish_frame_projected_to_2d) {
 return carto::transform::Embed3D(
 carto::transform::Project2D(tracking_to_local_3d));
 }
 return tracking_to_local_3d;
 }();

订阅话题的名字对不上时会提示这样的错误信息

```
Plaintext
[ INFO] [1625724722.715674522]: I0708 14:12:02.000000 20138
map_builder_bridge.cc:153] Added trajectory with ID '0'.
[ WARN] [1625724725.726943013]: W0708 14:12:05.000000 20138
```

```

node.cc:1267] Expected topic "points2" (trajectory 0) (resolved
topic "/points2") but no publisher is currently active.
[ WARN] [1625724725.727046746]: W0708 14:12:05.000000 20138
node.cc:1267] Expected topic "imu" (trajectory 0) (resolved topic
"/imu") but no publisher is currently active.
[ WARN] [1625724725.727145643]: W0708 14:12:05.000000 20138
node.cc:1280] Currently available topics are:
/constraint_list,/submap_list,/scan_matched_points2,/rosout,/tf,/clo
ck,/rosout_agg,/map,/trajectory_node_list,/landmark_poses_list,
[ WARN] [1625724739.386545399, 1606808654.481099623]: W0708
14:12:19.000000 20138 ordered_multi_queue.cc:232] Queue waiting for
data: (0, points2)
[ WARN] [1625724739.994835822, 1606808655.086546848]: W0708
14:12:19.000000 20138 ordered_multi_queue.cc:232] Queue waiting for
data: (0, points2)
[ WARN] [1625724740.588885518, 1606808655.682104741]: W0708
14:12:20.000000 20138 ordered_multi_queue.cc:232] Queue waiting for
data: (0, points2)
[ WARN] [1625724741.195557158, 1606808656.288400347]: W0708
14:12:21.000000 20138 ordered_multi_queue.cc:232] Queue waiting for
data: (0, points2)

```

错误信息截图展示如下

```

[ WARN] [1625724725.726943013]: W0708 14:12:05.000000 20138 node.cc:1267] Expect
ed topic "points2" (trajectory 0) (resolved topic "/points2") but no publisher i
s currently active.
[ WARN] [1625724725.727046746]: W0708 14:12:05.000000 20138 node.cc:1267] Expect
ed topic "imu" (trajectory 0) (resolved topic "/imu") but no publisher is curren
tly active.
[ WARN] [1625724725.727145643]: W0708 14:12:05.000000 20138 node.cc:1280] Curren
tly available topics are: /constraint_list,/submap_list,/scan_matched_points2,/r
osout,/tf,/clock,/rosout_agg,/map,/trajectory_node_list,/landmark_poses_list,
[ WARN] [1625724739.386545399, 1606808654.481099623]: W0708 14:12:19.000000 2013
8 ordered_multi_queue.cc:232] Queue waiting for data: (0, points2)
[ WARN] [1625724739.994835822, 1606808655.086546848]: W0708 14:12:19.000000 2013
8 ordered_multi_queue.cc:232] Queue waiting for data: (0, points2)
[ WARN] [1625724740.588885518, 1606808655.682104741]: W0708 14:12:20.000000 2013
8 ordered_multi_queue.cc:232] Queue waiting for data: (0, points2)
[ WARN] [1625724741.195557158, 1606808656.288400347]: W0708 14:12:21.000000 2013
8 ordered_multi_queue.cc:232] Queue waiting for data: (0, points2)
[ WARN] [1625724741.792187933, 1606808656.879707330]: W0708 14:12:21.000000 2013

```

3.1 开始轨迹的相关处理

详见视频

3.2 数据采样器

详见视频

3.3 订阅话题与注册回调函数

详见视频

3.4 传感器数据的走向与传递

传感器数据的回调函数

- HandleOdometryMessage
- HandleNavSatFixMessage
- HandleLandmarkMessage
- HandleImuMessage
- HandleLaserScanMessage
- HandleMultiEchoLaserScanMessage
- HandlePointCloud2Message

MapBuilderBridge

local frame 与 global frame

carographer 中存在两个地图坐标系, 分别为 global frame 与 local frame

local frame 是前端的坐标系, 这个坐标系与坐标系下的机器人的坐标, 一经扫描匹配之后就不会再被改变.

后端的坐标系, 回环检测与后端位姿图优化都不会对前端的任何坐标产生作用.

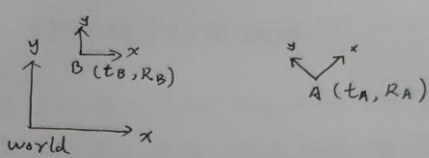
global frame 是后端的坐标系, 是表达被回环检测与位姿图优化所更改后的坐标系.

当优化之后, 这个坐标系下的节点坐标(包括点云的位姿, submap 的位姿)会发生变化.

但坐标系本身不会发生变化.

第四章 传感器的格式转换与类型变换

4.1 三维刚体坐标变换的理论实现



1. 假设 A 坐标系中有一点 P_A
 则 P_A 在 world 坐标系中的坐标为 P_W

$$P_W = R_A \cdot P_A + t_A, \quad P_A = R_A^{-1} (P_W - t_A)$$

$$P_A = \frac{R_A^{-1}}{R} \cdot P_W - \frac{R_A^{-1} \cdot t_A}{t}$$

2. 假设 W 坐标系下有一个坐标变换 B,
 则 B 作用在 A 坐标系后, A 的新坐标 A' 为

$$A': t' = R_A \cdot t_B + t_A$$

$$: K' = R_B \cdot R_A$$

$$P_W = T_{A,W} \cdot P_A$$

$$= \begin{bmatrix} R_A & t_A \\ 0 & 1 \end{bmatrix} \cdot P_A$$

$$T_{A,W}^{-1} = \begin{bmatrix} R_A^{-1} & -R_A^{-1} t_A \\ 0 & 1 \end{bmatrix}$$

机器人创新生态
ROBOTICS INNOVATION ECOSYSTEM

3. 假设 B 坐标系为 world 坐标系中的一个坐标系
 则 A 到 B 的坐标变换为

设 A 的原点为 P_A , P_A 在 world 下坐标为 P_{AW}
 设 B 的原点为 P_B , P_B 在 world 下坐标为 P_{BW}

① $P_{AW} = R_A \cdot P_A + t_A \quad P_{AW} = T_A \cdot P_A$
 ② $P_{BW} = R_B \cdot P_B + t_B$
 $\Downarrow$
 ③ $P_B = R_B^{-1} (P_{BW} - t_B) \quad P_B = T_B^{-1} \cdot P_{BW}$

将 ① 代入 ③ 中

$$P_{A,B} = R_B^{-1} (P_{AW} - t_B) \quad P_{A,B} = T_B^{-1} \cdot T_A \cdot P_A$$

$$= R_B^{-1} (R_A \cdot P_A + t_A - t_B)$$

$$= \underbrace{R_B^{-1} \cdot R_A}_R \cdot P_A + \underbrace{R_B^{-1} (t_A - t_B)}_t$$

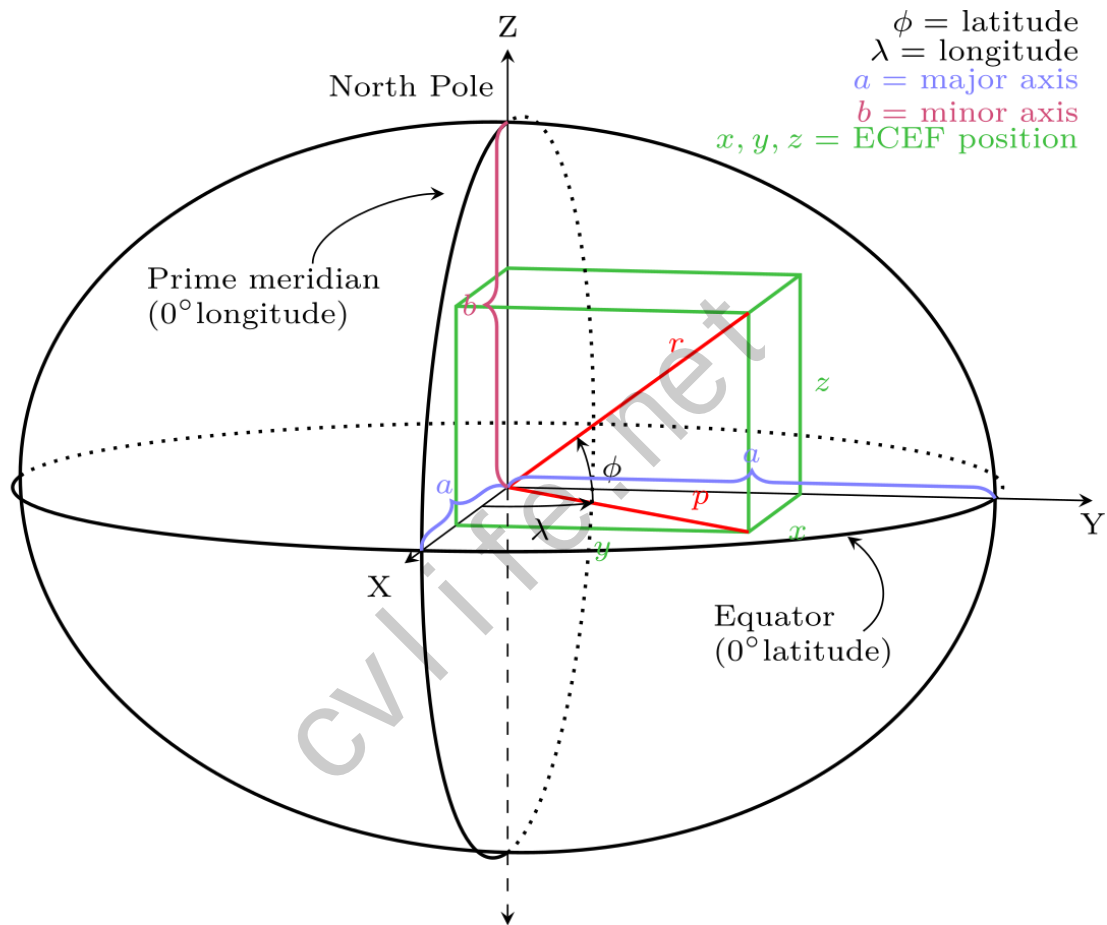
机器人创新生态
ROBOTICS INNOVATION ECOSYSTEM

4.2 IMU 数据、里程计数据与 Landmark 数据的格式转换与坐标变换

详见视频

4.3 GPS 数据的处理与编程实践

ECEF 坐标系



第一次编程作业

自己从头新建一个包, 实现视频里讲的 `ComputeLocalFrameFromLatLong()` 函数.

注意, 视频里的 `test_uint()` 函数被移动到后面进行讲解了, 所以视频里出现了这个函数先不管他.

4.4 雷达数据的格式转换与坐标变换

LaserScan 与 MultiEchoLaserScan 的数据类型

```
Plaintext
$ rosmmsg show sensor_msgs/LaserScan
std_msgs/Header header
  uint32 seq
  time stamp
  string frame_id
float32 angle_min
float32 angle_max
float32 angle_increment
float32 time_increment
float32 scan_time
float32 range_min
float32 range_max
float32[] ranges
float32[] intensities

$ rosmmsg show sensor_msgs/MultiEchoLaserScan
std_msgs/Header header
  uint32 seq
  time stamp
  string frame_id
float32 angle_min
float32 angle_max
float32 angle_increment
float32 time_increment
float32 scan_time
float32 range_min
float32 range_max
sensor_msgs/LaserEcho[] ranges
  float32[] echoes
sensor_msgs/LaserEcho[] intensities
  float32[] echoes
```

4.5 Cartographer_ros 中的传感器数据的传递过程总结

里程计与 IMU 数据的走向

里程计数据从 Node 类的回调函数进来, 有两个走向, 一个是 Node 类中的位姿估计器, 一个是 SensorBridge::HandleOdometryMessage, 再从 SensorBridge 传递给 cartographer.

IMU 数据的走向同理.

GPS 与 Landmark 数据的走向

GPS 数据从 Node 类的回调函数进来只有 1 个走向, 就是传入到 SensorBridge::HandleNavSatFixMessage()函数中, 再从 SensorBridge 传递给 cartographer.

Landmark 数据的走向同理.

雷达数据的走向

存在 3 中雷达数据, 一种是单线点云, 一种是多线点云, 一种是多回声波雷达点云, 都是直接传入到 SensorBridge 中.

单线点云与多回声波雷达点云都是先经过 ToPointCloudWithIntensities()函数, 处理成 carto 格式的点云, 然后在传入 HandleLaserScan()函数中.

根据参数配置, 将点云是否分段, 将分段的点云传入到 HandleRangeFinder()函数中.

多线点云是先经过 ToPointCloudWithIntensities()函数, 处理成 carto 格式的点云, 然后在传入 HandleRangeFinder()函数中.

HandleRangeFinder() 函数将点云点云从雷达坐标系下转到 tracking_frame 坐标系下, 并转成 TimedPointCloudData 格式的点云, 然后才传入到 cartographer 中.

第五章 传感器数据的分发处理

5.1 MapBuilder 的声明与构造

node_main.cc

```
C++
auto map_builder =
cartographer::mapping::CreateMapBuilder(node_options.map_builder_opt
ions);

Node node(node_options, std::move(map_builder), &tf_buffer,
FLAGS_collect_metrics);
```

Node 类的构造函数

```
C++
Node::Node(
    const NodeOptions& node_options,
    std::unique_ptr<cartographer::mapping::MapBuilderInterface>
map_builder,
    tf2_ros::Buffer* const tf_buffer, const bool collect_metrics)
    : node_options_(node_options),
      map_builder_bridge_(node_options_, std::move(map_builder),
tf_buffer)
{
    // Step: 1 声明需要发布的 topic
    // Step: 2 声明发布对应名字的 ROS 服务，并将服务的发布者放入到 vector 容
器中
    // Step: 3 处理之后的点云的发布者
    // Step: 4 进行定时器与函数的绑定，定时发布数据
}
```

MapBuilderBridge 类的构造函数

```
C++
MapBuilderBridge::MapBuilderBridge(
    const NodeOptions& node_options,
    std::unique_ptr<cartographer::mapping::MapBuilderInterface>
map_builder,
    tf2_ros::Buffer* const tf_buffer)
    : node_options_(node_options),
      map_builder_(std::move(map_builder)),
      tf_buffer_(tf_buffer) {}
```

MapBuilder 类的构造函数

```
C++
MapBuilder::MapBuilder(const proto::MapBuilderOptions& options)
    : options_(options),
      thread_pool_(options.num_background_threads()) {

    // 2d 位姿图(后端)的初始化
    if (options.use_trajectory_builder_2d()) {
        pose_graph_ = absl::make_unique<PoseGraph2D>(
            options_.pose_graph_options(),
            absl::make_unique<optimization::OptimizationProblem2D>(
options_.pose_graph_options().optimization_problem_options()),
            &thread_pool_);
    }

    if (options.collate_by_trajectory()) {
        sensor_collator_ =
absl::make_unique<sensor::TrajectoryCollator>();
    } else {
        sensor_collator_ = absl::make_unique<sensor::Collator>();
    }
}
```


5.2 开始轨迹时的处理

node_main.cc 的 StartTrajectoryWithDefaultTopics 函数

```
C++
node.StartTrajectoryWithDefaultTopics(trajecory_options);
```

Node 类的 StartTrajectoryWithDefaultTopics 函数

```
C++
StartTrajectoryWithDefaultTopics();
AddTrajectory(options);
    const int trajectory_id =
map_builder_bridge_.AddTrajectory(expected_sensor_ids, options);
    LaunchSubscribers(options, trajectory_id); // 开始订阅传感器数据话
题
```

MapBuilderBridge 类的 AddTrajectory 函数

```
C++
int MapBuilderBridge::AddTrajectory() {
    // Step: 1 开始一条新的轨迹, 返回新轨迹的 id, 需要传入一个函数
    const int trajectory_id = map_builder_->AddTrajectoryBuilder(
        expected_sensor_ids,
        trajectory_options.trajectory_builder_options,
        [this]() {
            OnLocalSlamResult(trajectory_id, time, local_pose,
            range_data_in_local);
        });

    // Step: 2 为这个新轨迹 添加一个 SensorBridge
    sensor_bridges_[trajectory_id] = absl::make_unique<SensorBridge>(
        trajectory_options.num_subdivisions_per_laser_scan,
        trajectory_options.tracking_frame,
        node_options_.lookup_transform_timeout_sec,
        tf_buffer_,
        map_builder_->GetTrajectoryBuilder(trajectory_id)); //
CollatedTrajectoryBuilder

    return trajectory_id;
}
```

MapBuilder 类的 AddTrajectoryBuilder 函数

```
C++
int MapBuilder::AddTrajectoryBuilder(
    const std::set<SensorId>& expected_sensor_ids,
    const proto::TrajectoryBuilderOptions& trajectory_options,
    LocalSlamResultCallback local_slam_result_callback) {

    // CollatedTrajectoryBuilder 初始化

    trajectory_builders_.push_back(absl::make_unique<CollatedTrajectoryBuilder>(
        trajectory_options,
        sensor_collator_.get(), // sensor::Collator
        trajectory_id,
        expected_sensor_ids,
        CreateGlobalTrajectoryBuilder2D(
            std::move(local_trajectory_builder), trajectory_id,
            static_cast<PoseGraph2D*>(pose_graph_.get()),
            local_slam_result_callback,
            pose_graph_odometry_motion_filter)));

    return trajectory_id;
}

// 返回指向 CollatedTrajectoryBuilder 的指针
mapping::TrajectoryBuilderInterface *GetTrajectoryBuilder(
    int trajectory_id) const override {
    return trajectory_builders_.at(trajectory_id).get();
}
```

5.3 传感器数据走向分析

5.3.1 Node 类的 HandleLaserScanMessage 函数

```
C++
void Node::HandleLaserScanMessage(const int trajectory_id,
                                  const std::string& sensor_id,
                                  const
    sensor_msgs::LaserScan::ConstPtr& msg) {
    map_builder_bridge_.sensor_bridge(trajectory_id)
        ->HandleLaserScanMessage(sensor_id, msg);
}
```

```
}
```

5.3.2 SensorBridge 类的 HandleLaserScanMessage 函数

```
C++
void SensorBridge::HandleLaserScanMessage(
    const std::string& sensor_id, const
    sensor_msgs::LaserScan::ConstPtr& msg) {
    carto::sensor::PointCloudWithIntensities point_cloud;
    carto::common::Time time;
    std::tie(point_cloud, time) = ToPointCloudWithIntensities(*msg);
    HandleLaserScan(sensor_id, time, msg->header.frame_id,
    point_cloud);
}

void SensorBridge::HandleRangefinder(
    const std::string& sensor_id, const carto::common::Time time,
    const std::string& frame_id, const
    carto::sensor::TimedPointCloud& ranges) {
    if (sensor_to_tracking != nullptr) {
        trajectory_builder_->AddSensorData(
            sensor_id, carto::sensor::TimedPointCloudData{
                time,
                sensor_to_tracking-
>translation().cast<float>(),
                carto::sensor::TransformTimedPointCloud(
                    ranges, sensor_to_tracking-
>cast<float>())} ); // 强度始终为空
    }
}
```

SensorBridge 的 **trajectory_builder_** 是指向 **CollatedTrajectoryBuilder** 的指针

5.3.3 CollatedTrajectoryBuilder 类的 AddSensorData 函数

```
C++
void AddSensorData(
    const std::string& sensor_id,
    const sensor::TimedPointCloudData& timed_point_cloud_data)
override {
    AddData(sensor::MakeDispatchable(sensor_id,
    timed_point_cloud_data));
}
```

```

void
CollatedTrajectoryBuilder::AddData(std::unique_ptr<sensor::Data>
data) {
    sensor_collator_->AddSensorData(trajecory_id_, std::move(data));
}

```

CollatedTrajectoryBuilder 的 **sensor_collator_**是指向 **Collator** 的指针

5.3.4 Collator 类的 AddSensorData 函数

```

C++
void Collator::AddSensorData(const int trajectory_id,
                             std::unique_ptr<Data> data) {
    QueueKey queue_key{trajectory_id, data->GetSensorId()};
    queue_.Add(std::move(queue_key), std::move(data));
}

```

5.3.5 OrderedMultiQueue 类的 Add 函数 - 生成者

```

C++
void OrderedMultiQueue::Add(const QueueKey& queue_key,
                             std::unique_ptr<Data> data) {
    auto it = queues_.find(queue_key);

    // 向数据队列中添加数据
    it->second.queue.Push(std::move(data));

    // 传感器数据的分发处理
    Dispatch();
}

```

5.3.6 BlockingQueue 类的 Push 函数 - 缓冲区(阻塞队列)

```

C++
void Push(T t) {
    // 将数据加入队列，移动而非拷贝
    deque_.push_back(std::move(t));
}

```

5.3.7 OrderedMultiQueue 类的 Dispatch 函数 - 消费者

```
C++
void OrderedMultiQueue::Dispatch() {
    while (true) {
        const Data* next_data = nullptr;
        Queue* next_queue = nullptr;
        QueueKey next_queue_key;

        // 遍历所有的数据队列，找到所有数据队列的第一个数据中时间最老的一个数据
        for (auto it = queues_.begin(); it != queues_.end(); ) {
            const auto* data = it->second.queue.Peek<Data>();
        } // end for

        // 正常情况，数据时间都超过 common_start_time
        if (next_data->GetTime() >= common_start_time) {
            last_dispatched_time_ = next_data->GetTime();
            // 将数据传入 callback() 函数进行处理，并将这个数据从数据队列中删除
            next_queue->callback(next_queue->queue.Pop());
        }
    }
}
```

5.3.8 CollatedTrajectoryBuilder 类的 HandleCollatedSensorData 函数 - 消费者

```
C++
void CollatedTrajectoryBuilder::HandleCollatedSensorData(
    const std::string& sensor_id, std::unique_ptr<sensor::Data>
    data) {
    // 将排序好的数据送入 GlobalTrajectoryBuilder 中的 AddSensorData()函数
    数中进行使用
    data->AddToTrajectoryBuilder(wrapped_trajectory_builder_.get());
}

void Dispatchable::AddToTrajectoryBuilder(
    mapping::TrajectoryBuilderInterface *const trajectory_builder)
    override {
    trajectory_builder->AddSensorData(sensor_id_, data_);
}
```

这里的 `wrapped_trajectory_builder_` 是指向 `GlobalTrajectoryBuilder2D` 类的指针.

从 `GlobalTrajectoryBuilder2D` 开始, 数据才真正走到 SLAM 的前端与后端部分.

5.4 传感器数据分发器的创建

5.4.1 数据分发器的初始化

`node_main.cc` 的 `StartTrajectoryWithDefaultTopics` 函数

```
C++
node.StartTrajectoryWithDefaultTopics(trajectory_options);
```

Node 类的 `StartTrajectoryWithDefaultTopics` 函数

```
C++
StartTrajectoryWithDefaultTopics();
  AddTrajectory(options);
    const int trajectory_id =
map_builder_bridge_.AddTrajectory(expected_sensor_ids, options);
    LaunchSubscribers(options, trajectory_id); // 开始订阅传感器数据话
题
```

MapBuilderBridge 类的 `AddTrajectory` 函数

```
C++
int MapBuilderBridge::AddTrajectory() {
  // Step: 1 开始一条新的轨迹, 返回新轨迹的 id, 需要传入一个函数
  const int trajectory_id = map_builder_>AddTrajectoryBuilder(
    expected_sensor_ids,
    trajectory_options.trajectory_builder_options,
    [this]() {
      OnLocalSlamResult(trajectory_id, time, local_pose,
        range_data_in_local);
    });

  // Step: 2 为这个新轨迹 添加一个 SensorBridge
  sensor_bridges_[trajectory_id] = absl::make_unique<SensorBridge>(
    trajectory_options.num_subdivisions_per_laser_scan,
    trajectory_options.tracking_frame,
    node_options_.lookup_transform_timeout_sec,
    tf_buffer_,
    map_builder_>GetTrajectoryBuilder(trajectory_id)); //
```

```
CollatedTrajectoryBuilder
```

```
    return trajectory_id;  
}
```

MapBuilder 类的 AddTrajectoryBuilder 函数

C++

```
int MapBuilder::AddTrajectoryBuilder(  
    const std::set<SensorId>& expected_sensor_ids,  
    const proto::TrajectoryBuilderOptions& trajectory_options,  
    LocalSlamResultCallback local_slam_result_callback) {  
  
    // CollatedTrajectoryBuilder 初始化  
  
    trajectory_builders_.push_back(absl::make_unique<CollatedTrajectoryBuilder>(  
        trajectory_options,  
        sensor_collator_.get(), // sensor::Collator  
        trajectory_id,  
        expected_sensor_ids,  
        CreateGlobalTrajectoryBuilder2D(  
            std::move(local_trajectory_builder), trajectory_id,  
            static_cast<PoseGraph2D*>(pose_graph_.get()),  
            local_slam_result_callback,  
            pose_graph_odometry_motion_filter)));  
  
    return trajectory_id;  
}  
  
// 返回指向 CollatedTrajectoryBuilder 的指针  
mapping::TrajectoryBuilderInterface *GetTrajectoryBuilder(  
    int trajectory_id) const override {  
    return trajectory_builders_.at(trajectory_id).get();  
}
```


SensorBridge 类的构造函数

```
C++
SensorBridge::SensorBridge(
    const int num_subdivisions_per_laser_scan,
    const std::string& tracking_frame,
    const double lookup_transform_timeout_sec,
    tf2_ros::Buffer* const tf_buffer,
    carto::mapping::TrajectoryBuilderInterface* const
trajectory_builder) // CollatedTrajectoryBuilder
:
num_subdivisions_per_laser_scan_(num_subdivisions_per_laser_scan),
  tf_bridge_(tracking_frame, lookup_transform_timeout_sec,
tf_buffer),
  trajectory_builder_(trajectory_builder) //
CollatedTrajectoryBuilder
{}

```

SensorBridge 的 **trajectory_builder_** 是指向 **CollatedTrajectoryBuilder** 的指针

5.4.2 数据分发器的构造

CollatedTrajectoryBuilder 类的构造函数

```
C++
CollatedTrajectoryBuilder::CollatedTrajectoryBuilder(
    const proto::TrajectoryBuilderOptions& trajectory_options,
    sensor::CollatorInterface* const sensor_collator, //
sensor::Collator
    const int trajectory_id,
    const std::set<SensorId>& expected_sensor_ids,
    std::unique_ptr<TrajectoryBuilderInterface>
wrapped_trajectory_builder) // GlobalTrajectoryBuilder
: sensor_collator_(sensor_collator),
// sensor::Collator
  trajectory_id_(trajectory_id),

wrapped_trajectory_builder_(std::move(wrapped_trajectory_builder)),
// GlobalTrajectoryBuilder
  last_logging_time_(std::chrono::steady_clock::now()) {

// sensor::Collator 的初始化
sensor_collator_->AddTrajectory(
    trajectory_id, expected_sensor_id_strings,

```

```

        [this](const std::string& sensor_id,
std::unique_ptr<sensor::Data> data) {
            HandleCollatedSensorData(sensor_id, std::move(data));
        });
}

```

在 Collator 构造的时候传入了一个函数 HandleCollatedSensorData()

Collator 类的 AddTrajectory 函数

```

C++
void Collator::AddTrajectory(
    const int trajectory_id,
    const absl::flat_hash_set<std::string>& expected_sensor_ids,
    const Callback& callback) {
    for (const auto& sensor_id : expected_sensor_ids) {
        const auto queue_key = QueueKey{trajectory_id, sensor_id};
        queue_.AddQueue(queue_key,
            // void(std::unique_ptr<Data> data) 带了个默认参
数 sensor_id
            [callback, sensor_id](std::unique_ptr<Data>
data) {
                callback(sensor_id, std::move(data));
            });
        queue_keys_[trajectory_id].push_back(queue_key);
    }
}

// note: CollatorInterface::Callback 2 个参数
using Callback = std::function<void(const std::string&,
std::unique_ptr<Data>)>;

```

在这将传入的 Callback 函数放入 queue_ 里, 并传入一个参数.

OrderedMultiQueue 类的 AddQueue 函数

```

C++
void OrderedMultiQueue::AddQueue(const QueueKey& queue_key, Callback
callback) {
    CHECK_EQ(queues_.count(queue_key), 0);
    queues_[queue_key].callback = std::move(callback);
}

// note: OrderedMultiQueue::Callback 1 个参数

```

```
using Callback = std::function<void(std::unique_ptr<Data>)>>;
```

这里的 Callback 是一个参数的。

5.5 阻塞队列与生产者消费者模式

为什么要使用生产者消费者模式, 顺序执行不就可以了吗?

第一个作用: 解耦

生产者和消费者之间不直接依赖, 通过缓冲区通讯, 将两个类之间的耦合度降到最低。

第二个作用: 并发 (异步)

生产者直接调用消费者, 两者是同步 (阻塞) 的, 如果消费者吞吐数据很慢, 这时候生产者白白浪费大好时光。

而使用这种模式之后, 生产者将数据丢到缓冲区, 继续生产, 完全不依赖消费者, 程序执行效率会大大提高。

生产者消费者模式有什么意义?

复用: 通过将生产者类和消费者类独立开来, 可以对生产者类和消费者类进行独立的复用与扩展

5.6 传感器数据的分发处理过程总结

详见视频。

第六章 点云数据的预处理

传感器数据的走向

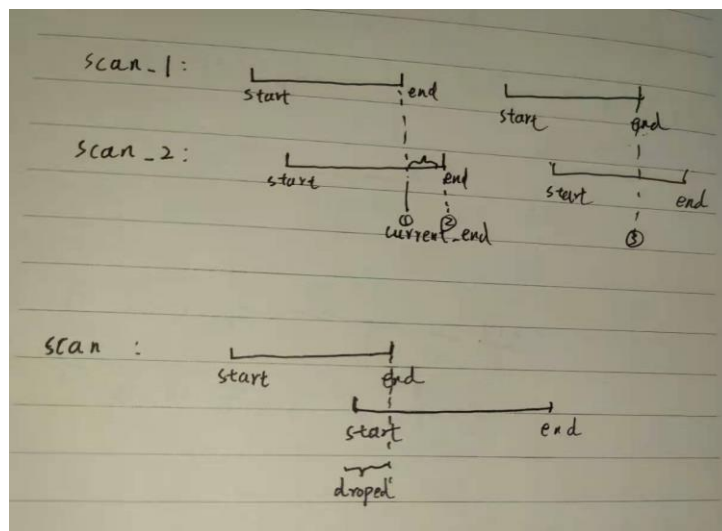
传感器数据从 CollatedTrajectoryBuilder 类的 HandleCollatedSensorData 函数 传递给 GlobalTrajectoryBuilder 类的相应函数。

从这里开始, 传感器数据才真正进入到 SLAM 的前端与后端部分。

6.1 GlobalTrajectoryBuilder 类讲解

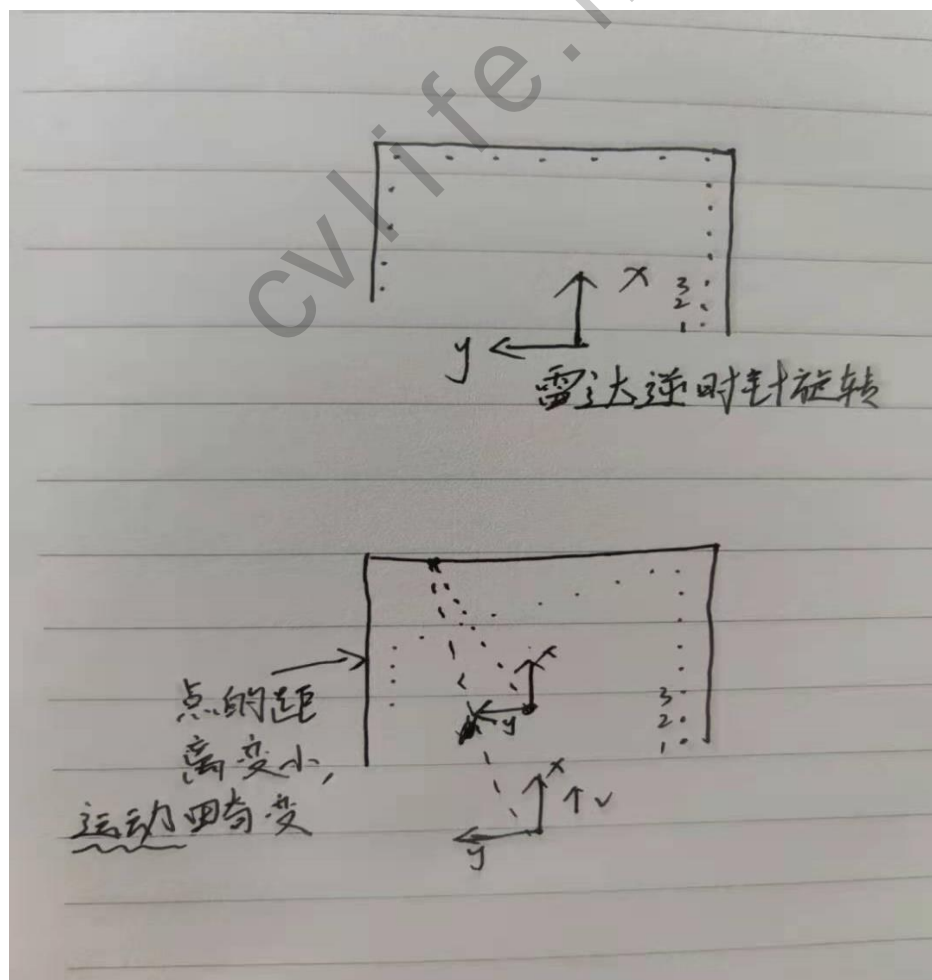
详见视频

6.2 多个激光雷达数据的时间同步与融合



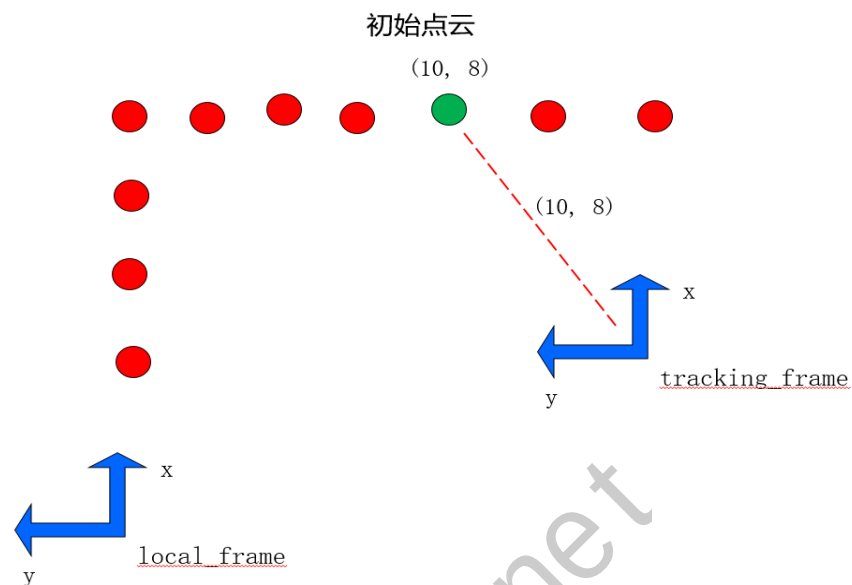
6.3 点云数据的运动畸变的校正

激光雷达数据运动畸变的校正, 同时将点云的相对于 tracking_frame 的点坐标 转成 相对于 local slam 坐标系的点坐标.



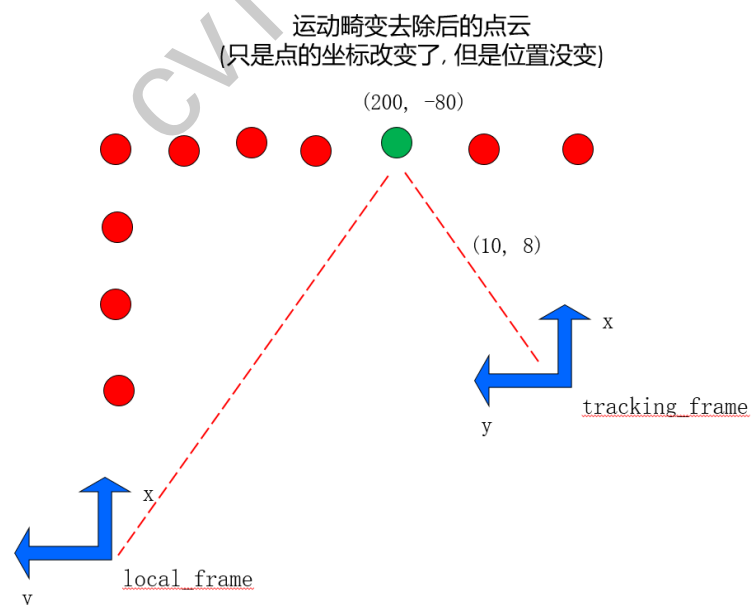
6.4 点云的坐标变换与 z 轴的过滤

- 初始点云, 点的坐标是相对于 tracking_frame 的, 点云是围绕着 tracking_frame 的

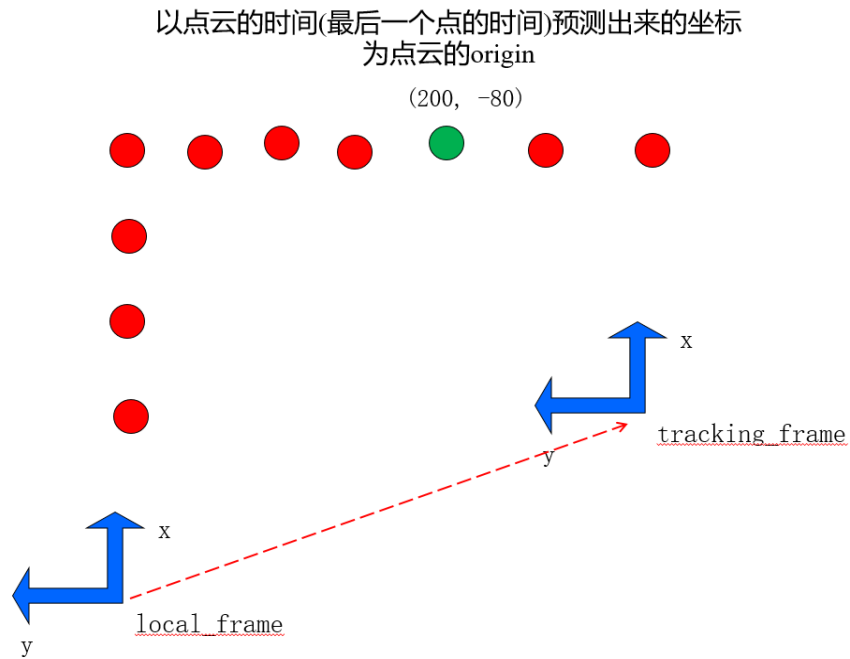


- 运动畸变去除后的点云, 点的坐标相对于 local_frame 了, 点云依然围绕着 tracking_frame

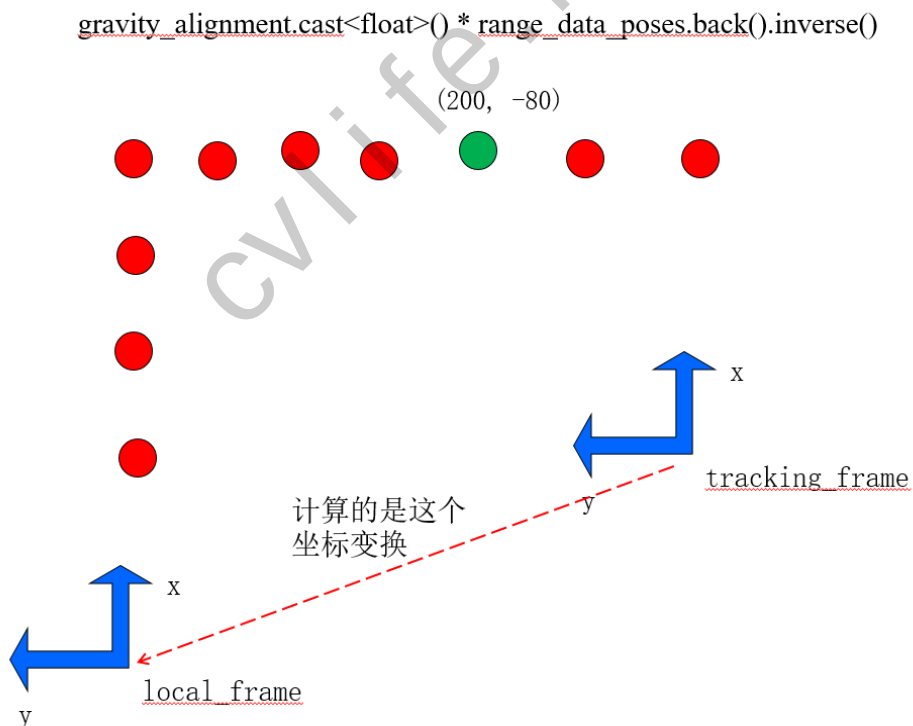
```
sensor::RangefinderPoint hit_in_local = range_data_poses[i] *  
sensor::ToRangefinderPoint(hit);
```



- 以点云的时间(也就是最后一个点的时间)预测出来的坐标做为点云的 origin
- `accumulated_range_data.origin = range_data_poses.back().translation()`



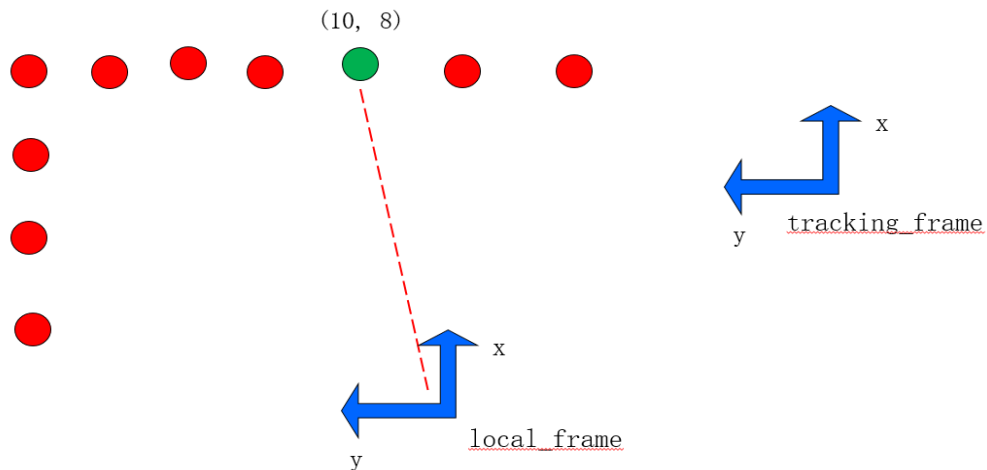
- 计算从 tracking_frame 变换到 local_frame 原点并且变换后姿态为 0 的坐标变换
`transform_to_gravity_aligned_frame = gravity_alignment.cast<float>()
 * range_data_poses.back().inverse()`



- 将点云进行平移与旋转, 点的坐标相对于 local_frame, 点云围绕这 local_frame 坐标系原点

```
sensor::TransformRangeData(range_data,  
transform_to_gravity_aligned_frame)
```

根据坐标变换, 将点云的位置进行平移与旋转



- 进行 z 轴的过滤

```
sensor::CropRangeData(sensor::TransformRangeData(range_data,  
transform_to_gravity_aligned_frame), options_.min_z(),  
options_.max_z())
```

- 单线雷达不能设置 大于 0 的 min_z, 因为单线雷达的 z 为 0

6.5 体素滤波与之后的处理

- 分别对 returns 点云 与 misses 点云 进行体素滤波

```
Plaintext  
sensor::RangeData{ cropped.origin,  
    sensor::VoxelFilter(cropped.returns,  
options_.voxel_filter_size()),  
    sensor::VoxelFilter(cropped.misses,  
options_.voxel_filter_size())};
```

- 对 returns 点云 进行自适应体素滤波

```
sensor::AdaptiveVoxelFilter(gravity_aligned_range_data.returns,  
options_.adaptive_voxel_filter_options())
```

- 将 原点位于 local 坐标系原点处的点云 变换成 原点位于匹配后的位姿处的点云

```
TransformRangeData(gravity_aligned_range_data,  
transform::Embed3D(pose_estimate_2d->cast<float>()) )
```

- 将 原点位于匹配后的位姿处的点云 返回到 node.cc 中, node.cc 将这个点云发布出去, 在 rviz 中可视化

6.6 3D 情况下的激光雷达数据的预处理

- 进行多个雷达点云数据的时间同步
- 对点云进行第一次体素滤波
- 激光雷达数据运动畸变的校正, 同时将点云的相对于 `tracking_frame` 的点坐标 转成 相对于 `local slam` 坐标系的点坐标
- 分别对 `returns` 与 `misses` 进行第二次体素滤波
- 将原点位于机器人当前位姿处的点云 转成 原点位于 `local` 坐标系原点处的点云
- 使用高分辨率进行自适应体素滤波 生成高分辨率点云
- 使用低分辨率进行自适应体素滤波 生成低分辨率点云
- 将 原点位于 `local` 坐标系原点处的点云 变换成 原点位于匹配后的位姿处的点云
- 将 原点位于匹配后的位姿处的点云 返回到 `node.cc` 中, `node.cc` 将这个点云发布出去, 在 `rviz` 中可视化

第七章 初始位姿估计

7.1 代码总体调用流程分析

详见附图

7.2 基于 IMU 与里程计的位姿推测器

详见附图

重要成员变量说明

姿态预测相关

- `imu_tracker_` 只在添加位姿时更新, 用于保存添加校位姿准时的姿态
- `odometry_imu_tracker_` 只在添加位姿时更新, 用于根据里程计数据计算线速度时姿态的预测
- `extrapolation_imu_tracker_` 只在添加位姿时更新, 用于位姿预测时的姿态预测

通过里程计计算的线速度与角速度

- `linear_velocity_from_odometry_` 只在添加里程计数据时更新, 用于位姿预测时的平移量预测
- `angular_velocity_from_odometry_` 只在添加里程计数据时更新, 用于不使用 `imu`

数据时的 `imu_tracker_` 的角速度的更新

通过 `pose` 计算的线速度与角速度

- `linear_velocity_from_poses_` 只在添加位姿时更新, 用于位姿预测时 不使用里程计数据时 平移量的预测
- `angular_velocity_from_poses_` 只在添加位姿时更新, 用于 不使用里程计数据时的 `imu_tracker_` 的角速度的更新

传感器数据队列的个数

- `imu_data_` 队列数据的个数最少是 1 个
- `odometry_data_` 队列数据的个数最少是 2 个, 所以, `odometry_data_.size() < 2` 就意味着不使用里程计
- `timed_pose_queue_` 队列数据的个数最少是 2 个

7.3 位姿推测器的优缺点分析与总结

预测位姿时的 4 种情况 都是匀速模型

- 使用 imu, 使用里程计
 - 平移的预测: 通过里程计数据队列开始和末尾的 2 个数据计算出的线速度乘以时间
 - 姿态的预测: 通过 imu 的角速度乘以时间
- 使用 imu, 不使用里程计
 - 平移的预测: 通过 `pose` 数据队列开始和末尾的 2 个数据计算出的线速度乘以时间
 - 姿态的预测: 通过 imu 的角速度乘以时间
- 不使用 imu, 使用里程计
 - 平移的预测: 通过里程计数据队列开始和末尾的 2 个数据计算出的线速度乘以时间
 - 姿态的预测: 通过里程计数据队列开始和末尾的 2 个数据计算出的角速度乘以时间
- 不使用 imu, 不是用里程计
 - 平移的预测: 通过 `pose` 数据队列开始和末尾的 2 个数据计算出的线速度乘以时间

- 姿态的预测: 通过 pose 数据队列开始和末尾的 2 个数据计算出的角速度乘以时间

总结:

- 预测平移时: 有里程计就用里程计的线速度, 没有里程计就用 pose 计算的线速度进行预测
- 预测姿态时: 有 IMU 就用 IMU 的角速度, 没有 IMU 时, 如果有里程计就用里程计计算出的角速度, 没有里程计就用 pose 计算的角速度进行预测
- 预测的都是相对值, 要加上最后一个 pose 的位姿

可能有问题的点

详见附图

- 计算 pose 的线速度与角速度时, 是采用的数据队列开始和末尾的 2 个数据计算的
- 计算里程计的线速度与角速度时, 是采用的数据队列开始和末尾的 2 个数据计算的
- 使用里程计, 不使用 imu 时, **计算里程计的线速度方向和姿态的预测时**, 用的是里程计数据队列开始和末尾的 2 个数据的平均角速度计算的, **时间长了就不准**
- 不使用里程计, 不使用 imu 时, 用的是 pose 数据队列开始和末尾的 2 个数据的平均角速度计算的, **时间长了就不准**
- **添加位姿时, 没有用 pose 的姿态对 imu_tracker_进行校准, 也没有对整体位姿预测器进行校准, 只计算了 pose 的线速度与角速度**
- 从代码上看, cartographer 认为位姿推测器推测出来的位姿与姿态是准确的

可能的改进建议

- pose 的距离越小, 匀速模型越能代替机器人的线速度与角速度, 计算 pose 的线速度与角速度时, 可以考虑使用最近的 2 个数据进行计算
- 里程计距离越短数据越准, 计算里程计的线速度与角速度时, 可以考虑使用最近的 2 个数据进行计算
- 使用里程计, 不使用 imu 时, 计算里程计的线速度方向时, 可以考虑使用里程计的角度进行计算
- 使用里程计, 不使用 imu 时, 进行姿态的预测时, 可以考虑使用里程计的角度进行预测
- 不使用里程计, 不使用 imu 时, 可以考虑用最近的 2 个 pose 计算线速度与角速度
- 使用 pose 对 imu_tracker_的航向角进行校准

第八章 概率栅格地图

8.1 概率栅格地图简介

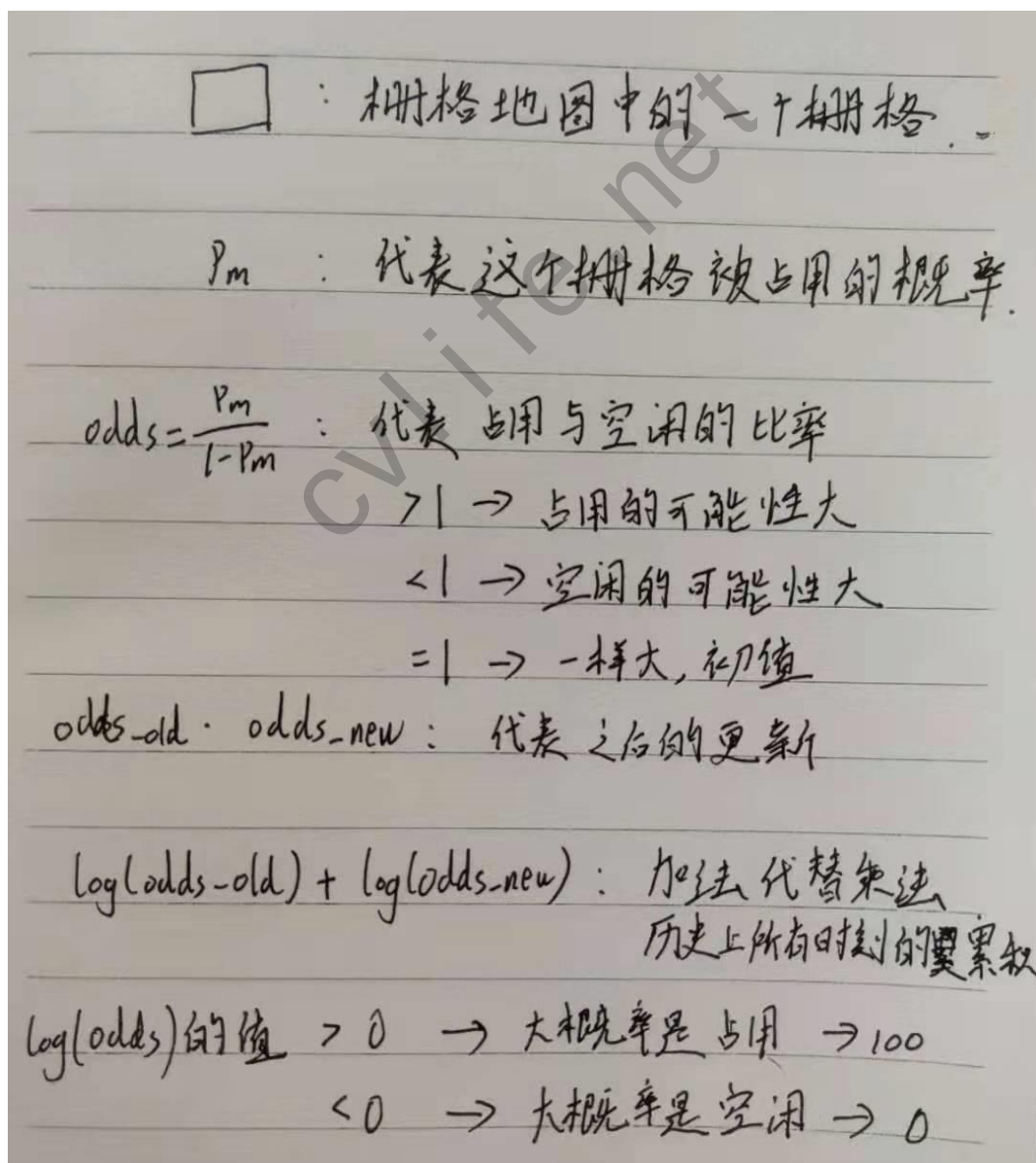
栅格地图是二维激光 SLAM 的特点, 能够将环境通过地图的形式表达出来.

栅格地图的实现是二维激光 SLAM 的一个难点

三维激光 SLAM 形成的点云地图不需要自己手动实现点云的数据结构, PCL 中有写好的数据类型, 直接调用就行. 视觉 SLAM 形成的点云地图也可以用 PCL 来实现

唯独二维激光 SLAM 的栅格地图需要自己手动实现, 目前所有的二维激光 SLAM 的栅格地图都是 SLAM 作者自己写的, 没有通用的数据结构

8.1.1 ActiveSubmaps2D 类讲解



8.1.2 Submap2D 类讲解

详见附图

8.2 地图的坐标系与原点

8.2.1 地图相关类总述

详见附图

8.2.2 MapLimits 类

地图的坐标系

```
C++
/**
 * ros 的地图坐标系      cartographer 的地图坐标系      cartographer 地图的
像素坐标系
 *
 * ^ y                      ^ x                      0-----> x
 * |                        |                        |
 * |                        |                        |
 * |                        |                        |
 * 0 -----> x            y <-----0                y
 *
 * ros 的地图坐标系：左下角为原点，向右为 x 正方向，向上为 y 正方向，角度以
x 轴正向为 0 度，逆时针为正
 * cartographer 的地图坐标系：坐标系右下角为原点，向上为 x 正方向，向左为
y 正方向
 *                      角度正方向以 x 轴正向为 0 度，逆时针为正
 * cartographer 地图的像素坐标系：左上角为原点，向右为 x 正方向，向下为 y
正方向
 */
```

栅格地图的原点的设置

```
C++
// Step: 8 将 原点位于 local 坐标系原点处的点云 变换成 原点位于匹配后的
位姿处的点云
sensor::RangeData range_data_in_local =
    TransformRangeData(gravity_aligned_range_data,
                       transform::Embed3D(pose_estimate_2d-
>cast<float>()));
// 将校正后的雷达数据写入 submap
```

```

    std::unique_ptr<InsertionResult> insertion_result =
    InsertIntoSubmap(
        time, range_data_in_local,
        filtered_gravity_aligned_point_cloud,
        pose_estimate, gravity_alignment.rotation());

// 将点云数据写入到 submap 中
std::vector<std::shared_ptr<const Submap2D>>
ActiveSubmaps2D::InsertRangeData(
    const sensor::RangeData& range_data) {
    if (submaps_.empty() ||
        submaps_.back()->num_range_data() ==
        options_.num_range_data()) {
        AddSubmap(range_data.origin.head<2>());
    }

    return submaps();
}

void ActiveSubmaps2D::AddSubmap(const Eigen::Vector2f& origin) {
    if (submaps_.size() >= 2) {
        submaps_.erase(submaps_.begin());
    }
    // 新建一个子图
    submaps_.push_back(absl::make_unique<Submap2D>(
        origin,
        std::unique_ptr<Grid2D>(
            static_cast<Grid2D*>(CreateGrid(origin).release()))),
        &conversion_tables_));
}

```

第一个雷达数据到来时的栅格地图的原点是如何确定的

```

C++
std::unique_ptr<transform::Rigid2d>
LocalTrajectoryBuilder2D::ScanMatch(
    const common::Time time, const transform::Rigid2d&
    pose_prediction,
    const sensor::PointCloud& filtered_gravity_aligned_point_cloud)
{
    if (active_submaps_.submaps().empty()) {
        return absl::make_unique<transform::Rigid2d>(pose_prediction);
    }
    //...
}

```

```

}

std::unique_ptr<LocalTrajectoryBuilder2D::MatchingResult>
LocalTrajectoryBuilder2D::AddAccumulatedRangeData(){
    const transform::Rigid3d non_gravity_aligned_pose_prediction =
        extrapolator_>ExtrapolatePose(time);
    // 将三维位姿先旋转到姿态为 0, 再取 xy 坐标将三维位姿转成二维位姿
    const transform::Rigid2d pose_prediction = transform::Project2D(
        non_gravity_aligned_pose_prediction *
gravity_alignment.inverse());
}

// 预测得到 time 时刻 tracking frame 在 local 坐标系下的位姿
transform::Rigid3d PoseExtrapolator::ExtrapolatePose(const
common::Time time) {
    const TimedPose& newest_timed_pose = timed_pose_queue_.back();
    CHECK_GE(time, newest_timed_pose.time);
    // 如果本次预测时间与上次计算时间相同 就不再重复计算
    if (cached_extrapolated_pose_.time != time) {
        // 预测 tracking frame 在 local 坐标系下 time 时刻的位置
        const Eigen::Vector3d translation =
            ExtrapolateTranslation(time) +
newest_timed_pose.pose.translation();
        // 预测 tracking frame 在 local 坐标系下 time 时刻的姿态
        const Eigen::Quaterniond rotation =
            newest_timed_pose.pose.rotation() *
            ExtrapolateRotation(time, extrapolation_imu_tracker_.get());
        cached_extrapolated_pose_ =
            TimedPose{time, transform::Rigid3d{translation, rotation}};
    }
    return cached_extrapolated_pose_.pose;
}

// 如果 Extrapolator 没有初始化就进行初始化
void LocalTrajectoryBuilder2D::InitializeExtrapolator(const
common::Time time) {
    // 如果已经初始化过了就直接返回
    if (extrapolator_ != nullptr) {
        return;
    }

    // 初始化位姿推测器
    extrapolator_ = absl::make_unique<PoseExtrapolator>(<

```



```

        ::cartographer::common::FromSeconds(options_.pose_extrapolator_
_options())
                                .constant_velocity()
                                .pose_queue_duration()
), // 0.001s
options_.pose_extrapolator_options()
    .constant_velocity()
    .imu_gravity_time_constant()); // 10
// 添加初始位姿
extrapolator_->AddPose(time, transform::Rigid3d::Identity());
}

```

8.3 栅格地图中栅格值的更新

8.3.1 栅格地图的栅格是如何更新的

□ : 栅格地图中的一个栅格.

p_m : 代表这个栅格被占用的概率.

$odds = \frac{p_m}{1-p_m}$: 代表 占用与空闲的比率

$> 1 \rightarrow$ 占用的可能性大

$< 1 \rightarrow$ 空闲的可能性大

$= 1 \rightarrow$ 一样大, 初值

$odds_{old} \cdot odds_{new}$: 代表 之后的更新

$\log(odds_{old}) + \log(odds_{new})$: 加法代替乘法
历史上所有时刻的累加

$\log(odds)$ 的值 $> 0 \rightarrow$ 大概率是占用 $\rightarrow 100$

$< 0 \rightarrow$ 大概率是空闲 $\rightarrow 0$

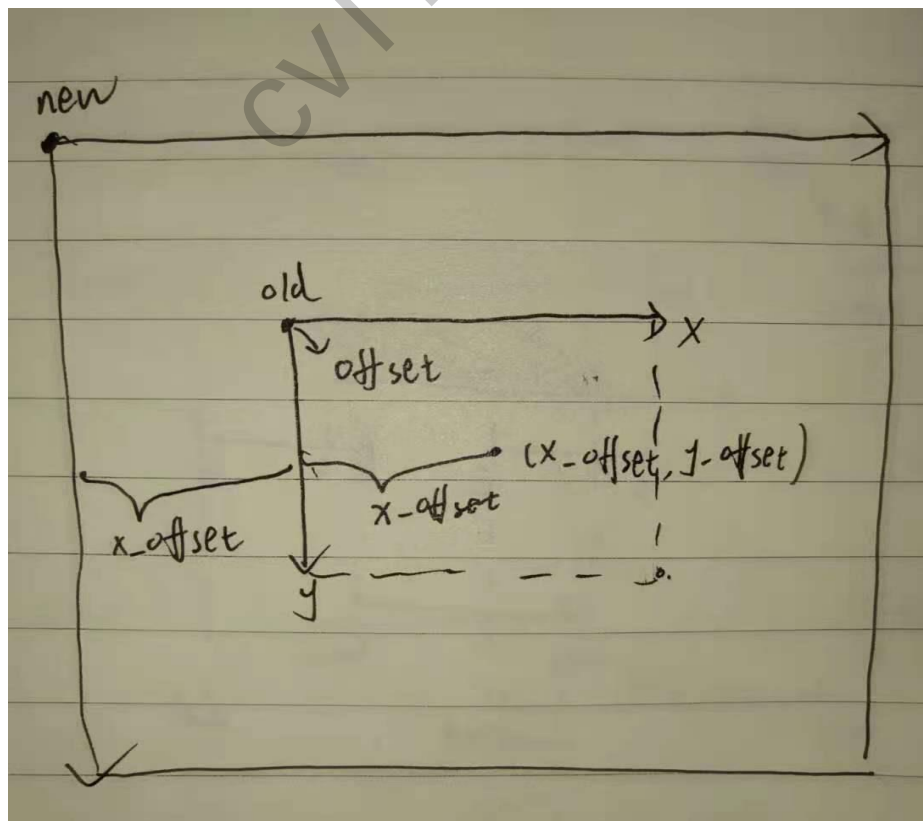
8.3.2 cartographer 中的栅格地图的相关概念与公式

- **probability**: 栅格被占据的概率
- $kMinProbability = 0.1$, $kMaxProbability = 0.9$, $kUnknownProbabilityValue = 0$
- **Odds**: $probability / (1.0f - probability)$
- **CorrespondenceCost**: 栅格是 free 的概率; $CorrespondenceCost + probability = 1$
- $kMinCorrespondenceCost = 0.1$, $kMaxCorrespondenceCost = 0.9$
- $kUnknownCorrespondenceValue = 0$, $kUpdateMarker = 32768$
- **Value**: 代码里存储的栅格值, 是 $[0, 32767]$ 范围内的 uint16 整数
- **value_to_correspondence_cost_table**: 将 $[0, 1 \sim 32767]$ 映射到 $[0, 0.1 \sim 0.9]$ 的转换表
- hit_table_ 计算 $[0, 1 \sim 32767]$ 按照占用概率 0.55 更新之后的值
- miss_table_ 计算 $[0, 1 \sim 32767]$ 按照空闲概率 0.49 更新之后的值

8.3.3 论文里的地图更新公式讲解

详见视频

8.4 Grid2D 类讲解

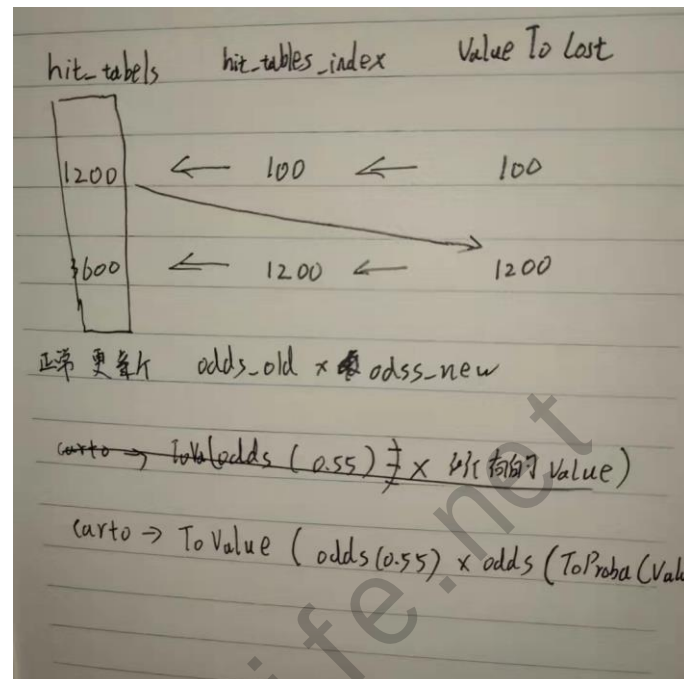


8.5 ProbabilityGrid 类讲解

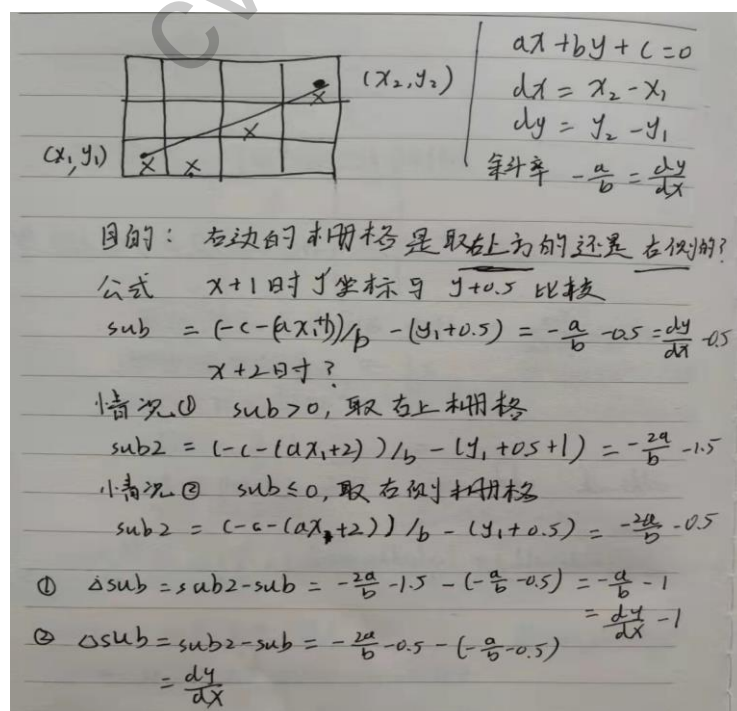
详见视频

8.6 将点云数据写入到栅格地图中

查找表



8.7 贝汉明算法



Bresenham 画线算法 <https://www.jianshu.com/p/d63bf63a0e28>

图解 cartographer 之雷达模型 CastRay

<https://blog.csdn.net/chaosir1991/article/details/109561010>

8.8 栅格地图的总结

代码混乱, 计算查找表的代码重复. 看代码时候一定要注意自己多写笔记, 把代码结构, 函数调用关系弄懂.

详见附图

第九章 Ceres 实现 2D 扫描匹配

9.1 扫描匹配的概念与公式推导

扫描匹配的概念

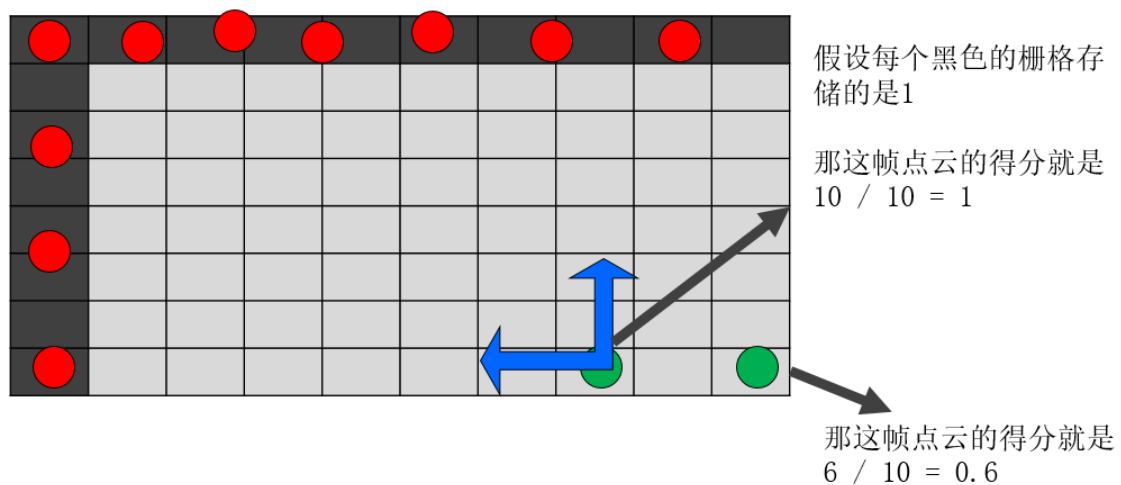
扫描匹配, 又叫 scan match, 是二维激光 SLAM 中独有的, 因为只有二维激光 SLAM 中存在栅格地图.

扫描匹配的目的是找到雷达点云在栅格地图中的位置与角度.

扫描匹配的实现

暴力匹配: 将点云的位置与角度在栅格地图上的每个栅格都获取一下得分, 选出得分最高的一个位置与姿态.

通过点云落在栅格上对应的栅格值的和, 当做这帧点云在这个位姿下的得分.



基于优化的方法:

优化方法: 最速下降法, 高斯牛顿法, LM 法

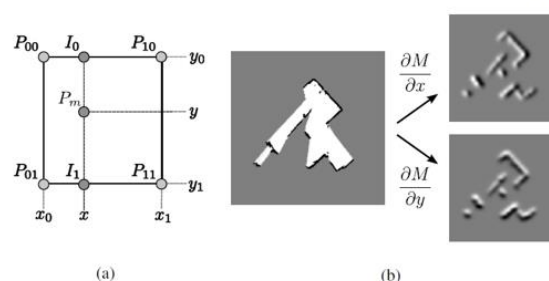
Hector 的论文中有公式推导 [A Flexible and Scalable SLAM System with Full 3D Motion Estimation]

Hector 论文公式推导与相关代码解析

<https://blog.csdn.net/tiancailx/article/details/113522899>

$$M(P_m) \approx \frac{y - y_0}{y_1 - y_0} \left(\frac{x - x_0}{x_1 - x_0} M(P_{11}) + \frac{x_1 - x}{x_1 - x_0} M(P_{01}) \right) + \frac{y_1 - y}{y_1 - y_0} \left(\frac{x - x_0}{x_1 - x_0} M(P_{10}) + \frac{x_1 - x}{x_1 - x_0} M(P_{00}) \right) \quad (4)$$

The derivatives can be approximated by:



$$\frac{\partial M}{\partial x}(P_m) \approx \frac{y - y_0}{y_1 - y_0} (M(P_{11}) - M(P_{01})) + \frac{y_1 - y}{y_1 - y_0} (M(P_{10}) - M(P_{00}))$$

$$\frac{\partial M}{\partial y}(P_m) \approx \frac{x - x_0}{x_1 - x_0} (M(P_{11}) - M(P_{10})) + \frac{x_1 - x}{x_1 - x_0} (M(P_{01}) - M(P_{00}))$$

目标函数:

$$\sum_{i=1} [1 - M(\mathbf{S}_i(\boldsymbol{\xi} + \Delta\boldsymbol{\xi}))]^2 \rightarrow 0.$$

第一步: 泰勒展开

$$\sum_{i=1}^n \left[1 - M(\mathbf{S}_i(\boldsymbol{\xi})) - \nabla M(\mathbf{S}_i(\boldsymbol{\xi})) \frac{\partial \mathbf{S}_i(\boldsymbol{\xi})}{\partial \boldsymbol{\xi}} \Delta\boldsymbol{\xi} \right]^2 \rightarrow 0.$$

第二步: 对 $\Delta\boldsymbol{\xi}$ 求偏导

$$2 \sum_{i=1}^n \left[\nabla M(\mathbf{S}_i(\boldsymbol{\xi})) \frac{\partial \mathbf{S}_i(\boldsymbol{\xi})}{\partial \boldsymbol{\xi}} \right]^T \left[1 - M(\mathbf{S}_i(\boldsymbol{\xi})) - \nabla M(\mathbf{S}_i(\boldsymbol{\xi})) \frac{\partial \mathbf{S}_i(\boldsymbol{\xi})}{\partial \boldsymbol{\xi}} \Delta\boldsymbol{\xi} \right] = 0$$

$$\mathbf{J}(\mathbf{x}) \Delta\mathbf{x}$$

第三步: 根据高斯牛顿解的公式

$$\Delta\boldsymbol{\xi} = \mathbf{H}^{-1} \sum_{i=1}^n \left[\nabla M(\mathbf{S}_i(\boldsymbol{\xi})) \frac{\partial \mathbf{S}_i(\boldsymbol{\xi})}{\partial \boldsymbol{\xi}} \right]^T [1 - M(\mathbf{S}_i(\boldsymbol{\xi}))]$$

$$\Delta\mathbf{x}^* = \arg \min_{\Delta\mathbf{x}} \frac{1}{2} \|f(\mathbf{x}) + \mathbf{J}(\mathbf{x}) \Delta\mathbf{x}\|^2.$$

$$\mathbf{J}(\mathbf{x})^T \mathbf{J}(\mathbf{x}) \Delta\mathbf{x} = -\mathbf{J}(\mathbf{x})^T f(\mathbf{x}).$$

$$\mathbf{H} \Delta\mathbf{x} = \mathbf{g}.$$

$$\mathbf{H} = \left[\nabla M(\mathbf{S}_i(\boldsymbol{\xi})) \frac{\partial \mathbf{S}_i(\boldsymbol{\xi})}{\partial \boldsymbol{\xi}} \right]^T \left[\nabla M(\mathbf{S}_i(\boldsymbol{\xi})) \frac{\partial \mathbf{S}_i(\boldsymbol{\xi})}{\partial \boldsymbol{\xi}} \right]$$

0.05, 0.1, 0.2, 3个分辨率地图, 迭代3次

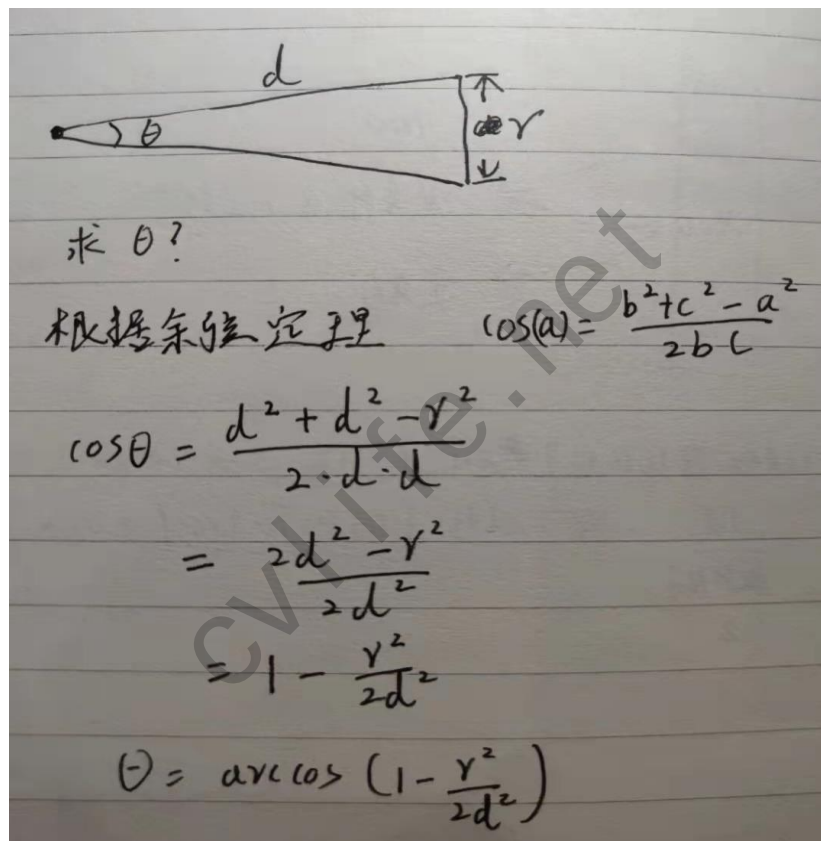
扫描匹配的应用

- 前端部分: 存在先验位姿的情况下确定机器人在地图中的位置
- 回环检测: 通过当前点云与之前的地图进行匹配, 如果匹配上了就证明存在回环
- 重定位: 在不存在先验位姿的情况下确定机器人在地图中的位置

9.2 实时相关性扫描匹配

RealTimeCorrelativeScanMatcher2D : 就是暴力搜索

如何求角度分辨率



9.3 Ceres 简介

概念介绍

http://www.ceres-solver.org/nnls_tutorial.html#introduction

官方教程

http://www.ceres-solver.org/nnls_tutorial.html#hello-world

其他博客: 都是翻译的官方文档

Ceres 详解 (一) (二) (三)

https://blog.csdn.net/weixin_43991178/article/details/100532618

使用步骤

使用 Ceres 求解非线性优化问题，一共分为四个步骤

- 第一步 声明 CostFunction

```
C++
class MyScalarCostFunction {
    MyScalarCostFunction(double k): k_(k) {}

    template <typename T>
    bool operator()(const T* const x , const T* const y, T* e) const {
        // (k - x^T * y)^2, 平方由 ceres 自动添加
        e[0] = k_ - x[0] * y[0] - x[1] * y[1];
        return true;
    }

private:
    double k_;
};
```

- 第二步 调用 AutoDiffCostFunction 函数

```
C++
CostFunction* cost_function
    = new AutoDiffCostFunction<MyScalarCostFunction, 1, 2, 2>(
        new MyScalarCostFunction(1.0));
                                     ^   ^   ^
                                     |   |   |
        Dimension of residual -----+ |   |
        Dimension of x -----+       |
        Dimension of y -----+

CostFunction* cost_function
    = new AutoDiffCostFunction<MyScalarCostFunction, DYNAMIC, 2, 2>(
        new CostFunctionWithDynamicNumResiduals(1.0),
        runtime_number_of_residuals); <-----+   ^   ^   ^
                                     |           |   |
                                     |           |   |
                                     |           |   |
        Actual number of residuals -----+   |   |   |
        Indicate dynamic number of residuals -----+   |   |
        Dimension of x -----+                   |
        Dimension of y -----+
```

- 第三步 添加残差块

```
C++
ceres::Problem problem;
problem.AddResidualBlock(cost_function);
```

- 第四步 进行求解

```
C++
ceres::Solver::Summary summary;
ceres::Solve(ceres_solver_options_, &problem, summary);
std::cout << summary.BriefReport() << std::endl; //
summary.FullReport()
```

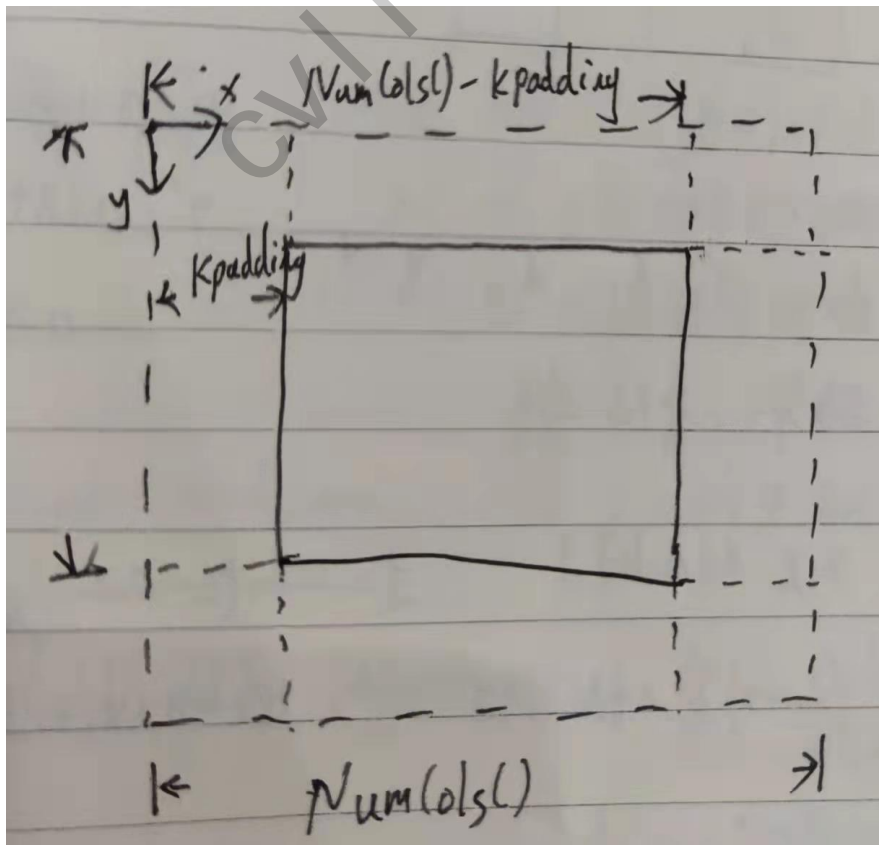
class BiCubicInterpolator

http://www.ceres-solver.org/nns_modeling.html#_CPPv4N5ceres19BiCubicInterpolatorE

9.4 Ceres 编程实践

第二次编程作业, 详见视频

9.5 基于优化的扫描匹配



9.6 前端扫描匹配的总结

位姿的连续性

`RealTimeCorrelativeScanMatcher2D::Match` 是通过对像素坐标进行遍历, 所以这个函数计算出的坐标的分辨率是栅格的分辨率, 不是连续的.

`CeresScanMatcher2D::Match` 优化的是物理坐标, 不是像素坐标, 所以坐标是连续的.

可能有问题的点

- 平移和旋转的残差项是逼近于先验位姿的, 当先验位姿不准确时会产生问题

可能的改进建议

- 先将地图的权重调大, 平移旋转的权重调小, 如 1000, 1, 1, 或者 100, 1, 1
- 调参没有作用的时候可以将平移和旋转的残差项注释掉

第十章 2D 后端优化 (上)

10.1 后端优化的概念与理论过程分析

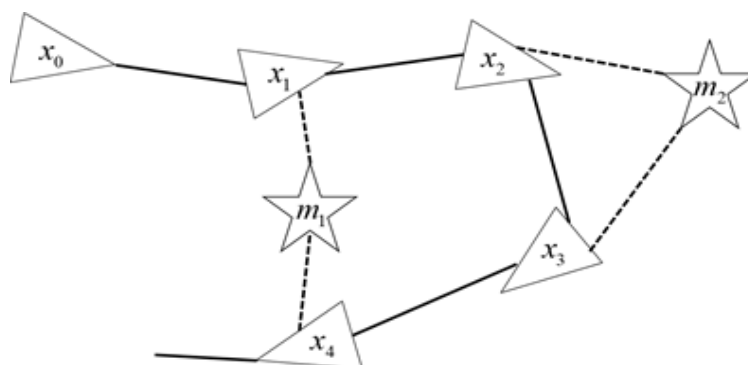
什么是图

由节点和边组成的一种数据结构, 节点之间的关系可以是任意的, 图中任意两节点之间都可能相关(存在边).

什么是位姿图

SPA 论文中对位姿图的定义: 位姿图是一组通过非线性约束连接的机器人位姿, 这些非线性约束是从对附近位姿共有的特征的观察中获得的.

位姿图是一种图, 节点代表位姿, 边代表 2 个位姿间的相对坐标变换(也叫约束).



三角形表示机器人位姿, 三角形之间的连线表示约束(坐标变换)

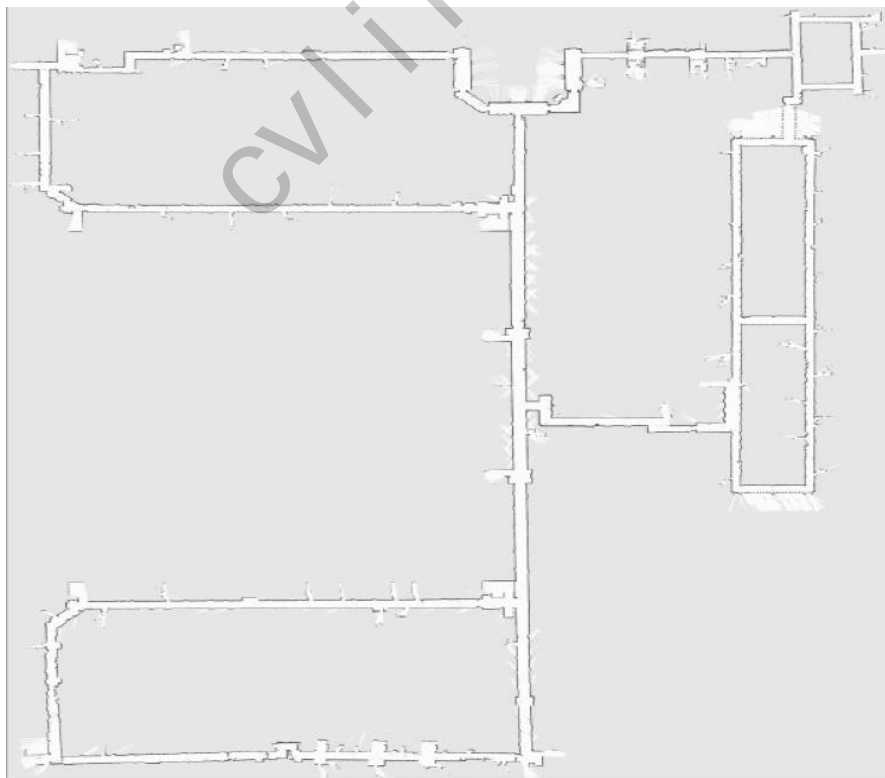
什么是优化

由于前端里程计会有累计误差, 那有没有一种方法可以将这种累计误差减小甚至消除掉呢?

这就是优化的目的与作用.



优化前



优化后

SPA 论文中对优化的定义：其目标是寻找一个节点间的配置关系，使得节点间约束的测量概率最大。

通过图结构，根据约束与所有的节点，将所有的误差减小，并且平均分散到每个节点上去，以达到减小整体误差的作用。

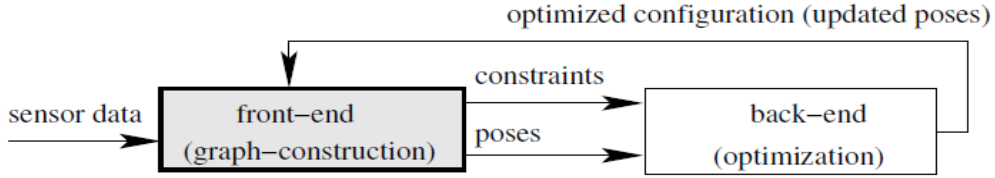


Fig. 2. Typical graph-based SLAM system. The front-end and the back end are executed in an interleaved way.

优化的这部分模块，一般称为后端部分，这就是 **后端优化** 模块的由来。

如何做优化呢

SPA 优化

论文存放在 `cartographer_detailed_comments_ws/src/cartographer/docs` 文件夹内

论文名字为: 2010 - Efficient Sparse Pose Adjustment for 2D Mapping - Konolige et al.pdf

推荐自己阅读一下 前第 3 节的内容以及第 4 节的 AB 两部分

论文翻译 <https://blog.csdn.net/u014527548/article/details/106238658>

残差方程

The variables of the system are the set of global poses $\mathbf{c}$ of the robot, parameterized by a translation and angle: $c_i = [t_i, \theta_i] = [x_i, y_i, \theta_i]^\top$. A *constraint* is a measurement of one node c_j from another's (c_i) position. The measured offset between c_i and c_j , in c_i 's frame, is $\bar{z}_{ij}$, with precision matrix Λ_{ij} (inverse of covariance). For any actual poses of c_i and c_j , their offset can be calculated as

$$h(c_i, c_j) \equiv \begin{cases} R_i^\top (t_j - t_i) \\ \theta_j - \theta_i \end{cases} \quad (1)$$

Here R_i is the 2x2 rotation matrix of θ_i . $h(c_i, c_j)$ is called the *measurement equation*.

The error function associated with a constraint, and the total error, are

$$\begin{aligned} e_{ij} &\equiv \bar{z}_{ij} - h(c_i, c_j) \\ \chi^2(\mathbf{c}, \mathbf{p}) &\equiv \sum_{ij} e_{ij}^\top \Lambda_{ij} e_{ij} \end{aligned} \quad (2)$$

Note that the angle parameter in $h(c_i, c_j)$ is not unique, since adding or subtracting 2π yields the same result. Angle differences are always normalized to the interval $(-\pi, \pi]$ when they occur.

目标

- 第一步: 确定 2 个节点在 global 坐标系下的相对位姿变换
- 第二步: 通过其他方式再次获取这 2 个节点的相对位姿变换
- 第三步: 对这 2 个相对位姿变换的差 的最小二乘问题进行求解
- 第四步: 进行求解之后会得到一个增量 Δx , 将当前位姿加上这个增量后就得到了优化后的位姿

Solving the linear equation yields an increment Δx that can be added back in to the current value of x as follows:

$$\begin{aligned} t_i &= t_i + \Delta t_i \\ \theta_i &= \theta_i + \Delta \theta_i \end{aligned} \quad (6)$$

cartographer 代码里实现位姿图优化的思路

cartographer 使用 ceres 进行位姿图优化, ceres 求解的是残差的和.

单一约束是不能进行优化的, 至少要有 2 个约束才能进行基于 ceres 的位姿图优化.

所以现在问题就是如何找到 2 个约束

第一个约束, 根据 SPA 论文, 就是节点 global 坐标系下的相对坐标变换.

所以, 第二个约束怎么求呢?

cartographer 中设置了如下 5 种第二个约束及残差, 共同进行优化.

- 第一种残差 将节点(tracking 的位姿)与节点(子图原点位姿)在 global 坐标系下的相对位姿 与 约束(包含子图内约束与子图间约束) 的差值作为残差项
- 第二种残差 landmark 数据 与 通过 2 个节点位姿插值出来的相对位姿 的差值作为残差项
- 第三种残差 节点与节点间在 global 坐标系下的相对坐标变换 与 通过里程计数据插值出的相对坐标变换 的差值作为残差项
- 第四种残差 节点与节点间在 global 坐标系下的相对坐标变换 与 相邻 2 个节点在 local 坐标系下的相对坐标变换 的差值作为残差项
- 第五种残差 节点与 gps 坐标系原点在 global 坐标系下的相对坐标变换 与 通过 gps 数据进行插值得到的相对坐标变换 的差值作为残差项

所以, 整个后端优化问题, 可以简化成

- 求节点在 global 坐标系下的位姿 (节点)
- 求节点的约束(子图内约束与子图间约束, 子图间约束又称为回环约束) (边)

- 求其他四种残差共同构建残差方差
- 进行优化求解

节点在 global 坐标系下的位姿与子图内约束

- PoseGraph2D::ComputeConstraintsForNode

子图间约束又称为回环约束

- ConstraintBuilder2D
- FastCorrelativeScanMatcher2D
- PrecomputationGridStack2D

构建残差方差与优化求解

- OptimizationProblem2D::Solve

10.2 后端优化整体函数调用流程分析

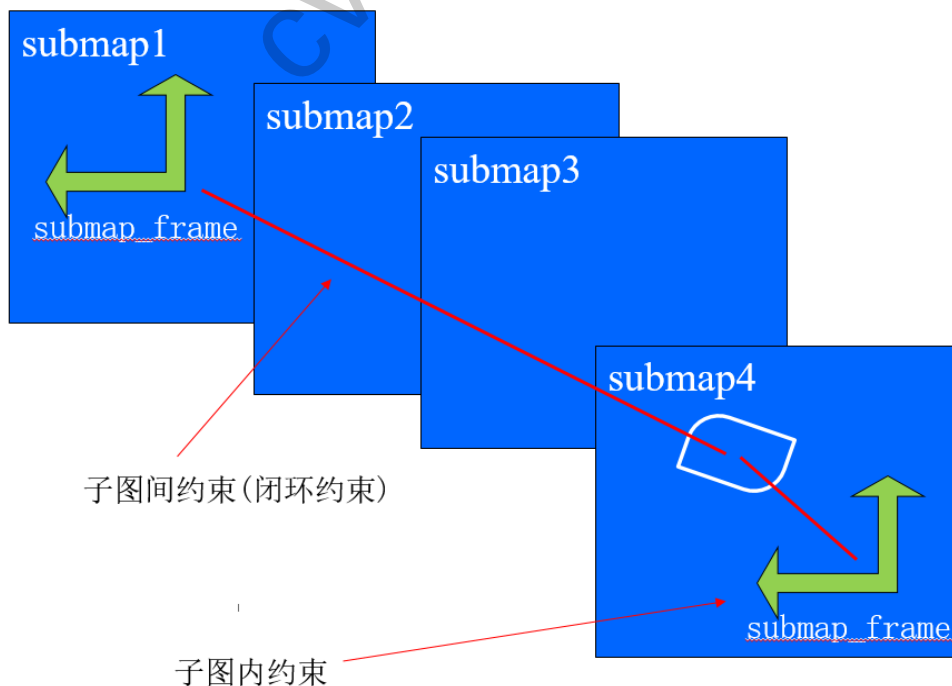
详见附图

10.3 添加节点相关函数

节点与约束的概念

节点: tracking_frame 的位姿, 子图原点的位姿

约束: tracking_frame 与子图原点间的坐标变换



ComputeLocalToGlobalTransform

这里更正一下: 视频里讲这个函数时说的是 后端优化的第一个节点的位姿是(0, 0, 0), 是错误的, 应该是

后端优化的 **global** 到 **local** 间的坐标变换, 如果没提前设置的话就是平移 0 与旋转 0.

下边的函数, 如果是后端的第一个节点, **GetLocalToGlobalTransform** 会返回(0, 0, 0), 然后还要乘以这个节点再 **local** 坐标系下的 **pose**, 乘的结果才是 **global** 坐标系下第一个节点的位姿.

C++

```
NodeId PoseGraph2D::AddNode(
    std::shared_ptr<const TrajectoryNode::Data> constant_data,
    const int trajectory_id,
    const std::vector<std::shared_ptr<const Submap2D>>&
    insertion_submaps) {

    // GetLocalToGlobalTransform
    const transform::Rigid3d optimized_pose(
        GetLocalToGlobalTransform(trajectory_id) * constant_data-
        >local_pose);
```

C++

```
transform::Rigid3d PoseGraph2D::ComputeLocalToGlobalTransform(
    const MapById<SubmapId, optimization::SubmapSpec2D>&
    global_submap_poses,
    const int trajectory_id) const {
    auto begin_it =
    global_submap_poses.BeginOfTrajectory(trajectory_id);
    auto end_it = global_submap_poses.EndOfTrajectory(trajectory_id);
    // 没找到这个轨迹 id
    if (begin_it == end_it) {
        const auto it =
    data_.initial_trajectory_poses.find(trajectory_id);
        // 如果设置了初始位姿
        if (it != data_.initial_trajectory_poses.end()) {
            return GetInterpolatedGlobalTrajectoryPose(it-
            >second.to_trajectory_id,
                                                    it->second.time) *
            it->second.relative_pose;
        }
        // note: 没设置初始位姿就将返回(0,0,0)的平移和旋转
    else {
```

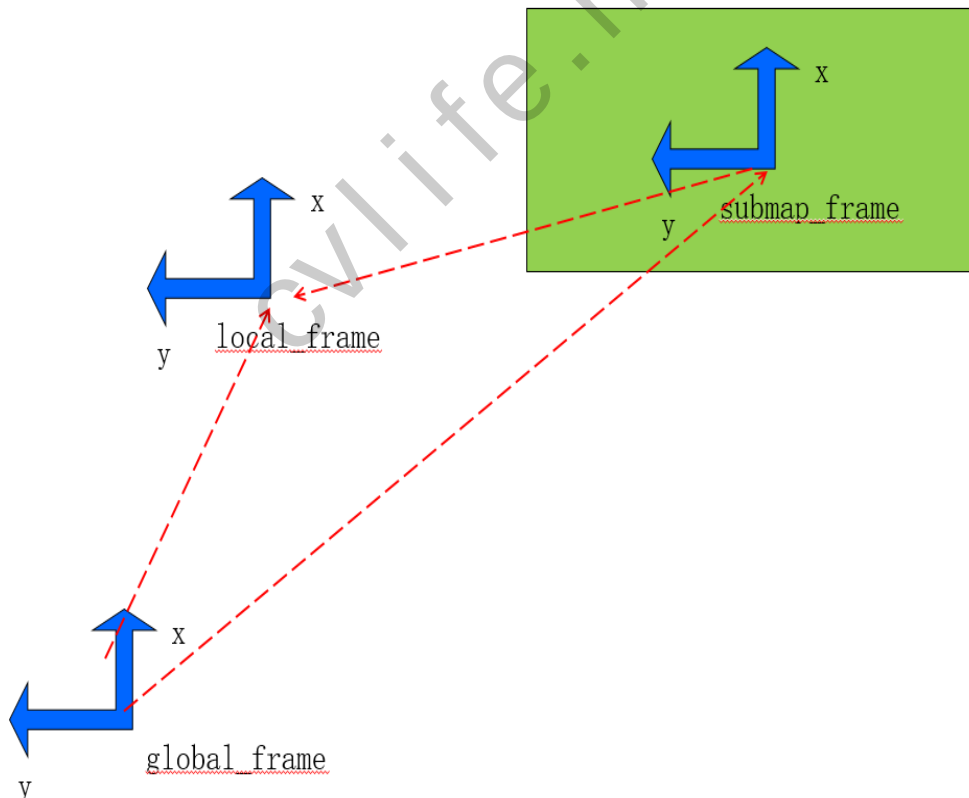
```

    return transform::Rigid3d::Identity();
}
}

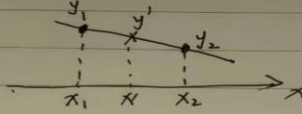
// 找到了就获取优化后的最后一个子图的 id
const SubmapId last_optimized_submap_id = std::prev(end_it)->id;
// Accessing 'local_pose' in Submap is okay, since the member is
const.
// 通过最后一个优化后的 global_pose * local_pose().inverse() 获取
global_pose->local_pose 的坐标变换
// tag: 画图说明一下
return transform::Embed3D(

global_submap_poses.at(last_optimized_submap_id).global_pose) *
    data_.submap_data.at(last_optimized_submap_id)
        .submap->local_pose()
        .inverse();
}

```



线性差值公式推导



已知 (x_1, y_1) (x_2, y_2) , 求 y'

直线方程: 两点式 $\frac{y-y_1}{x-x_1} = \frac{y_2-y_1}{x_2-x_1}$

$$(y-y_1) \cdot (x_2-x_1) = (x-x_1) \cdot (y_2-y_1)$$
$$y \cdot x_2 - y \cdot x_1 - y_1 x_2 + y_1 x_1 = x \cdot y_2 - x \cdot y_1 - x_1 y_2 + x_1 y_1$$
$$y(x_2-x_1) = y_1 x_2 + x y_2 - x y_1 - x_1 y_2$$
$$= (x_2-x) \cdot y_1 + (x-x_1) \cdot y_2$$
$$y = \frac{x_2-x}{x_2-x_1} \cdot y_1 + \frac{x-x_1}{x_2-x_1} \cdot y_2$$

$$y = \frac{x_2-x}{x_2-x_1} \cdot y_1 + \frac{x-x_1}{x_2-x_1} \cdot y_2$$
$$= \frac{x_2-x_1+x_1-x}{x_2-x_1} \cdot y_1 + \frac{x-x_1}{x_2-x_1} \cdot y_2$$
$$= y_1 + \frac{x_1-x}{x_2-x_1} \cdot y_1 + \frac{x-x_1}{x_2-x_1} \cdot y_2$$
$$= y_1 + \left(\frac{x-x_1}{x_2-x_1} \right) \cdot (y_2-y_1)$$

10.4 子图的保存与任务处理函数

第一次看到的子图的指针进行保存

data_.submap_data: 保存子图的指针

Submap1 num=91

Submap2 num=1

Submap1 num=180
设置为完成状态

Submap2 num=90

Submap1 删掉

Submap2 num=90

Submap3 num=0

InsertRangeData之后

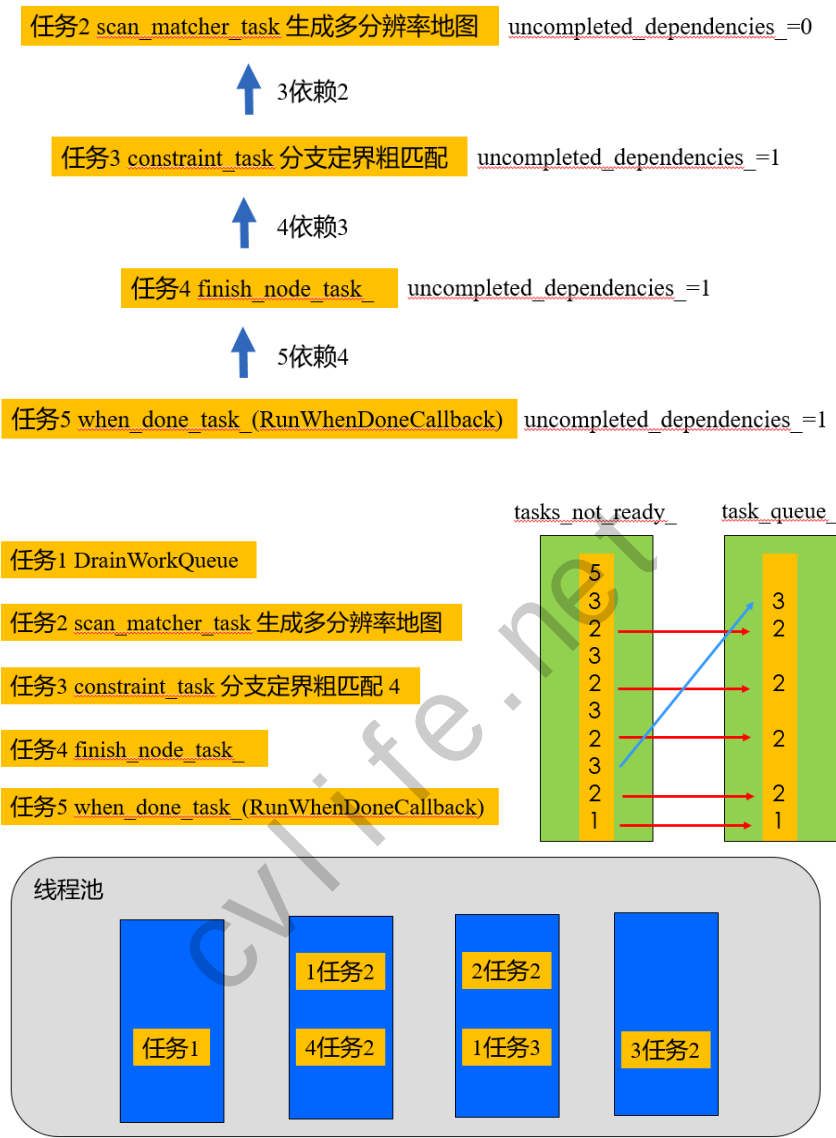
Submap2 num=91

Submap3 num=1

新看见的子图, 保存指针

10.5 线程池与任务调度

线程池



详见附图

第十一章 2D 后端优化（中）

详见附图

11.1 子图内约束的计算

子图内约束就是 属于这个 submap 的节点(是组成 submap 的节点之一)与这个 submap 坐标系原点间的坐标变换.

这个坐标变换是在 local 坐标系下, 由 submap 的坐标系原点指向 tracking_frame 的.

InitializeGlobalSubmapPoses

data_.global_submap_poses_2d: 全都是优化后的子图在 global 坐标系下的 pose

optimization_problem_->submap_data(): 包含了优化后和还没有进行优化的 子图在 global 坐标系下的 pose

ComputeLocalToGlobalTransform()这个函数的参数, 始终都是

data_.global_submap_poses_2d, 计算的是优化后的 global 指向 local 的坐标变换.

```
C++
transform::Rigid3d PoseGraph2D::ComputeLocalToGlobalTransform(
    const MapById<SubmapId, optimization::SubmapSpec2D>&
    global_submap_poses,
    const int trajectory_id) const {
    auto begin_it =
global_submap_poses.BeginOfTrajectory(trajectory_id);
    auto end_it = global_submap_poses.EndOfTrajectory(trajectory_id);
    const SubmapId last_optimized_submap_id = std::prev(end_it)->id;
    return transform::Embed3D(

global_submap_poses.at(last_optimized_submap_id).global_pose) *
        data_.submap_data.at(last_optimized_submap_id)
            .submap->local_pose()
            .inverse();
}

// 1 如果只有 1 个子图
optimization_problem_->AddSubmap(
    trajectory_id, transform::Project2D(
        ComputeLocalToGlobalTransform( // 会返回(0, 0, 0)
            data_.global_submap_poses_2d, trajectory_id) *
            insertion_submaps[0]->local_pose())));

// 2 有 2 个子图, 但是第二个子图没保存位姿的情况
const auto& first_submap_pose =
submap_data.at(last_submap_id).global_pose;
optimization_problem_->AddSubmap(
```

```

trajectory_id,
first_submap_pose *

constraints::ComputeSubmapPose(*insertion_submaps[0]).inverse() *
constraints::ComputeSubmapPose(*insertion_submaps[1]));

```

计算子图内约束

```

C++
const transform::Rigid2d local_pose_2d =
    transform::Project2D(constant_data->local_pose * // 三维转平面
                        transform::Rigid3d::Rotation(
                            constant_data-
>gravity_alignment.inverse())));

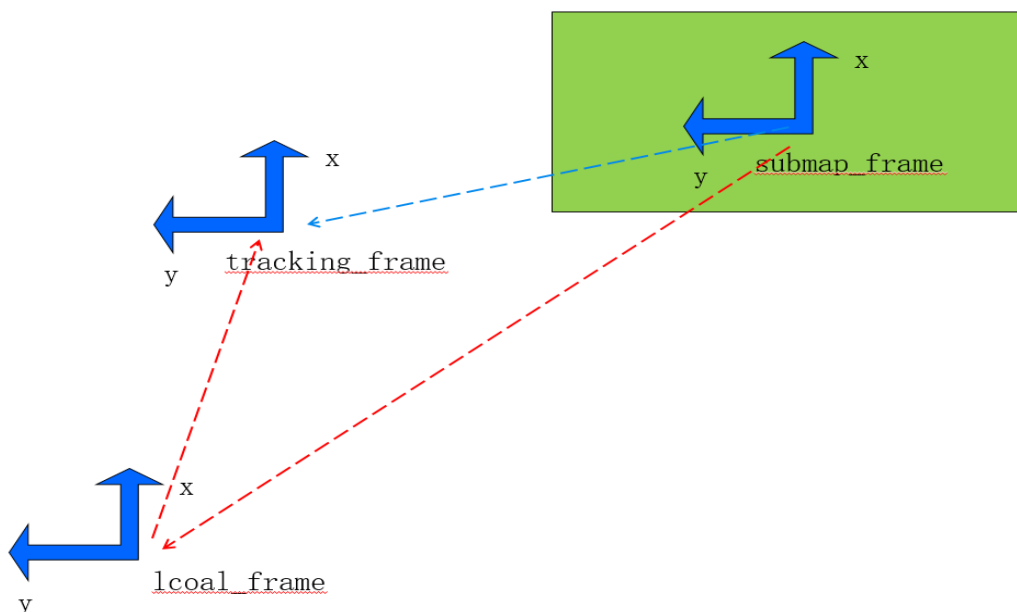
const transform::Rigid2d global_pose_2d =
    optimization_problem_->submap_data().at(matching_id).global_pose
*

constraints::ComputeSubmapPose(*insertion_submaps.front()).inverse()
*
    local_pose_2d;

const transform::Rigid2d constraint_transform =

constraints::ComputeSubmapPose(*insertion_submaps[i]).inverse() *
    local_pose_2d;

```



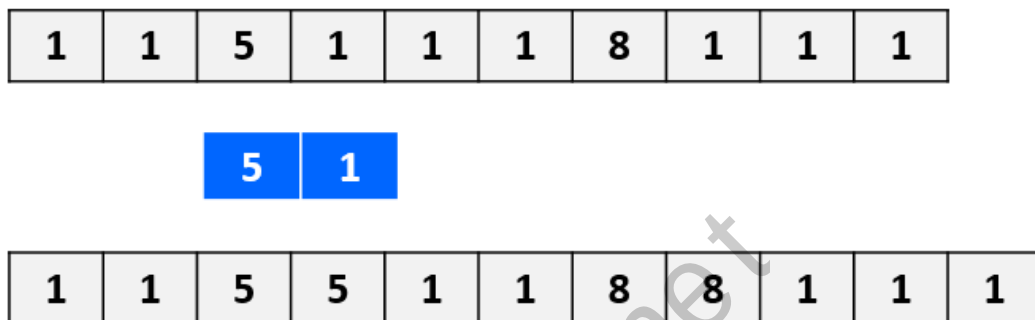
11.2 子图间约束的计算(回环检测)

子图间约束

子图间约束就是 不属于这个 submap 的节点与这个 submap 坐标系原点间的坐标变换.

与子图内约束是一样的, 也是在 local 坐标系下, 由这个 submap 的坐标系原点指向 tracking_frame 的.

11.3 使用滑动窗口算法生成多分辨率栅格地图



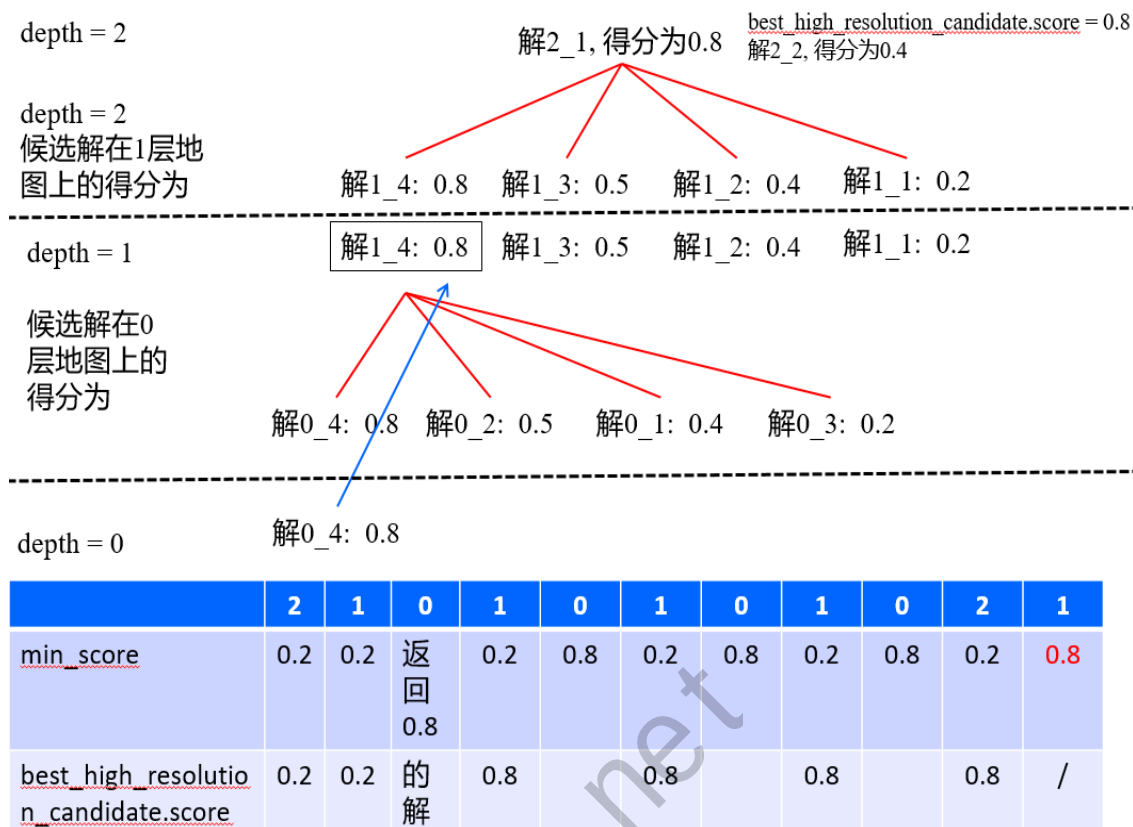
11.4 滑动窗口算法编程实践

第三次编程作业

11.5 基于多分辨率地图的分支定界粗匹配

分支定界算法

- **分支:** 对当前层候选解进行分支(扩充), 生成下一层分辨率地图上的 4 个候选解
- **排序:** 对下一层分辨率地图上的 4 个候选解进行打分并**降序排序**
- **定界:** 将当前层的最高得分, 当做下一次分支定界算法的分数阈值
- **剪枝:** 只要当前层的候选解的得分, 有小于传入的阈值的, 就 **break**, 因为是排好序的



第十二章 2D 后端优化（下）

12.1 优化时的函数调用流程分析

详见视频

12.2 后端计算位姿的方法总结

12.2.1 向优化问题中添加数据

添加节点数据

```
C++
optimization_problem_->AddTrajectoryNode(
    matching_id.trajectory_id,
    optimization::NodeSpec2D{constant_data->time, local_pose_2d,
        global_pose_2d,
        constant_data->gravity_alignment});
```

添加子图坐标原点数据

```
C++
optimization_problem_->AddSubmap(
    trajectory_id,
    first_submap_pose *
    constraints::ComputeSubmapPose(*insertion_submaps[0]).inverse() *
    constraints::ComputeSubmapPose(*insertion_submaps[1]));
```

添加其他传感器数据

```
C++
optimization_problem_->AddImuData(trajectory_id, imu_data);
optimization_problem_->AddOdometryData(trajectory_id,
    odometry_data);
optimization_problem_->AddFixedFramePoseData(trajectory_id,
    fixed_frame_pose_data);
```

12.2.2 几种计算相对位姿的方式

节点在 global 坐标系下的位姿

```
C++
const transform::Rigid3d optimized_pose(
    GetLocalToGlobalTransform(trajectory_id) * constant_data->
    local_pose);
```

```
C++
ComputeLocalToGlobalTransform() // 传入的始终是
data_.global_submap_poses_2d
data_.global_submap_poses_2d // 只有在第一次优化完之后才有数据
```

节点在 global 坐标系下的第一帧的位姿, 就是这个节点在 local 坐标系下的位姿.

之后的节点在 global 坐标系下的位姿 是通过 global 到 local 的坐标变换乘以 local 坐标系下的位姿得到的.

submap 在 global 坐标系下的位姿

```
C++
optimization_problem_->AddSubmap(
    trajectory_id, transform::Project2D(
        ComputeLocalToGlobalTransform(
            data_.global_submap_poses_2d, trajectory_id) *
            insertion_submaps[0]->local_pose()));

optimization_problem_->AddSubmap(
    trajectory_id,
    first_submap_pose *
        constraints::ComputeSubmapPose(*insertion_submaps[0]).inverse()
    *
        constraints::ComputeSubmapPose(*insertion_submaps[1]));
```

子图在 global 坐标系下的第一帧的位姿,就是这个子图在 local 坐标系下的位姿.

之后的子图在 global 坐标系下的位姿 是通过 第一个子图在 global 坐标系下的 pose 乘以 第一个子图到第二个子图在 local 坐标系下的位姿变换 得到的.

子图内约束

local 坐标系下, 子图原点指向 tracking_frame 的坐标变换

```
C++
const transform::Rigid2d local_pose_2d =
    transform::Project2D(constant_data->local_pose * // 三维转平面
        transform::Rigid3d::Rotation(
            constant_data->gravity_alignment.inverse()));

// 计算 子图原点 指向 node 坐标 间的坐标变换(子图内约束)
const transform::Rigid2d constraint_transform =
    constraints::ComputeSubmapPose(*insertion_submaps[i]).inverse() *
    local_pose_2d;
```

子图间约束

先通过子图在 global 坐标系下的坐标的逆, 乘以节点在 global 坐标系下的坐标, 获取子图原点在 global 坐标系下指向节点的相对坐标变换.

然后根据子图在 local 坐标系下的位姿乘以这个坐标变换, 得到节点在 local 坐标系下的预测位姿, 在通过分枝定界粗匹配与 ceres 的精匹配, 对这个节点位姿进行校准, 校准后的位姿还是 local 坐标系下的.

最后, 通过子图在 local 坐标系下位姿的逆, 乘以这个节点校准后的位姿, 得到子图间约束, 是 local 坐标系下, 子图原点指向节点的相对坐标变换.

```
C++
const transform::Rigid2d initial_relative_pose =
    optimization_problem_ -
    >submap_data().at(submap_id).global_pose.inverse() *
    optimization_problem_ ->node_data().at(node_id).global_pose_2d;

const transform::Rigid2d initial_pose =
    ComputeSubmapPose(*submap) * initial_relative_pose;

const transform::Rigid2d constraint_transform =
    ComputeSubmapPose(*submap).inverse() * pose_estimate;
```

4 种计算相对位姿的方式总结

1. 节点 通过 `GetLocalToGlobalTransform * constant_data->local_pose` 进行 global 下位姿的计算
2. 子图 通过对前一个子图到后一个子图的坐标变换进行累计, 得到子图在 global 坐标系下的位姿
3. 子图内约束 local 坐标系下, 子图原点指向节点间的坐标变换
4. 子图间约束 根据 global 坐标计算初值, 然后通过分支定界算法粗匹配与 ceres 的精匹配, 获取校准后的位姿, 最后计算 local 坐标系下, 子图原点指向校准后的节点间的坐标变换

12.3 第一种残差的构建

将节点与子图原点在 global 坐标系下的相对位姿 与 约束 的差值作为残差项

- 第一种坐标变换: 节点与子图原点在 global 坐标系下的坐标变换
- 第二种坐标变换: 子图内约束与子图间约束

12.4 第二种残差的构建-Landmark 残差

landmark 数据 与 通过 2 个节点位姿插值出来的相对位姿 的差值作为残差项

- 第一种坐标变换: landmark 数据的时间在 2 个节点位姿中插值出来的位姿
- 第二种坐标变换: landmark 数据中的 `landmark_to_tracking_transform_`

```

C++
// 第一种坐标变换
// start:
    const std::tuple<std::array<T, 4>, std::array<T, 3>>
        interpolated_rotation_and_translation = InterpolateNodes2D(
            prev_node_pose, prev_node_gravity_alignment_,
next_node_pose,
            next_node_gravity_alignment_, interpolation_parameter_);
// end:
    const std::tuple<std::array<double, 4>, std::array<double, 3>>
        rotation_and_translation =
            InterpolateNodes2D(prev_node_pose.data(),
prev_node.gravity_alignment,
                                next_node_pose.data(),
next_node.gravity_alignment,
                                interpolation_parameter_);
    // 将 landmark 的数据从 tracking_frame 下的位姿转到 global 坐标系下
    return
transform::Rigid3d::FromArray<std::get<0>(rotation_and_translation)
,

std::get<1>(rotation_and_translation)>> *
        observation.landmark_to_tracking_transform;
// 第二种坐标变换
landmark_to_tracking_transform_

// 在初始时刻这 2 个约束计算的结果是一样的, 直到
// landmark_node.second.global_landmark_pose.has_value 为 true 时

const transform::Rigid3d starting_point =
    andmark_node.second.global_landmark_pose.has_value()
        ? landmark_node.second.global_landmark_pose.value()
        : GetInitialLandmarkPose(observation, prev->data, next-
>data,
            *prev_node_pose, *next_node_pose);

// 更新 data_.landmark_nodes
for (const auto& landmark : optimization_problem_->landmark_data())
{
    data_.landmark_nodes[landmark.first].global_landmark_pose =
landmark.second;
}

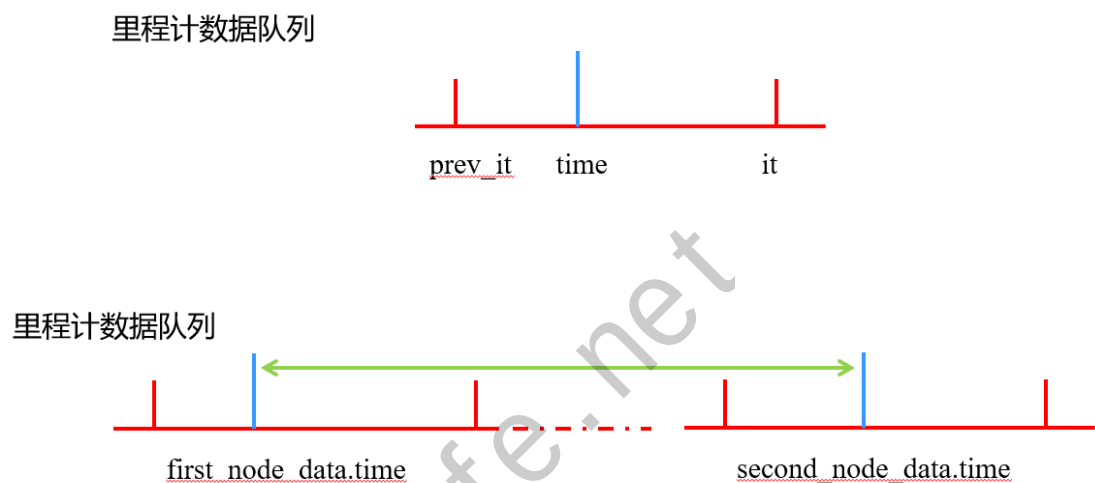
```


12.5 其余三种残差的构建

第三种残差

节点与节点间在 **global** 坐标系下的相对坐标变换 与 通过里程计数据插值出的相对坐标变换 的差值作为残差项

- 第一种坐标变换: 相邻 2 个节间在 **global** 坐标系下的坐标变换
- 第二种坐标变换: 再分别计算这 2 个节点的时间在里程计数据队列中插值得到的 2 个里程计位姿, 计算这 2 个里程计位姿间的坐标变换



第四种残差

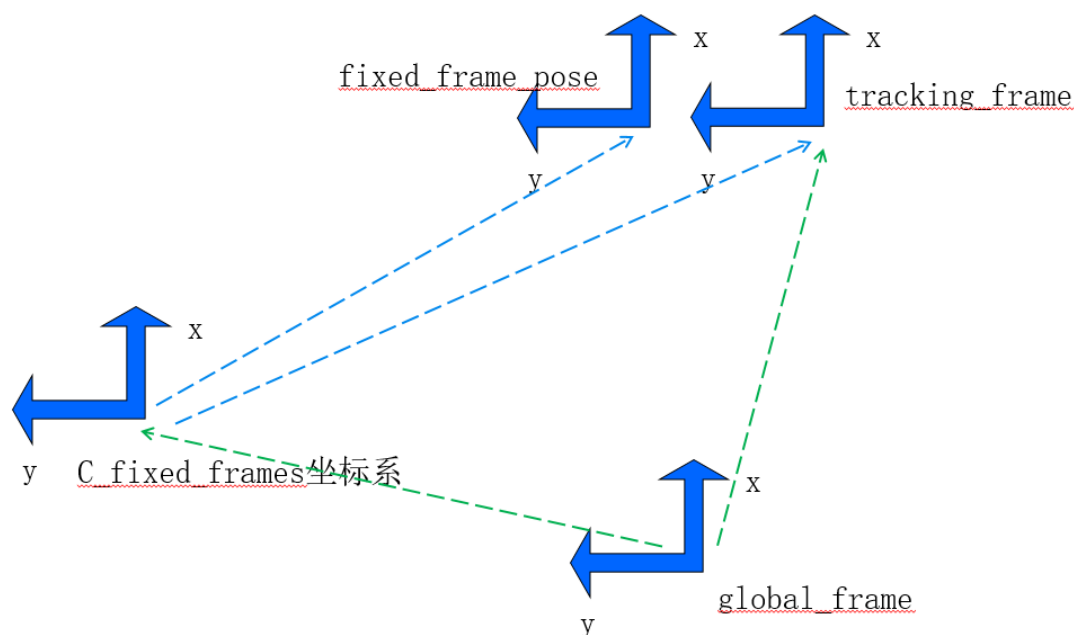
节点与节点间在 **global** 坐标系下的相对坐标变换 与 相邻 2 个节点在 **local** 坐标系下的相对坐标变换 的差值作为残差项

- 第一种坐标变换: 相邻 2 个节间在 **global** 坐标系下的坐标变换
- 第二种坐标变换: 相邻 2 个节点在 **local** 坐标系下的坐标变换

第五种残差

节点与 **gps** 坐标系原点在 **global** 坐标系下的相对坐标变换 与 通过 **gps** 数据进行插值得到的相对坐标变换 的差值作为残差项

- 第一种坐标变换: 节点对应的时刻在 **gps** 数据中插值得到的 **gps** 相对于 **gps** 坐标系原点的位姿
- 第二种坐标变换: 节点在 **global** 坐标系下 与 **gps** 坐标系原点 的相对坐标变换



12.6 使用 Ceres 进行全局优化求解

详见附图

12.7 后端优化部分的总结

12.7.1 优化后位姿的更新

```
C++
for (const auto& C_submap_id_data : C_submaps) {
    submap_data_.at(C_submap_id_data.id).global_pose =
        ToPose(C_submap_id_data.data);
}
for (const auto& C_node_id_data : C_nodes) {
    node_data_.at(C_node_id_data.id).global_pose_2d =
        ToPose(C_node_id_data.data);
}
for (const auto& C_fixed_frame : C_fixed_frames) {
    trajectory_data_.at(C_fixed_frame.first).fixed_frame_origin_in_map =
        transform::Embed3D(ToPose(C_fixed_frame.second));
}
for (const auto& C_landmark : C_landmarks) {
    landmark_data_[C_landmark.first] = C_landmark.second.ToRigid();
}
```

12.7.2 优化前后的不变量与变化量

不变量

- 第一帧子图在 global 坐标系下的位姿不会被优化所改变

```
C++
problem.SetParameterBlockConstant(C_submaps.at(submap_id_data.id).data());
```

由于优化前后第一个子图原点在 global 坐标系下的位姿是不会发生变化的, 所以 global 坐标系是不会改变的

- 冻结轨迹的节点 pose, 子图 pose, landmark 的 pose 是不变的

变化量

- 除第一个子图外的其他子图的 global 坐标系下的 pose
- 所有节点在 global 坐标系下的 pose
- 所有 landmark 数据在 global 坐标系下的 pose
- gps 第一帧数据在 global 坐标系下的 pose

12.7.3 优化的贡献

- 第一个贡献 优化了位姿: 子图, 节点, landmark, gps 第一帧坐标
- 第二个贡献 预测了没有优化的节点位姿
- 第三个贡献 ComputeLocalToGlobalTransform

```
C++
NodeId PoseGraph2D::AddNode(
    std::shared_ptr<const TrajectoryNode::Data> constant_data,
    const int trajectory_id,
    const std::vector<std::shared_ptr<const Submap2D>>&
insertion_submaps) {
    // 将节点在 local 坐标系下的坐标转成 global 坐标系下的坐标
    const transform::Rigid3d optimized_pose(
        GetLocalToGlobalTransform(trajectory_id) * constant_data->local_pose);
```

ComputeLocalToGlobalTransform() 函数是计算最后一个子图原点的 global 坐标指向 local 坐标的坐标变换, 为什么要最后一个是子图的坐标呢?

local 坐标系下的位姿都是相对与前一个节点的相对位姿, 时间越短, 相对位姿越准。

优化后的最后一个子图位姿, 与当前节点的位姿在时间上距离最近.

所以, 最后的一个优化后的子图位姿的相对坐标变换, 在时间上最接近当前节点与当前子图, 时间越短, 乘以相对坐标变换后得到的节点或者子图在 global 坐标系下的位姿越接近真实值.

- 第四个贡献 MapBuilderBridge

```
C++
MapBuilderBridge::GetSubmapList()
    map_builder_->pose_graph()->GetAllSubmapPoses();

MapBuilderBridge::GetLocalTrajectoryData()
    map_builder_->pose_graph()-
>GetLocalToGlobalTransform(trajectory_id),

MapBuilderBridge::HandleTrajectoryQuery
    map_builder_->pose_graph()->GetTrajectoryNodePoses()

MapBuilderBridge::GetTrajectoryNodeList()
    map_builder_->pose_graph()->GetTrajectoryNodePoses()
    map_builder_->pose_graph()->constraints()

MapBuilderBridge::GetLandmarkPosesList()
    map_builder_->pose_graph()->GetLandmarkPoses()

MapBuilderBridge::GetConstraintList()
    pose_graph()->GetTrajectoryNodePoses()
    map_builder_->pose_graph()->GetAllSubmapPoses()
    map_builder_->pose_graph()->constraints()
```

所以, 后端优化后的结果是与前端无关的.

这也解释了为什么 carto 官方文档说的是前端坐标系是不会变化的, 就是由于后端的結果不会作用在前端上.

第十三章 SLAM 的结果输出

13.1 优化后结果的传递与处理

MapBuilderBridge 类

前端中的数据

通过 OnLocalSlamResult()函数将前端匹配的结果保存到 local_slam_data_中.

```
C++  
void MapBuilderBridge::OnLocalSlamResult()
```

后端中的数据

```
C++  
map_builder_->pose_graph()->GetTrajectoryStates()    // 获取轨迹状态  
map_builder_->pose_graph()->GetAllSubmapPoses()      // 获取所有  
submap 的信息  
map_builder_->pose_graph()->IsTrajectoryFrozen()     // 获取轨迹  
的冻结状态  
  
map_builder_->pose_graph()->GetLocalToGlobalTransform(trajectory_id)  
// 获取最新的优化后的 local frame 到 global frame 间的坐标变换  
map_builder_->pose_graph()->GetTrajectoryNodePoses() // 获取轨迹的所有位姿  
map_builder_->pose_graph()->constraints()           // 获取所有的约束  
map_builder_->pose_graph()->GetLandmarkPoses()      // 获取 landmark  
poses
```

Node 类

Node 类使用前端数据的函数

```
C++  
Node::PublishLocalTrajectoryData  
// 发布点云与 tf
```

Node 类使用后端数据的函数

```
C++  
Node::PublishLocalTrajectoryData  
// 发布点云与 tf  
Node::PublishTrajectoryNodeList()  
// 可视化所有的节点位姿  
Node::PublishLandmarkPosesList()  
// 可视化所有的 landmark 位姿  
Node::PublishConstraintList  
// 可视化所有的约束
```

13.2 TF 的发布

详见视频

13.3 轨迹与约束的可视化

详见视频

13.4 ROS 地图的发布

Cairo 简介

Cairo 是非常流行的开源 2D 图形渲染引擎库，它支持包括 X-Windows, Win32, 图像, pdf 在内的各种输出设备。目前, Cairo 已被广泛的使用在多个平台上来渲染图形界面, 包括 Firefox/Webkit-EFL/GTK+/Poppler 等等。

学习资料

Cairo 官网 <https://www.cairographics.org/tutorial/>
cairo_t 相关函数说明(API) <https://www.cairographics.org/manual/cairo-cairo-t.html>
Image Surfaces 相关函数说明(API) <https://cairographics.org/manual/cairo-Image-Surfaces.html>

扩展资料

Cairo 学习 (一) <http://diff3.com/f255/>
Cairo 图形指南 [https://blog.csdn.net/haiwil/category_833019.html](

13.5 结束轨迹时的处理

详见视频

13.6 Pbstream 文件的保存

详见视频

第十四章 纯定位模式与 3D 建图

14.1 纯定位模式函数调用流程分析

```
C++
// node_main.cc
if (!FLAGS_load_state_filename.empty()) {
    node.LoadState(FLAGS_load_state_filename, FLAGS_load_frozen_state);
}

// node.cc
void Node::LoadState(const std::string& state_filename, const bool
load_frozen_state) {
    absl::MutexLock lock(&mutex_);
    map_builder_bridge_.LoadState(state_filename, load_frozen_state);
    load_state_ = true;
}

// map_builder_bridge.cc
void MapBuilderBridge::LoadState(const std::string& state_filename,
bool load_frozen_state) {
    // Check if suffix of the state file is ".pbstream".
    const std::string suffix = ".pbstream";

    // 检查后缀是否是.pbstream
    CHECK_EQ(state_filename.substr(
        std::max<int>(state_filename.size() - suffix.size(), 0)), suffix)
        << "The file containing the state to be loaded must be a "
        ".pbstream file.";
    LOG(INFO) << "Loading saved state '" << state_filename << "'...";
    // 加载文件内容
    cartographer::io::ProtoStreamReader stream(state_filename);
    // 解析数据
    map_builder_->LoadState(&stream, load_frozen_state);
}

// map_builder.cc
// 构造时候初始化 pose_graph_, 之后在重定位时候才能向 pose_graph_中添加数
据
std::map<int, int> MapBuilder::LoadState(
    io::ProtoStreamReaderInterface* const reader, bool
load_frozen_state) {
    // 从文件中添加轨迹
```

```

// 更新约束中节点与子图的轨迹 id
// 从获取到的位姿图中生成 submap_poses
// 从获取到的位姿图中生成 node_poses
// 将 landmark_poses 添加到位姿图中
// 向 pose_graph_中添加信息
// 添加子图的附属的节点
}

// node.cc
if (FLAGS_start_trajectory_with_default_topics) {
    node.StartTrajectoryWithDefaultTopics(trajectory_options);
}

// pose_graph_2d.cc 优化之后进行子图的裁剪
{
    TrimmingHandle trimming_handle(this);
    // 进行子图的裁剪, 如果没有裁剪器就不裁剪
    for (auto& trimmer : trimmers_) {
        trimmer->Trim(&trimming_handle); //
PureLocalizationTrimmer::Trim()
    }
    // 如果裁剪器处于完成状态, 就把裁剪器删除掉
    trimmers_.erase(
        // c++11: std::remove_if 如果回调函数函数返回真, 则将当前所指向的参数
        // 移到尾部, 返回值是被移动区域的首个元素
        std::remove_if(trimmers_.begin(), trimmers_.end(),
            [](std::unique_ptr<PoseGraphTrimmer>& trimmer) {
                return trimmer->IsFinished(); // 调用
PureLocalizationTrimmer::IsFinished()
            })),
        trimmers_.end());
}

// 裁剪子图与节点, 及相关的约束
void PoseGraph2D::TrimmingHandle::TrimSubmap(const SubmapId&
submap_id) {
    // 获取除 submap_id 外的所有子图的所有节点的 id(nodes_to_retain), 这些节
    // 点是需要保留的
    // 找到在 submap_id 的子图内部同时不在别的子图内的节点
    (nodes_to_remove), 这些节点需要删除
    // 删除 submap_id 相关的约束
    // 删除与 nodes_to_remove 中节点相关联的约束, 并对对应的 submap_id 进行标

```


记

```
// 对标记的 submap_id 进行检查，看是否还存在其他子图间约束
// 删除没有子图间约束的标记的子图的扫描匹配器
// 删除 submap_id 这个子图的指针
// 删除 submap_id 这个子图的匹配器，与多分辨率地图
// 删除 optimization_problem_ 中的 submap_id 这个子图
// 删除 nodes_to_remove 中的节点
}
```

14.2 Pbstream 文件的加载

详见视频

14.3 子图的修剪

详见视频

14.4 TSDF 地图简介

TSDF 地图

推荐 2 篇文章

- TSDF 算法简述
<https://zhuanlan.zhihu.com/p/390276710>
- TSDF 算法学习
- <https://blog.csdn.net/zfjBIT/article/details/104648505>

TSDF 地图与 ProbabilityGrid 地图的区别

TSDF2D 类继承了 Grid2D 类

具体的栅格值保存在 Grid2D 里的 `correspondence_cost_cells_` 中，只不过这里保存的不再是空闲的概率了。而是 tsd 值转成的 value。

C++

```
ProbabilityGrid::ProbabilityGrid(const MapLimits& limits,
                                ValueConversionTables*
                                conversion_tables)
    : Grid2D(limits, kMinCorrespondenceCost, kMaxCorrespondenceCost,
              conversion_tables),
      conversion_tables_(conversion_tables) {}
```

```

/**
 * @brief 构造函数
 *
 * @param[in] limits 地图坐标信息
 * @param[in] truncation_distance 0.3
 * @param[in] max_weight 10.0
 * @param[in] conversion_tables 转换表
 */
TSDF2D::TSDF2D(const MapLimits& limits, float truncation_distance,
               float max_weight, ValueConversionTables*
               conversion_tables)
    : Grid2D(limits, -truncation_distance, truncation_distance,
             conversion_tables),
      conversion_tables_(conversion_tables),
      value_converter_(abs1::make_unique<TSDValueConverter>(
          truncation_distance, max_weight, conversion_tables_)),
      weight_cells_(
          limits.cell_limits().num_x_cells *
limits.cell_limits().num_y_cells,
          value_converter_->getUnknownWeightValue()) {}

```

可以看到, ProbabilityGrid 地图的栅格值的最大最小分别是 0.9 与 0.1, 而 TSDF 地图的上遏制的最大最小分别是 0.3 与 -0.3.

TSDF 地图保存 tsd 值的同时还保存了权重值, 权重值保存在 TSDF2D 类的 weight_cells_ 中.

获取 TSDF 地图栅格值是通过 TSDF2D::GetTSDAndWeight 获取栅格值的, 同时获取到 TSD 值与权重值.

栅格值更新的方式

新的权重 = 之前的 weight + 新的 weight

新的 tsd 值 = (之前的 tsd 值 * 之前的 weight + 新的 tsd 值 * 新的 weight) / (新的权重)

```

C++
// TSDF 地图栅格的更新, 分别更新 tsd 值与权重值
void TSDFRangeDataInserter2D::UpdateCell(const Eigen::Array2i& cell,
                                          float update_sdf, float
                                          update_weight,
                                          TSDF2D* tsdf) const {
    if (update_weight == 0.f) return;
    // 获取 TSD 值与权重值
    const std::pair<float, float> tsd_and_weight = tsdf-

```

```

>GetTSDAndWeight(cell);
    float updated_weight = tsd_and_weight.second + update_weight;
    float updated_sdf = (tsd_and_weight.first * tsd_and_weight.second
+
                        update_sdf * update_weight) /
                        updated_weight;
    updated_weight =
        std::min(updated_weight,
static_cast<float>(options_.maximum_weight()));
    tsdf->SetCell(cell, updated_sdf, updated_weight);
}

```

相关性扫描匹配时使用 TSDF 计算得分

```

C++
float ComputeCandidateScore(const TSDF2D& tsdf,
                           const DiscreteScan2D& discrete_scan,
                           int x_index_offset, int y_index_offset)
{
    float candidate_score = 0.f;
    float summed_weight = 0.f;
    for (const Eigen::Array2i& xy_index : discrete_scan) {
        const Eigen::Array2i proposed_xy_index(xy_index.x() +
x_index_offset,
                                                xy_index.y() +
y_index_offset);
        const std::pair<float, float> tsd_and_weight =
            tsdf.GetTSDAndWeight(proposed_xy_index);
        const float normalized_tsd_score =
            (tsdf.GetMaxCorrespondenceCost() -
std::abs(tsd_and_weight.first)) /
            tsdf.GetMaxCorrespondenceCost();
        const float weight = tsd_and_weight.second;
        candidate_score += normalized_tsd_score * weight;
        summed_weight += weight;
    }
    if (summed_weight == 0.f) return 0.f;
    candidate_score /= summed_weight;
    CHECK_GE(candidate_score, 0.f);
    return candidate_score;
}

```

TSDF 地图的扫描匹配

InterpolatedTSDF2D

CreateTSDFMatchCostFunction2D

14.5 3D 网格地图

3D 扫描匹配

HybridGridBase 相当于 Grid2D

HybridGrid 相当于 ProbabilityGrid

就是用三维网格替换了二维网格, 其余是差不多的.

和 2D 不同的是, 地图里保存的是 odd, 而不是 costodd, 即是占用的概率, 而不是 miss 的概率.

14.6 3D 扫描匹配

LocalTrajectoryBuilder3D

点云的处理之前讲过了, 位姿预测器是一样的. 大部分是一样的, 不一样的点在下边.

RealTimeCorrelativeScanMatcher3D

首先分别对 xyz 与 绕 xyz 的旋转 这 6 个维度进行遍历, 生成所有的可能解
对所有的可能解进行打分, 选出最高分的解

CeresScanMatcher3D

基本上与 2D 是一样的, 只是地图这的残差变了.

OccupiedSpaceCostFunction3D

地图变成 2 个了, 一个高分辨率地图, 一个低分辨率地图.

地图残差的计算基本也是一样的, 就是拿点云对应的栅格值当做残差, 只不过作为残差的是 $(1. - \text{probability})$.

InterpolatedGrid

手动实现了双三次插值

将点云插入到三维网格地图里

旋转直方图

首先计算了点云的旋转直方图, 子图的旋转直方图就是点云的旋转直方图的累加.

表征点云与地图的角度特征.

推荐 2 篇文章

- cartographer 3D scan matching 理解
<https://www.cnblogs.com/mafuqiang/p/10885616.html>
- Cartographer 源码阅读 3D-Submap 创建
<https://blog.csdn.net/yeluohanchan/article/details/109462508?spm=1001.2014.3001.5501>

插入器 RangeDataInserter3D

将点云分别插入到低分辨率地图与高分辨率地图中。

14.7 3D 后端优化

PoseGraph3D

基本一样, 只不过位姿是 6 维的了, 不需要再去与重力对齐向量相乘了, 直接获取 local_pose, 不用进行旋转变换了。

ConstraintBuilder3D

基本一样, 只不过调用的是 FastCorrelativeScanMatcher3D.

FastCorrelativeScanMatcher3D

构造

构造的时候需要传入高分辨率地图与低分辨率地图, 以及地图对应的旋转直方图。

之后将高分辨率地图弄成多分辨率地图, 保存低分辨率地图与旋转直方图。

构造时

匹配

生成了一个低分辨率地图的匹配器。

在离散点云的时候通过旋转直方图匹配滤掉了一部分候选 scan。

分枝定界算法

每一个候选解生成 8 个候选解, 3 个方向。

第 0 层的时候将叶节点与低分辨率地图再进行匹配一下, 得分大于阈值才能返回。

OptimizationProblem3D

根据参数选择是否对节点与子图的 pose 的 z 坐标进行优化

优化时第一个子图固定了 xyz, 旋转固定了 yaw, 只优化绕 xy 的旋转, 因为绕 xy 的旋转可以通过重力的方向进行约束。

由于旋转是通过四元数表示, 所以在 ceres 中添加了 QuaternionParameterization, 以对四元数进行更新.

多了根据 imu 计算的残差, 分为加速度的残差与旋转的残差

其余的残差基本一样.

第十五章 总结与工程化建议

15.1 代码的全面总结

详见视频

15.2 Cartographer 的优缺点分析

15.2.1 优点

代码架构十分优美

各个模块独立性很强, 可以很方便的进行修改, 或则是单独拿出来做其他应用

代码鲁棒性非常高, 很少出现莫名崩掉的情况, 错误提示很好

代码命名非常规范, 能够清楚的根据变量名与函数名判断其代表的含义

总之, cartographer 的代码十分值得学习与借鉴.

15.2.2 缺点

点云的预处理

发生的拷贝次数太多

自适应体素滤波如果参数不好时计算量太大

位姿推测器

可能有问题的点

- 计算 pose 的线速度与角速度时, 是采用的数据队列开始和末尾的 2 个数据计算的
- 计算里程计的线速度与角速度时, 是采用的数据队列开始和末尾的 2 个数据计算的
- 使用里程计, 不使用 imu 时, 计算里程计的线速度方向和姿态的预测时, 用的是里程计数据队列开始和末尾的 2 个数据的平均角速度计算的, 时间长了就不准
- 不使用里程计, 不使用 imu 时, 用的是 pose 数据队列开始和末尾的 2 个数据的平均角速度计算的, 时间长了就不准
- 添加位姿时, 没有用 pose 的姿态对 imu_tracker_进行校准, 也没有对整体位姿预

测器进行校准, 只计算了 pose 的线速度与角速度

- 从代码上看, cartographer 认为位姿推测器推测出来的位姿与姿态是准确的

可能的改进建议

- pose 的距离越小, 匀速模型越能代替机器人的线速度与角速度, 计算 pose 的线速度与角速度时, 可以考虑使用最近的 2 个数据进行计算
- 里程计距离越短数据越准, 计算里程计的线速度与角速度时, 可以考虑使用最近的 2 个数据进行计算
- 使用里程计, 不使用 imu 时, 计算里程计的线速度方向时, 可以考虑使用里程计的角度进行计算
- 使用里程计, 不使用 imu 时, 进行姿态的预测时, 可以考虑使用里程计的角度进行预测
- 不使用里程计, 不使用 imu 时, 可以考虑用最近的 2 个 pose 计算线速度与角速度
- 使用 pose 对 imu_tracker_ 的航向角进行校准

基于 Ceres 的扫描匹配

可能有问题的点

- 平移和旋转的残差项是逼近于先验位姿的, 当先验位姿不准确时会产生问题

可能的改进建议

- 先将地图的权重调大, 平移旋转的权重调小, 如 1000, 1, 1, 或者 100, 1, 1
- 调参没有作用的时候可以将平移和旋转的残差项注释掉

后端优化

优化时的计算量太大, 可以根据自己需求调整参数, 或者增加计算前的过滤.

在计算子图间约束的时候, 目前 cartographer 是根据节点个数来做的, 定位时又根据时间来决定是否进行全子图的匹配, 这部分计算的判断可以根据自己的需求增加一些, 以减少计算量.

15.3 工程化建议

15.3.1 工程化的目的

根据机器人的**传感器硬件**, 最终能够实现**稳定地**构建一张**不叠图**的二维栅格地图.

由于 cartographer 代码十分的优秀, 所以 cartographer 的稳定性目前已经很好了, 比目前的大部分 slam 的代码都稳定, 很少会出现崩掉的情况, 最多就是会由于某些原因提示错误.

15.3.2 如何提升建图质量

最简单的一种方式, 选择好的传感器. 选择频率高(25hz 以上), 精度高的雷达, 精度高的 imu, 这样的传感器配置下很难有建不好的地图.

如果只能用频率低的雷达呢

由于频率低时的叠图基本都是在旋转时产生的, 所以推荐使用一个好的 imu, 然后建图的时候让机器人的**移动与旋转速度慢一点**(建图轨迹与建图速度十分影响建图效果), 这时候再看建图效果.

如果效果还不行, 调 ceres 的匹配权重, 将地图权重调大, 平移旋转权重调小.

如果效果还不行, 可以将代码中平移和旋转的残差注释掉.

如果效果还不行, 那就得改代码了, 去改位姿推测器那部分的代码, 让预测的准一点.

里程计

为什么一直没有说里程计, 就是由于 cartographer 中对里程计的使用不太好.

cartographer 中对里程计的使用有 2 部分, 一个是前端的位姿推测器, 一个是后端根据里程计数据计算残差. 后端部分的使用是没有问题的.

如果想要在 cartographer 中使用里程计达到比较好的效果, 前端的位姿推测器这部分需要自己重写.

可以将 karto 与 gmapping 的使用里程计进行预测的部分拿过来进行使用, 改完了之后就能够达到比较好的位姿预测效果了.

粗匹配

cartographer 的扫描匹配中的粗匹配是一种暴力匹配的方法, 目的是对位姿预测出的位姿进行校准, 但是这个扫描匹配的计算量太大了, 导致不太好用.

这块可以进行改进, 可以将 karto 的扫描匹配的粗匹配放过来, karto 的扫描匹配的计算量很小, 当做粗匹配很不错.

地图

有时前端部分生成的地图出现了叠图, 而前端建的地图在后端是不会被修改的, 后端优化只会优化节点位姿与子图位姿.

同时 `cartographer_ros` 最终生成的地图是将所有地图叠加起来的, 就会导致这个叠图始终都存在, 又或者是后边的地图的空白部分将前边的地图的边给覆盖住了, 导致墙的黑边消失了.

后端优化会将节点与子图的位姿进行优化, 但是不会改动地图, 所以可以在最终生成地图的时候使用后端优化后的节点重新生成一次地图, 这样生成的地图的效果会比前端地图的叠加要好很多.

这块的实现可以参考一下我写的实时生成三维点云地图部分的代码.

更极致的修改

后端优化后的节点与子图位姿是不会对前端产生影响的, 这块可以进行优化一下, 就是前端匹配的时候, 不再使用前端生成的地图进行匹配, 而是使用后端生成的地图进行匹配, 这样就可以将后端优化后的效果带给前端. 但是这要对代码进行大改, 比较费劲.

15.3.3 降低计算量与内存

- 体素滤波与自适应体素滤波的计算量(不是很大)
- 后端进行子图间约束时的计算量很大
- 分支定界算法的计算量很大
- 降低内存, 内存的占用基本就是多分辨率地图这, 每个子图的多分辨率地图都进行保存是否有必要

15.3.4 纯定位的改进建议

目前 `cartographer` 的纯定位和正常的建图是一样的, 只是仅保存 3 个子图, 依然要进行后端优化.

这就导致了几个问题:

第一个: 前端的扫描匹配, 是当前的雷达点云与当前轨迹的地图进行匹配, 而不是和之前的地图进行匹配, 这就导致了定位时机器人当前的点云与之前的地图不一定能匹配很好, 就是因为当前的点云是匹配当前轨迹的地图的, 不是与之前的地图进行匹配.

第二个: 纯定位其实就是建图, 所以依然会进行回环检测与后端优化, 而后端优化的计算在定位这是没有必要的, 带来了额外的计算量.

第三个: 纯定位依然会进行回环检测, 回环检测有可能导致机器人的位姿发生跳变.

改进思路

将纯定位模式与建图拆分开, 改成读取之前轨迹的地图进行匹配.

新的轨迹只进行位姿预测, 拿到预测后的位姿与之前轨迹的地图进行匹配, 新的轨迹不再进行地图的生成与保存. 同时将整个后端的功能去掉.

去掉了后端优化之后, 会导致没有重定位功能, 这时候可以将 `cartographer` 的回环检测(子图间约束的计算)部分单独拿出来, 做成一个重定位功能. 通过服务来调用这个重定位功能, 根据当前点云确定机器人在之前地图的位姿.

这样才是一个比较好的定位功能的思路.

15.3.5 去 ros 的参考思路

有一些公司不用 ros, 所以就要进行去 ros 的开发.

咱讲过数据是怎么通过 `cartographer_ros` 传到 `cartographer` 里去的, 只要仿照着 `cartographer_ros` 里的操作, 获取到传感器数据, 将数据转到 `tracking_frame` 坐标系下并进行格式转换, 再传入到 `cartographer` 里就行了.

`cartographer_ros` 里使用 ros 的地方比较少, 只有在 `node.cc`, `sensor_bridge` 等几个类中进行使用, 只需要改这个类接受数据的方式以及将 ros 相关的格式修改一下就行了.

代码框架梳理

cartographer_ros

- CMakeLists.txt文件
- gflags与glog简介
- main函数讲解
- 配置参数的加载
- Node类
 - 开始轨迹
 - 数据采样器
 - 订阅话题与注册回调函数
 - 传感器数据的走向与传递
- MapBuilderBridge类
- SensorBridge类
 - Rigid类 - 三维刚体的坐标变换
 - TfBridge类
 - msg_conversion.cc 数据类型的转换
 - 传感器数据的传递

数据的分发处理-生产者消费者模式

- CollatedTrajectoryBuilder
- Collator
- OrderedMultiQueue
- BlockingQueue 阻塞队列(缓冲区)

GlobalTrajectoryBuilder

前端 LocalTrajectoryBuilder2D::AddRangeData

- 雷达数据的处理
 - 多个激光雷达数据的时间同步与融合
 - 激光雷达数据的运动畸变的校正
 - 点云的坐标变换与z轴的过滤
 - 体素滤波与之后的处理
- 位姿预测 PoseExtrapolator
- 概率栅格地图
 - ActiveSubmaps2D Submap2D
 - MapLimits Grid2D ProbabilityGrid
- 扫描匹配
 - 相关性扫描匹配 RealTimeCorrelativeScanMatcher2D
 - ceres扫描匹配 CeresScanMatcher2D
- 将雷达写入栅格地图 ProbabilityGridRangeDataInserter2D

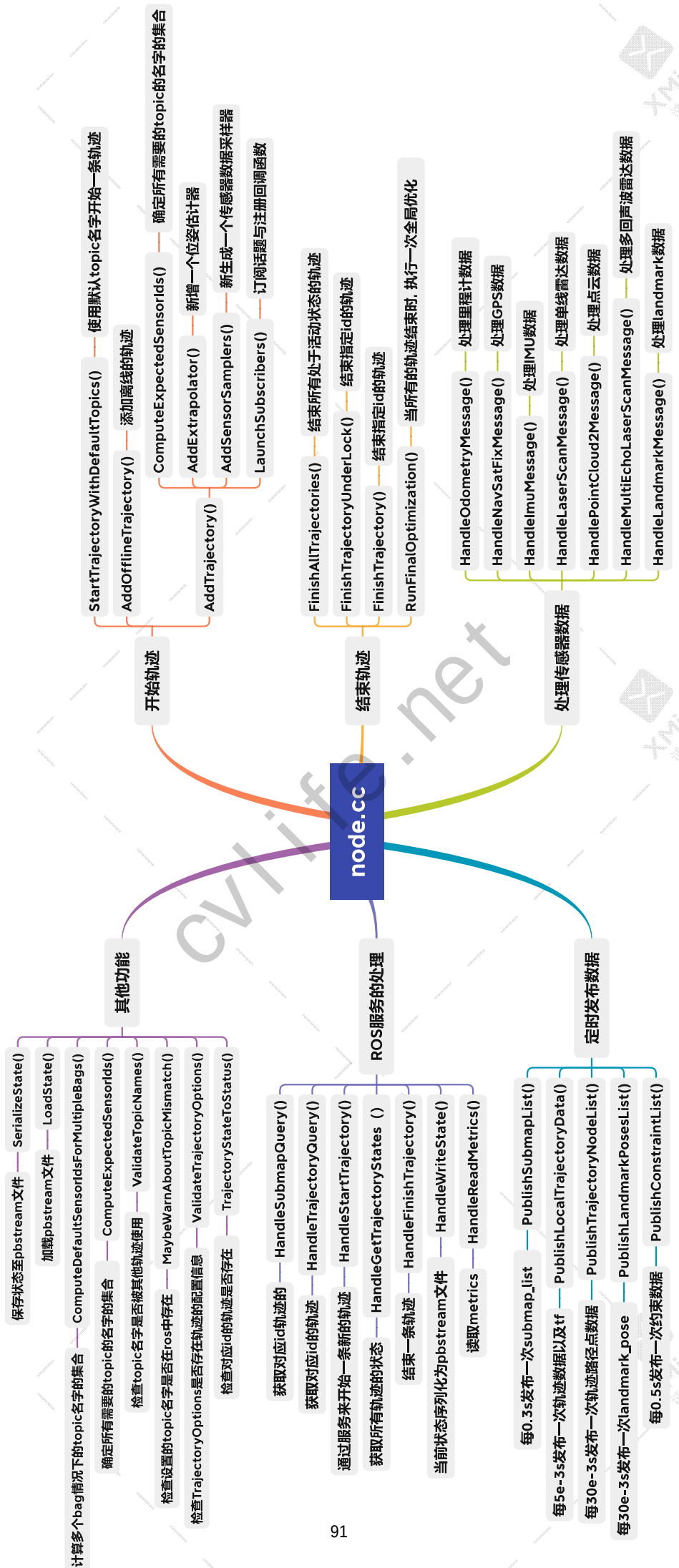
后端

线程池与任务调度

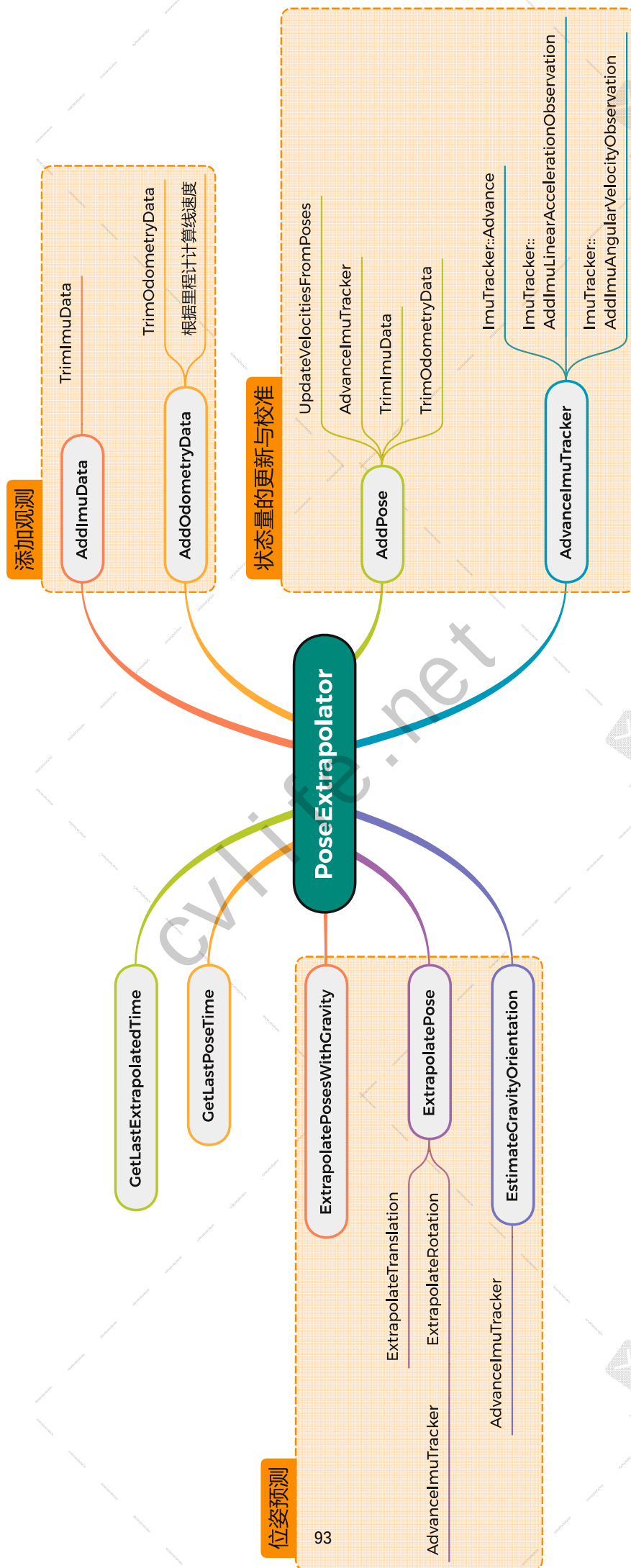
- Task
- ThreadPool

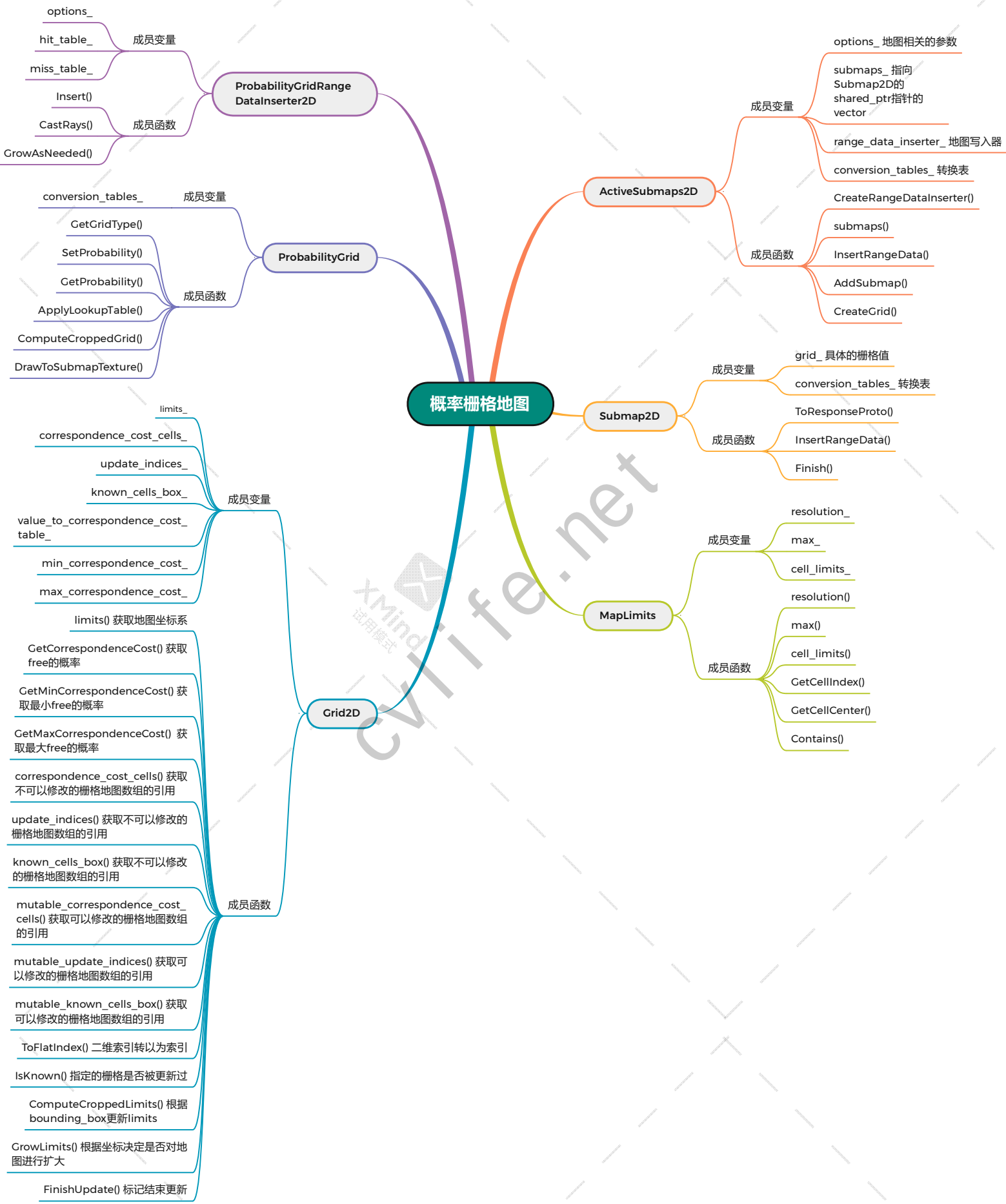
位姿图 PoseGraph2D

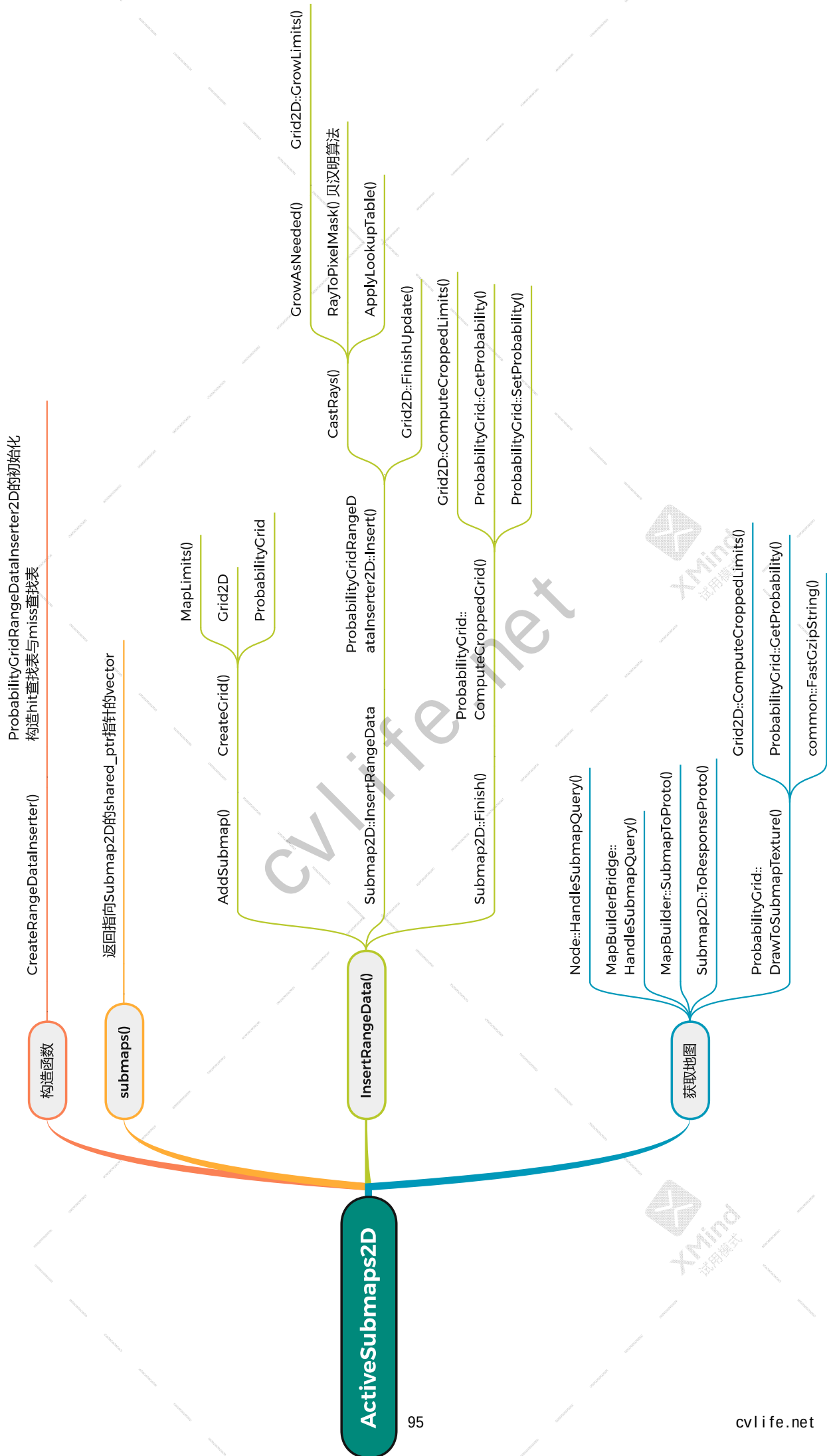
- 向位姿图中添加节点
- 计算节点的子图内约束与子图间约束(回环检测) ComputeConstraintsForNode
- 回环检测(计算子图间约束) ConstraintBuilder2D
- 多分辨率地图 PrecomputationGridStack2D
- 基于分支定界算法的粗匹配 FastCorrelativeScanMatcher2D
- 优化问题的构建与求解 OptimizationProblem2D

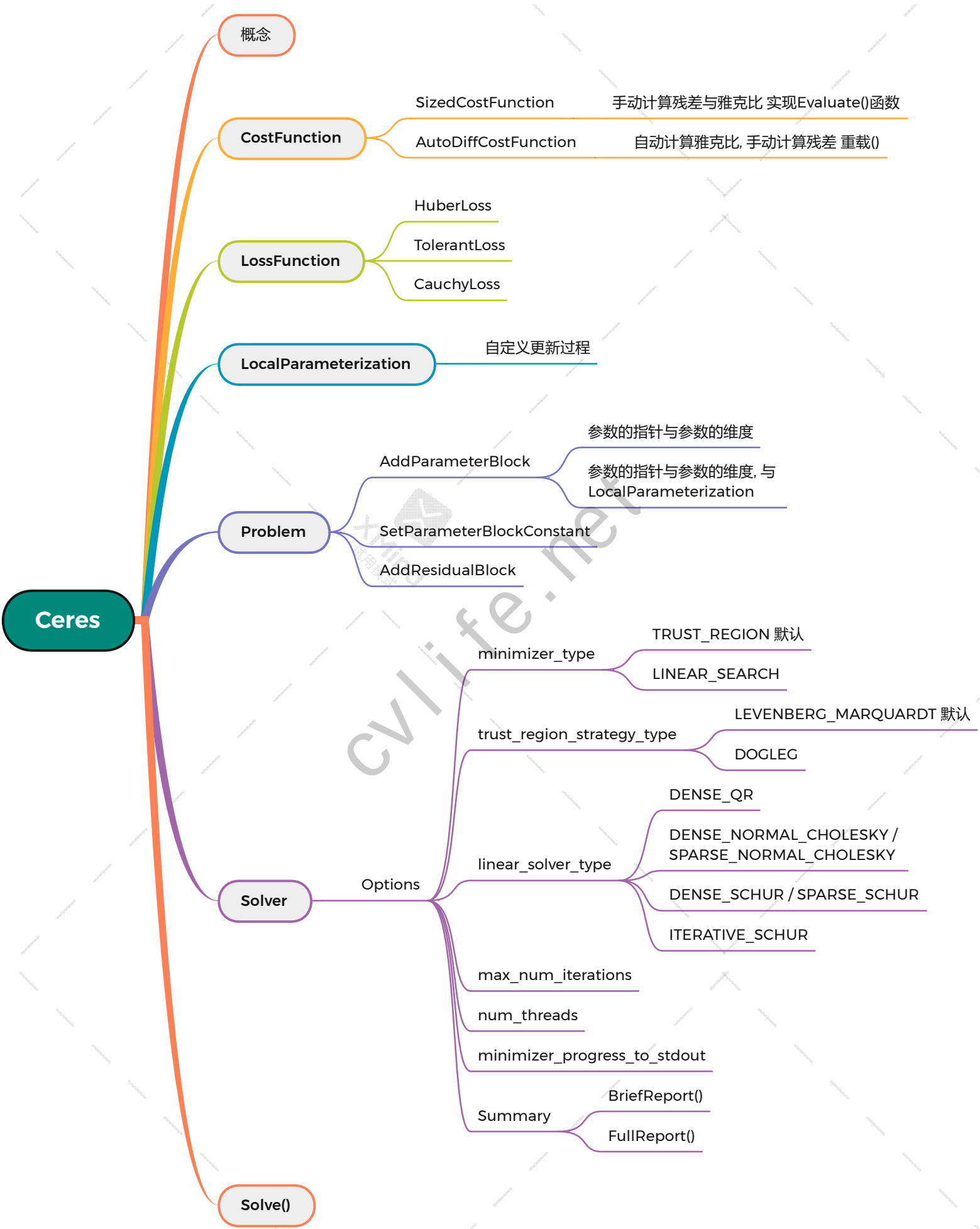


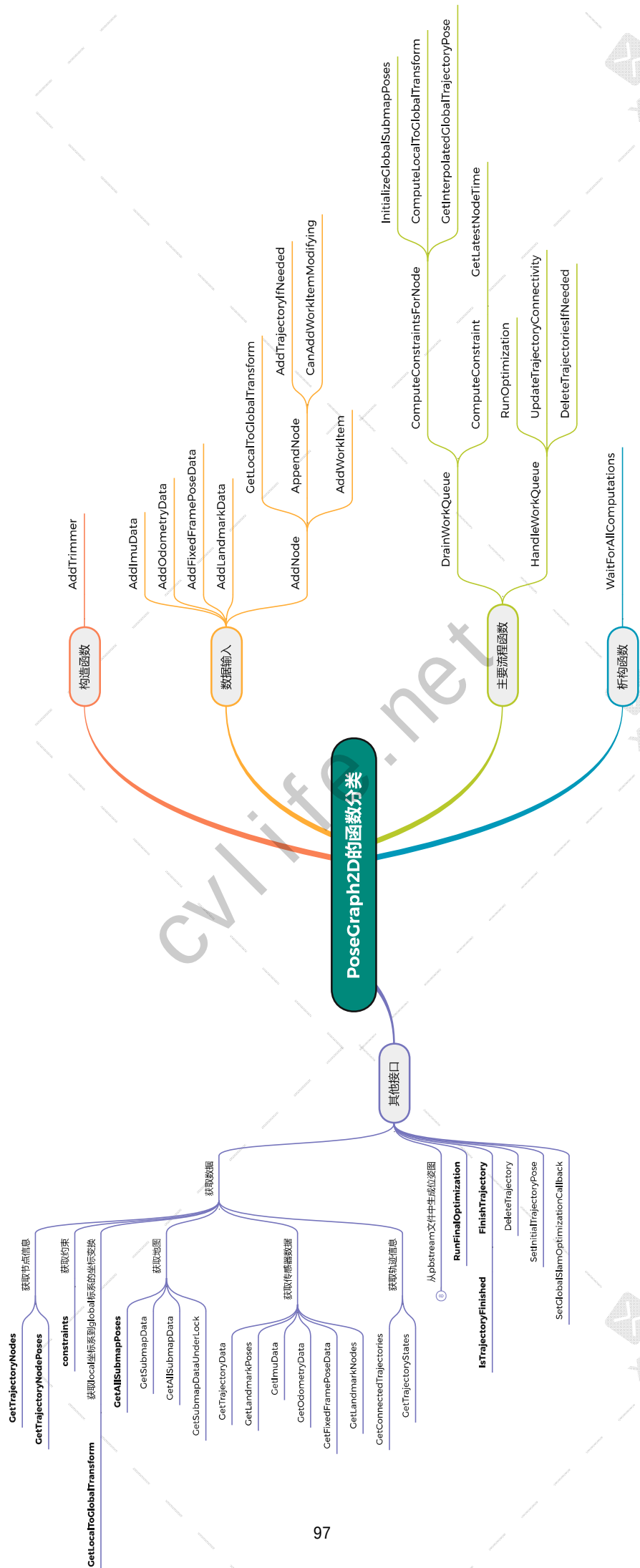


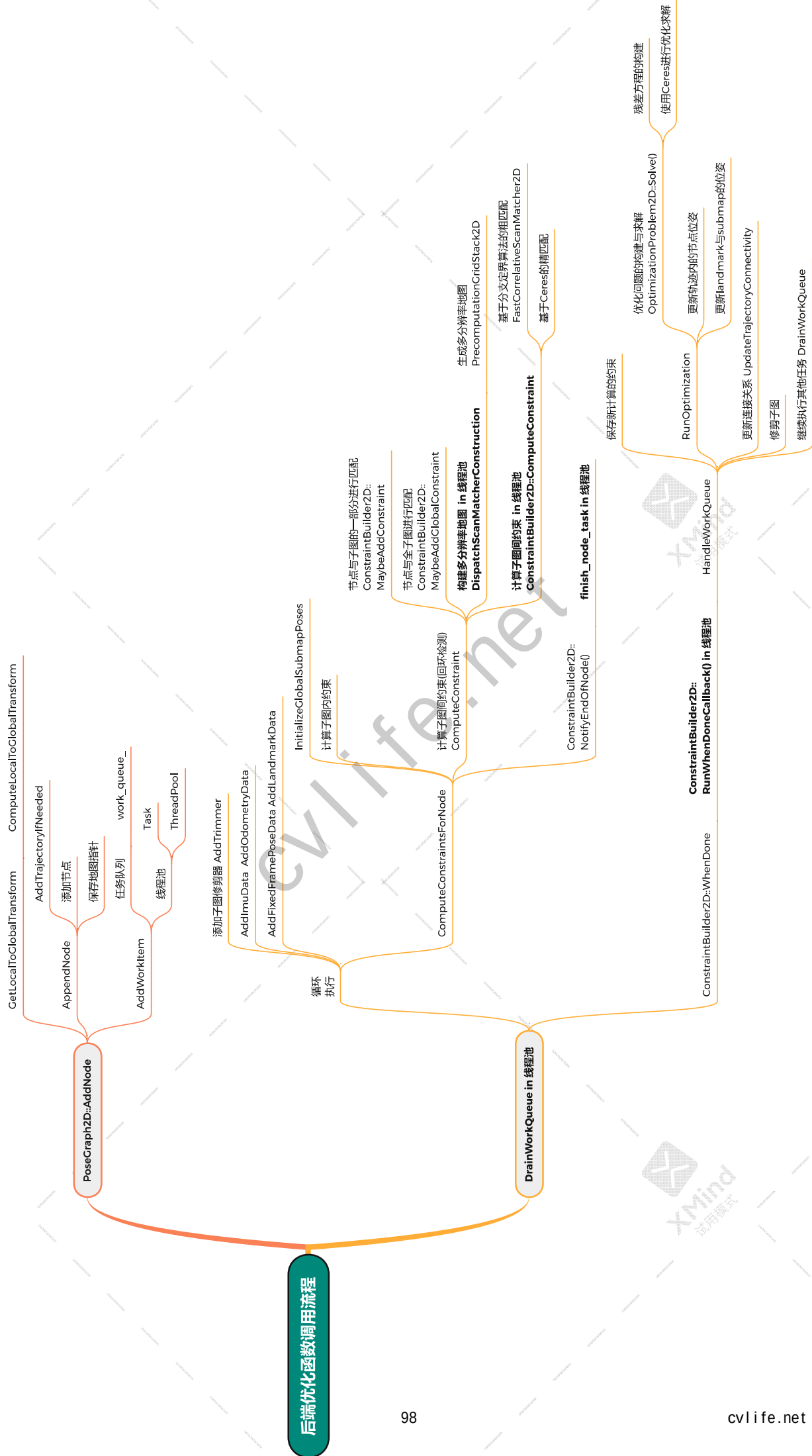












Task::SetWorkItem

2种情况

ThreadPool::Schedule

Task::AddDependentTask

ThreadPoolInterface::SetThreadPool

Task::OnDependencyCompleted()

Task::SetThreadPool

ThreadPool::
NotifyDependenciesCompleted

线程池

线程1

线程2

线程3

线程4

Task::Execute

Task::Execute

Task::Execute

Task::Execute